



Diploma Thesis

**Aggregation and Pre-Processing of
Network and Process Data for Moni-
toring Purposes in ROS2 Applications**

TU Bergakademie Freiberg

submitted to the

Faculty of Mathematics and Computer Science

to attain the academic degree of

Diplom-Ingenieur

submitted by

Georg Muck

born on 14.06.2000 in Plauen

Supervisors: Prof. Dr. Sebastian Zug
Dr Georg Jäger

Examiners: Prof. Dr. Sebastian Zug
Dr Georg Jäger

Course: Institut für Informatik

Date of submission: 07.07.2025

Acknowledgments

I would like to express my sincere gratitude to my supervisors, Prof. Dr. Sebastian Zug and Dr. Georg Jäger, for their continuous support, expert guidance, and valuable feedback throughout the process of writing this thesis. This gratitude is not limited to this thesis, but also extends to the trust that has been placed in me over the past years. I have gained a great deal of knowledge and experience through you. Thank you.

Furthermore, I am also grateful to Johannes Kohl, who wrote his diploma thesis on a related topic during the same period, for the pleasant communication and many hours of technical discussion together.

In addition to the professional support, I would also like to thank my family, who supported me throughout the course of my studies.

Last but not least, I would like to thank my friends. Thanks to you, my studies were as enjoyable, exciting, and encouraging as they were. Special thanks go to Lukas, Robias, Pascal, Benjamin, and Lydia.

Abstract

The increasing complexity and dynamic nature of modern robotic systems presents significant challenges regarding their analysis and Fault Detection. Manual model-based approaches are often too cost-, time-, and labour-intensive and thus do not match the concept of rapid prototyping and its swift, iterative development cycles. Moreover, the rapid changes in system design foster a rapid change in expert knowledge, which leads to inconsistency and thus degenerated system models, especially in systems that are maintained by multiple teams. This is compounded by the difficulty of creating dynamic system models that represent the actual runtime behaviour of a system, rather than simulations.

This thesis addresses these challenges by introducing a Digital Shadow – a passive, data-driven digital representation of a system – that generates system specific models at runtime. To this end, multiple concepts of ROS2 are explored and abstracted by deriving a mathematical meta-model that enables the generation of dynamic system models that are capable of integrating platform-specific information. To enable faster feedback cycles and accelerate debugging in order to reduce development time and costs while increasing system reliability, the meta-model integrates several mechanisms to facilitate Fault Detection and Diagnosis. These mechanisms aggregate the system to create holistic perspectives that provide a broad view of the system while keeping information fine-grained.

A Digital Shadow is employed in order to generate a digital representation of a system in question. It aims to reduce necessitated expert knowledge by employing a non-intrusive approach that leverages existing system data and runtime information. The Digital Shadow is designed to be lightweight in order to minimise its impact on the performance of the underlying system. Nevertheless, it captures both structural and continuous data to enable a comprehensive analysis and monitoring of the system's behaviour. To enable the subsequent analysis of the system, it additionally provides an interface for further applications that analyse the system in order to, e.g., perform model-based Fault Detection and Diagnosis.

The approach is implemented in a proof-of-concept, mainly leveraging tracepoints in the ROS2 source code to achieve the objectives of this thesis. The feasibility of the mathematical meta-model is evaluated in a case study. Additionally, multiple experiments, which explore the performance of the proof-of-concept, are executed.

The results indicate that the proposed approach enables the dynamic and holistic modelling of ROS2 systems at runtime within defined constraints. However, within these constraints, the ap-

proach is capable of automatically generating ROS2 system specific models while minimising the required expert knowledge and negating the need for manipulating the software stack of the respective ROS2 system. Thereby, the overhead that is typically associated with manual modelling is reduced. Moreover, the Digital Shadow supports online Fault Detection and Diagnosis by facilitating system analysis, aiding the development of more resilient, reliable, and maintainable robotic applications.

Contents

1. Motivation	1
2. Model Based Systems Engineering and the Characteristics of Digital Shadows	5
2.1. Modelling Complex Systems through Systems Engineering	5
2.1.1. Document-Centric Systems Engineering	5
2.1.2. Model Based Systems Engineering	6
2.1.3. Systems Modeling Language	6
2.2. Virtual Representation of Systems	7
3. Digital Shadows versus ROS2 Systems: Verification and Analysis	9
3.1. Digital Twins and Digital Shadows in Previous Work	9
3.2. Creation of a Digital Shadow	11
3.2.1. Conceptual Model of a Digital Shadow	11
3.2.2. Workflow for Creating a Shadow Instance	12
3.3. Runtime Verification in ROS2 Systems	13
3.4. Analysis of ROS2 Systems	14
3.5. Modelling ROS2 Systems	15
4. Modelling ROS2 using Model Based Systems Engineering and SysML	17
4.1. The Concept of Nodes in ROS2	18
4.2. Communication Mechanisms of ROS2	19
4.2.1. The Concept of Topics in ROS2	19
4.2.2. The Concept of Services in ROS2	20
4.2.3. The Concept of Actions in ROS2	23
4.3. The Concept of Containers in ROS2	23
4.4. The Concept of Timers in ROS2	24
4.5. The Concept of Executors in ROS2	24
4.6. Considering Operating System Processes within ROS2 Systems	25

5. Deriving a Mathematical Meta-Model to represent the Fundamentals of ROS2 Systems	27
5.1. Establishing a Directed Graph for Modelling ROS2 Systems	28
5.1.1. Creating the Fundamental ROS2 Graph	28
5.1.2. Enabling the Evolution of the ROS2 Graph	31
5.1.3. Representing the Communication Structure of ROS2 Systems	32
5.2. Aggregating ROS2 Systems to Support Fault Detection and Diagnosis	35
5.2.1. Creating Relations to Aggregate Member- and Edge-Specific Attributes	35
5.2.2. Creating Organisational Trees	38
5.2.3. Creating Subtrees and Adding Binary Operations	42
5.3. Discussing the Mathematical Meta-Model: Capabilities and Challenges	44
6. Employing the Mathematical Meta-Model to create a ROS2 System Digital Shadow in Support of Fault Detection and Diagnosis	47
6.1. Requirements for a Shadow Caster Targeting ROS2 Systems	47
6.2. Refining the Conceptual Model for Digital Shadows to ROS2 Systems	48
6.3. Logical Model to Generate Shadow Instances	50
6.3.1. Representing Shadow Instances	50
6.3.2. Creating the Shadow Caster	51
6.3.3. Exemplary Interactions with the Shadow Caster	54
6.3.4. Discussing the Logical Model for the Shadow Caster	56
7. Evaluation	59
7.1. Employed Tools and Implementation	60
7.2. Feasibility of the Mathematical Meta-Model	61
7.2.1. Generating an Exemplary ROS2 Graph	62
7.2.2. Aggregating ROS2 Systems	65
7.2.3. Continuous Values in the ROS2 Digital Shadow	66
7.2.4. Integrating System Specific Information into the Model	67
7.3. Challenges while Generating System Specific Models	67
7.3.1. Reassigning Communication Edges across Multiple Restarts	67
7.3.2. Differentiating between Multiple Message Types of a Single Topic	68
7.3.3. Generating the Process-Driven, Edge-Induced Tree	68
7.4. Defining Constraints on ROS2 Systems to Ensure Consistency and Correctness of the Generated Model	69
7.5. Generating ROS2 System Models Using the Shadow Caster: Performance Tests	71
7.5.1. The Test-Setup	71
7.5.2. Latencies under Several Conditions	72
7.5.3. CPU Utilisation under Several Conditions	73
7.5.4. RAM Utilisation under Several Conditions	74

7.6. Requesting Information of the ROS2 System	76
7.6.1. Analysing the Event Propagation Latency	76
7.6.2. Analysing the Overhead of Information Requests	77
7.6.3. The Performance of the Shadow Caster	78
7.7. Applying the Created Concept to a Real-World Scenario	79
7.8. Summary	82
8. Conclusion and Future Work	83
A. Further Tree Structures	92
B. Further Setup Information	94
C. Performance Test: Detailed Results	95
D. Real World Scenario	100

List of Figures

2.1. Data Flow of Digital Model, Digital Shadow, and Digital Twin	8
3.1. Conceptual Model of a Digital Shadow	11
3.2. Workflow of a Digital Shadow	12
4.1. Elements of ROS2 and their Respective Relations	17
4.2. The ROS2 Node and Adjacent Elements of ROS2	18
4.3. Namespace Hierarchy in ROS2 Systems	19
4.4. Communication via Topics in ROS2	20
4.5. Calculating a Topic's Forwarding Rate	20
4.6. Communication via Services in ROS2	21
4.7. Communication via Actions in ROS2	22
4.8. Grouping Nodes by Containers	23
4.9. Executors within ROS2	25
4.10. Process Hierarchy in ROS2 Systems	25
5.1. Legend to represent Directed Graph	27
5.2. Adding and Disabling Entities from Exemplary Graph	32
5.3. Representing the Concept of ROS2 Topics with the Meta-Model	33
5.4. Representing the Concept of ROS2 Services with the Meta-Model	33
5.5. Representing the Concept of ROS2 Actions with the Meta-Model	34
5.6. Timers in ROS2 Communication Graph	34
5.7. Proof to Consider Edge Direction in order to Prevent Ambiguity	37
5.8. Relation to Accumulate Adjacent Communication Edges	37
5.9. Totaling Edge and Member-related attribute values	38
5.10. Exemplary Namespace-Driven Tree	39
5.11. Exemplary and Joined Executor-Driven Tree	40
5.12. Exemplary Process-Driven Tree	40
5.13. Exemplary Single Step of Totaling	41
5.14. Exemplary Top-Down Aggregation	42
5.15. Exemplary Binary Operations	44
5.16. Aggregating Values by Callback Groups	45

6.1. Conceptual Model for Digital Shadow Targeting ROS2 Systems	48
6.2. Representing the Shadow Instance with three Databases	50
6.3. Logical Model of Shadow Caster Targeting ROS2 Systems	51
6.4. Extending the Logical Model to enable the Integration of Domain Knowledge . .	54
7.1. Partially Updated Legend	60
7.2. Instantiating Active Members that Represent Nodes	62
7.3. Adding Timers to the Generated ROS2 Graph	62
7.4. Adding Sending Edges that Represent Server-Client Communication	63
7.5. Extending the Example by Topics Represented as Passive Members	63
7.6. Extending the ROS2 Graph by its Respective Process-Driven Tree	64
7.7. Extending the ROS2 Graph by its Respective Namespace-Driven Tree	65
7.8. Exemplary Publishing Rate of all Instances in the Generated ROS2 System Model	66
7.9. Challenge with Reassigning Communication Edges across Multiple Restarts . .	68
7.10. Challenge of Differentiating Multiple Message Types on a Single Topic	69
7.11. Challenge while Constructing the Process-Driven, Edge-Induced Tree	69
7.12. Latency Overhead Originating from Differing Test Configurations	73
7.13. CPU Utilisation of the Shadow Implementation Compared to the ROS2 System .	74
7.14. RAM Utilisation of the Shadow Implementation Compared to the ROS2 System .	75
7.15. Latency from Initialisation to Requesting Client	77
7.16. Influence of Requests on the CPU Utilisation of the Shadow Implementation . .	78
7.17. Real-World Scenario: Generated ROS2 Graph (Stage 1)	80
7.18. Real-World Scenario: Generated ROS2 Graph (Stage 2)	81
A.1. Exemplary and Joined Container-Driven Tree	92
A.2. Exemplary Process-Driven Tree	93
C.1. Mann-Whitney U Test: Significance Results for Measured Latencies	95
C.2. CPU Utilisation of Shadow Caster and Shadow Instance	96
C.3. Detailed RAM Utilisation of Shadow Caster, Shadow Instance, and ROS2 system	97
C.4. Relative CPU Utilisation of Shadow Implementation	98
C.5. Relative RAM Utilisation of Shadow Caster and Shadow Instance	98
C.6. CPU Utilisation for Differing Tracer Implementations	99
C.7. Influence of Requests on the RAM Utilisation of the Shadow Implementation . .	99
D.1. Real-World Scenario: Generated ROS2 Graph (Stage 3)	101

List of Tables

2.1. Several SysML Diagrams and their Purpose	7
6.1. DataTracers to Construct Structural ROS2 Graph	49
6.2. DataTracers to Capture Continuous Information	49
7.1. Specifications of the Testsystem	72
7.2. Sensors that are Assembled at rosee for the Real-World Scenario	79
7.3. Specifications of rosee's Host Computer	79
B.1. Tracepoints Employed for the Proof-of-Concept	94
D.1. List of Active Nodes in Stage 3 of the Real-World Scenario	100

List of Definitions

- 5.1.1. Directed Graph to Represent ROS2 Systems 29
- 5.1.2. Attribute Function to Assign Properties to Graph Entities 29
- 5.1.3. Version the Directed Graph to Enable Evolving Nature 31
- 5.1.4. ROS2 Communication 32
- 5.1.5. The ROS2 Communication Graph 33
- 5.2.1. Creating Relations between communication Members and Edges 36
- 5.2.2. Total Adjacent Edge-Related Attribute Values 36
- 5.2.3. Rooted Tree for Various Perspectives 38
- 5.2.4. Top-Down Approach to Analyse Directed, Rooted Tree 41
- 5.2.5. Extracting Subtrees for a Fine-Grained Perspective 43
- 5.2.6. Binary Operations on Trees 43
- 5.2.7. Accumulation of Binary Aggregated Subsets 43

List of Symbols

- $\mathbb{T}$ Temporal domain to represent a time series.
- $\Psi_{i,in}$ Recursively accumulated attributes function for Tree T_i based on Φ_{in} .
- $\Psi_{i,out}$ Recursively accumulated attributes function for Tree T_i based on Φ_{out} .
- ρ_{in} The sets of relations R , reaching to incoming, communication Edges.
- ρ_{out} The sets of relations R , reaching to outgoing, communication Edges.
- θ The attributes function to represent time-varying attributes of Members and communication Edges.
- Θ_{in} The attributes function encompassing accumulated Member and incoming Edge attribute values.
- Θ_{out} The attributes function encompassing accumulated Member and outgoing Edge attribute values.
- Υ_{in} Binary aggregated attributes function based on Φ_{in} .
- Υ_{out} Binary aggregated attributes function based on Φ_{out} .
- E The Edges representing relations between Members within a ROS2 system.
- E_0 The communication Edges representing communication and Timers within a ROS2 system.
- E_{∇} Publishing Edges, representing communication from an active to a passive Member.
- $E_{\uparrow}$ Sending Edges, representing communication between two active Members.
- $E_{\wedge}$ Subscribing Edges, representing communication from a passive to an active Member
- $E_{\odot}$ Sending Edges, representing virtual self-communication of an active Member.
- E_i The Edges representing additional relations between Members within a ROS2 system.
- G The directed Graph representing a ROS2 system.
- G_0 The directed Graph representing the communication structure of a ROS2 system.

M	The Members of a ROS2 system.
M_0	The communication Members representing the Members of the ROS2 communication Graph.
M_A	The active Members of a ROS2 system.
M_i	The Members within a specific directed, rooted Tree T_i .
M_P	The passive Members of a ROS2 system.
R	The relation reaching from a Member M_0 to a communication Edge E_0 .
T_i	Directed, rooted Tree that is edge-induced by E_i .
$T_{i,m}$	Directed, rooted Subtree of T_i , rooted at $m \in M_i$.

List of Abbreviations

CPU	Central Processing Unit
DDS	Data Distribution Service
FDD	Fault Detection and Diagnosis
FMEA	Failure Mode and Effects Analysis
INCOSE	International Council on Systems Engineering
LTTng	Linux Trace Toolkit: next generation
MBSE	Model Based Systems Engineering
OMG	Object Management Group
RAM	Random Access Memory
ROS	Robot Operating System
SE	Systems Engineering
STREAM	Sensor and Telemetry: ROS2 Event-Based Aggregation and Monitoring
SysML	Systems Modeling Language
UDP	User Datagram Protocol
UID	Unique Identifier
UML	Unified Modeling Language
YARP	Yet another Robot Platform

1. Motivation

Robots are exposed to a wide range of environmental influences and are taking on increasingly far-reaching and complex tasks [1]. At the same time, their integration into the human environment is expanding, which increases both system complexity and development costs. As a result, there is a growing demand for early-stage evaluation of new approaches through concepts such as rapid prototyping [2, p. 5][3, p. 161]. Rapid prototyping is an iterative process that has been employed for the development of robotic systems throughout the past 25 years [4][5][6][7]. It is characterised by continuous changes in system behaviour, which are accompanied by a continuous change in expert knowledge about the robotic system. Thus, managing this knowledge poses a significant challenge.

This challenge is particularly evident in modern robotic development frameworks, where modularity, flexibility, and distributed architectures are key elements. A prominent example of such a framework is the Robot Operating System (ROS), which is considered the „de facto standard for robot application development“ [8, p. 2545] and is widely used, particularly within the open-source community. However, since ROS1 had drawbacks such as limited support for multi-robot systems, no built-in security features and reliance on a centralised master node, it was not well-suited for safety-critical or industrial applications. These limitations ultimately led to the development of ROS2, which became the new de facto standard [9, p. 80]. Nevertheless, the distributed nature of ROS2 introduces further, significant challenges in understanding and analysing system behaviour, especially in dynamic and complex environments. For instance, when multiple Nodes communicate asynchronously, they can cause timing-related issues, message loss, or unintended interactions between perception, planning, and control components that are difficult to trace or reproduce.

Virtual representations in the form of models provide an essential means to address these challenges by offering an abstract and structured view of the system under development [10]. This modelling approach is a central aspect of Model-Based Systems Engineering (MBSE), which promotes the use of formal models to support system development and validation [10, p. 105], as developers are not required to directly interact with the physical system. In particular, such models facilitate the identification, localisation, and analysis of faults. When combined with automated mechanisms for Fault Detection and Diagnosis, they enable faster feedback cycles and more efficient debugging, ultimately accelerating development, reducing costs, and increasing system reliability [11, p. 1]. However, creating and maintaining such models for ROS2-based systems remains a significant challenge. Manual modelling is time-consuming and therefore

particularly unsuitable for the methodology of rapid prototyping, whose dynamic and iterative nature requires frequent re-modelling, inducing a substantial overhead. Automated generation of the model would reduce the generated overhead.

Therefore, this thesis proposes an approach to model ROS2 systems at runtime to support Fault Detection and Diagnosis by facilitating their analysis. To this end, a meta-model is introduced to represent fundamental ROS2 systems. This meta-model serves as a blueprint for a Digital Shadow – a passive, data-driven digital representation of a system – that aims at reducing the required expert knowledge and facilitating system analysis, in the domain of Fault Detection and Diagnosis, at runtime. It thus represents the first part of STREAM (Sensor and Telemetry: ROS2 Event-Based Aggregation and Monitoring), a pipeline for model-based Fault Detection and Diagnosis for ROS2 systems. Kohl [12] employs the generated ROS2 system model to perform an Fault Detection and Diagnosis at runtime.

In order to elaborate on the objective of generating system specific models while ensuring broad applicability across various ROS2 systems, multiple key objectives, which shape the key contributions and design principles of this thesis, are listed:

Generic Meta-Model for ROS2 Systems

The meta-model has to be enabled to represent the fundamental concepts of ROS2 systems to support a wide range of ROS2 platforms. It is also required to enable the integration of platform-specific information where needed.

Holistic Perspectives onto ROS2 Systems

A comprehensive representation of the running ROS2 system, including perspectives beyond ROS2 (e.g., operating system aspects), is necessitated to facilitate Fault Detection and Diagnosis. This also involves the creation of various perspectives on ROS2 systems.

Runtime Representation

The generated ROS2 system specific model has to be dynamic, meaning that it represents the dynamic nature and the runtime behaviour of a ROS2 system. To this end, the meta-model must enable this characteristics.

The introduced meta-model, creating a dynamic and holistic view onto a wide range of ROS2 systems while enabling the integration of platform-specific information, forms the main contribution of this thesis.

Lightweight Digital Shadow

The Digital Shadow has to remain lightweight to minimise overhead and avoid influencing the behaviour of a ROS2 system in question. Therefore, it is built upon two key design decisions:

- Event-based data handling, where only requested and essential data is transmitted to the client.
- Monitoring of continuous data (e.g. utilisation metrics) is performed only on demand.

Non-Intrusiveness

The Digital Shadow is required to operate without requiring changes to the existing codebase of ROS2 systems in order to keep the software stack clean.

Minimising Expert Knowledge

The non-intrusive approach further aims at minimising the required expert knowledge, meaning that the Digital Shadow should be enabled to operate with a bare minimum of required expert knowledge about the monitored ROS2 system.

The structure of this thesis is organised as follows:

Chapter 2 introduces the fundamental principles of Model Based Systems Engineering and the characteristics of Digital Shadows. Subsequently, Chapter 3 analyses previous work on Digital Shadows, especially in the domain of robotic systems with a focus on ROS2. Furthermore, runtime verification, analysis, and modelling of ROS2 systems are also considered as they mirror related topics. Chapter 4 then delves into the modelling of ROS2 systems, covering various concepts ROS2 encompasses, thus marking the commencement of the author's contribution. Chapter 5 introduces the main contribution of this thesis by presenting the meta-model that serves as a representation of ROS2 systems and explores how it supports Fault Detection and Diagnosis. Employing the introduced meta-model as a blueprint for fundamental ROS2 systems, Chapter 6 creates a Digital Shadow that contributes to the objectives of this thesis. Finally, Chapter 7 evaluates the proposed approach, followed by Chapter 8, which concludes the thesis and outlines directions for future research.

2. Model Based Systems Engineering and the Characteristics of Digital Shadows

For deriving the meta-model, the fundamentals of ROS2 systems are to be modelled first. Therefore, the author utilises Model Based Systems Engineering and the Systems Modeling Language (SysML) described in Section 2.1. In order to provide a deeper understanding of Digital Shadows, Section 2.2 introduces its concept accompanied by the related concepts of Digital Models and Digital Twins, as the three terms are often used synonymously and share overlapping characteristics.

2.1. Modelling Complex Systems through Systems Engineering

Systems Engineering provides a structured approach to the development and representation of complex systems, aiming to ensure that all technical and non-technical aspects are considered. As time has passed, various methodologies have emerged within this field. This chapter presents two fundamental approaches, the Document-Centric approach in Section 2.1.1 and the Model Based approach in Section 2.1.2. The subsequent Section 2.1.3 focuses on the Systems Modeling Language (SysML) as an emerging approach for modelling systems.

2.1.1. Document-Centric Systems Engineering

The Document-Centric approach is characterised by the creation of textual specifications and design documents. These documents often include written descriptions, graphics created with drawing tools, and other communication materials such as emails. Thus, maintaining consistency and traceability becomes increasingly complex and time-consuming, as the model rapidly attains a size that is impractical for humans to manage [13, p. 234]. Furthermore, this approach becomes challenging when dealing with complex systems, as it is often decentralised, resulting in an increased risk of overlooking key interfaces or critical information [14, p. 173]. Furthermore, the validation of information contained within these documents may be challenging, which can compromise the overall reliability and coherence of the system design. [15, p. 15]

Therefore, the Document-Centric approach to Systems Engineering is not conducive to the creation of a Digital Shadow, as the information in it is static, difficult to automate, and may be

redundant. In contrast, Model Based Systems Engineering facilitates the establishment of a consistent, digital representation of a system, which is essential for a precise Digital Shadow.

2.1.2. Model Based Systems Engineering

Model Based Systems Engineering (MBSE) utilises system models to represent the design and functionality of complex systems. Despite its long-term cost-effectiveness, the adoption of MBSE is often limited by high initial costs and significant effort required to implement it, as well as a lack of knowledge about its potential benefits [16, p. 2341][17, p. 7ff]. However, this approach allows for comprehensive management of a product from initial design through to its final stages [15, p. 17]. One of the key advantages of MBSE is the establishment of a single source of truth, enabling a clear and unified understanding of the system. Furthermore, MBSE aids in capturing and articulating requirements and domain knowledge, ensuring that all stakeholders can grasp the essential, latest details [10, p. 102ff]. By providing different levels of abstraction, MBSE facilitates the identification of optimal solutions while enhancing the detection of potential risks or faults throughout the system's lifecycle and thus supports the prevention of failures [18, p. 103]. The identification and therefore the possibility for elimination of faults is further facilitated by the ability to analyse the system from multiple perspectives.

In summary, MBSE fosters effective system design by facilitating the organisation, examination, and modification of information related to large systems, while providing a single source of truth that additionally supports the creation of a reliable and consistent Digital Shadow.

The following section describes the widely used Systems Modeling Language that will be utilised to model different aspects of ROS2 systems.

2.1.3. Systems Modeling Language

The Object Management Group (OMG) and the International Council on Systems Engineering (INCOSE) developed the Systems Modeling Language (SysML), which is based on the Unified Modeling Language (UML) [19, p. 4] and is „considered as the next de facto modelling language for SE“ [10, p. 104]. It facilitates MBSE by providing multiple diagrams that reflect several aspects of the system in question [10, p. 104]. These aspects correspond to particular perspectives and thus enable a holistic view of a system.

The diagrams are categorised into three groups, namely A) Behavior Diagrams: containing Activity Diagrams, Sequence Diagrams, State Machine Diagrams, and Use Case Diagrams; B) Requirement Diagrams; and C) Structure Diagrams: containing Block Definition Diagrams, Internal Block Diagrams, Parameter Diagrams, and Package Diagrams [20, p. 96]. Table 2.1 briefly describes the five diagrams that are employed throughout this thesis.

Table 2.1.: Several SysML Diagrams and their Purpose

Diagram	Purpose
Block Definition (BDD)	Represents system elements, such as blocks and value types, along with their relationships and connections. [20, p. 97]
Package (PKG)	Reveals the organisation of the system. [19, p. 4]
Parameter (PAR)	Defines a mathematical rule and its parameters, ensuring consistent propagation of value property changes. [19, p. 4]
Requirement (REQ)	Links system requirements with text-based requirements. [19, p. 5]
Sequence (SD)	Provides a dynamic representation of a system's behavior by illustrating the order of interactions and events that take place over time. [20, p. 105]

SysML will be utilised to gain a holistic view of fundamental ROS2 systems. Based on this clear understanding, the meta-model will be derived and serve as a blueprint for the Digital Shadow described in the subsequent.

2.2. Digital Model, Digital Shadow, and Digital Twin as Virtual Representations of Systems

The concept of a Digital Twin was introduced first in 2002 during a presentation by Michael Grieves. The fundamental concept posits that every system comprises two interconnected components: A) the physical system that has always existed and B) a newly introduced virtual system containing all relevant information about the physical system. To elaborate, the virtual system is meant for mirroring the physical system. This twin-like property, wherein physical elements are reflected in the virtual space and vice versa, is designated as a Digital Twin. [21, p. 93][22, p. 1016f]

Even though the fundamental concept of Digital Twins remained stable since its introduction [21, p. 92], multiple definitions exist throughout several industries [22, p. 1017]. This results in a synonymously yet misleading use of the terms Digital Model, Digital Shadow, and Digital Twin as types of a Digital Twin, which vary in their degree of data integration.

Kritzinger et al. [22, p. 1017ff] categorises the related concepts of a Digital Model, Digital Shadow, and Digital Twin like illustrated in Figure 2.1. Solid lines represent an automated data flow while dashed lines refer to a manual data flow. The digital object of a *Digital Model* (Figure 2.1a) does not automatically receive information about the physical system and vice versa, it needs to be transmitted manually. As an evolution, the digital object of a *Digital Shadow* (Figure 2.1b) receives an automated data flow from the physical system, but not vice versa. Only the *Digi-*

tal Twin (Figure 2.1c) integrates an automated data flow into both directions, reaching from the physical system to the digital object and vice versa.

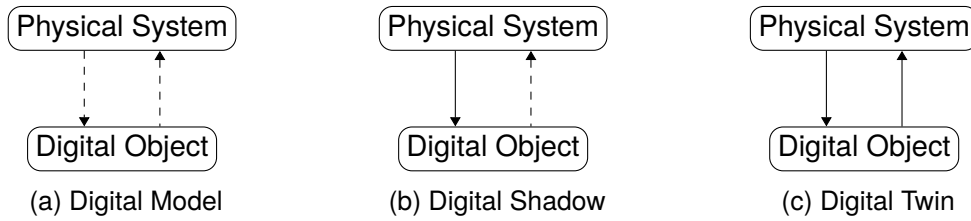


Figure 2.1.: Data Flow of Digital Model, Digital Shadow, and Digital Twin [22, p. 1017]

The objectives defined in Chapter 1 necessitate an automated data flow from the physical system to the digital object, but not vice versa. Based on these objectives and the present insights, the concept of a Digital Shadow will be employed utilising Model Based Systems Engineering and the Systems Modeling Language (SysML). However, given the similarity among the three approaches and the frequent use as synonymous terms [22, p. 1020], all three approaches are considered during the analysis of the literature. For clearance, in this thesis the term „Digital Shadow“ from now on refers to the concept of Digital Shadows, while a „Shadow Instance“ refers to a specific Digital Shadow, reflecting a specific physical system.

3. Digital Shadows versus ROS2 Systems: Verification and Analysis

Building on the fundamentals described in the previous chapter, the focus now shifts to an analysis of the preceding studies on creating Digital Shadows in robotics in Section 3.1. Section 3.2 introduces the conceptual model and workflow for Digital Shadows that will be utilised in the present thesis. Subsequently, the runtime verification of ROS2 systems, as it reflects another use case of dynamic system models, is examined in Section 3.3, followed by an exploration of the analysis and monitoring of ROS2 systems in Section 3.4. Previous work on modelling ROS2 is described in Section 3.5.

3.1. Digital Twins and Digital Shadows in Previous Work

It is observed that the main focus of Digital Shadows and Digital Twins is on the industry [23, p. 375] and has a growing interest in robotics [24, p. 4]. This section presents several approaches that arise from different robotic applications.

Zhang et al. [25] proposes a framework for the development of an Industrial Robot Digital Twin. It develops a comprehensive ontology model, encompassing structural and functional data such as state information and measurement values. Furthermore, the approach encompasses static information, such as names and types, as well as dynamic, time-dependent information, including movement, position, and load. To obtain an accurate Digital Twin instance, it covers the entire lifecycle of the industrial robot, reaching from the design phase to manufacturing, utilisation, and maintenance to recycling. Consequently, the information from each phase influences the Digital Twin instance, which combines model-based and simulation-based techniques. This facilitates real-time monitoring and predictive maintenance – a proactive maintenance concept that utilises continuous monitoring and data analysis to predict the condition of a system. The model is based on MBSE using SysML, ensuring that all aspects of the system are modelled consistently and comprehensively. In order to enable different variants, the paper utilises a feature model that categorises features into mandatory, optional, and alternative features, thus enabling flexibility while maintaining a basic structure.

Kropatschek et al. [26] proposes a Digital Shadow to support Failure Mode and Effects Analysis (FMEA) – a systematic method for identifying and evaluating potential error sources in a

system – in robotic systems on the shop floor. FMEA experts and developers collaborate to link FMEA models with production, process, and resource data, forming a comprehensive system that maps various types of faults and their effects on the running system. This knowledge is structured and stored inside a knowledge graph, which represents the virtual model and refers to the physical system. Afterwards, experts select relevant data sources and variables to capture the physical system's characteristics and continuously update the virtual model. This facilitates the monitoring of the running system's performance over a specified time period.

Pisanelli et al. [27] proposes ROSIE, a ROS adapter that aims to provide a plug-and-play solution to create a Digital Twin for ROS1 systems. The primary objective of this proposal is to facilitate the operation and configuration of industrial robot arm movements by providing two types of goals that can be set using augmented reality. The first goal is to move the robot's arm to an arbitrary position in space. The second is to set up a trajectory that the robot's arm shall follow. The integration is achieved by augmenting the ROS-control package with an additional abstraction layer. The ROS-control package provides access to robotic actuators and consists of several abstraction layers to fit agnostic, ROS-based robots. By adding another abstraction layer, Pisanelli et al. [27] achieves robot-independent control of the physical system. The virtual model is updated by subscribing to the „Joint State“ topic, which contains the latest information about the joint states. It is important to note that ROSIE is designed for ROS1. However, the authors are currently working on migrating the tool to ROS2.

In summary, Zhang et al. [25], Kropatschek et al. [26], and Pisanelli et al. [27] offer valuable contributions, yet each is tailored to a specific industrial context. Zhang et al. [25] presents an Industrial Robot Digital Twin that spans the entire lifecycle and utilises a feature-driven approach. However, the need to predefine features would limit flexibility for broader ROS2 applications and thus contradicts to the objectives of this thesis. Another focus is on predictive maintenance and therefore the aim is to prevent failures rather than to detect and diagnose faults. In contrast, Kropatschek et al. [26] focuses on the production phase, integrating Failure Mode and Effects Analysis to evaluate product quality. Pisanelli et al. [27] employs a distinct approach by leveraging the ROS-control package to mirror physical systems into augmented reality, facilitating the deployment of industrial robots. However, it restricts flexibility and does not specifically address Fault Detection and Diagnosis needs, thereby contradicting the objectives of this thesis.

Multiple elements, such as the division of static and dynamic data in Zhang et al. [25] and Kropatschek et al. [26], the utilisation of SysML for modelling in Zhang et al. [25], and the utilisation of a knowledge graph in Kropatschek et al. [26] are promising. Nevertheless, these approaches do not fully satisfy the requirement for a flexible, ROS2-wide solution that facilitates Fault Detection and Diagnosis in the development phase during rapid prototyping.

To create a Digital Shadow that matches our requirements, the subsequent section presents a conceptual model for Digital Shadows and a workflow for creating a Shadow Instance.

3.2. Creation of a Digital Shadow

In accordance with the descriptions provided in Section 2.2 and the required automated data flow from the physical object to the virtual model exclusively, the Digital Shadow proves to be a suitable solution that meets our requirements. Furthermore, this thesis adopts the following definition: „A digital shadow is a set of temporal data traces and/or their aggregation and abstraction collected concerning a system for a specific purpose with respect to the original system.“ [28, p. 3].

3.2.1. Conceptual Model of a Digital Shadow

The conceptual model utilised in this thesis is informed by the conceptual model proposed by Brecher et al. [29] and Becker et al. [30], and illustrated in Figure 3.1. In general, it specifies which concepts and relations a structure requires to function as a Digital Shadow.

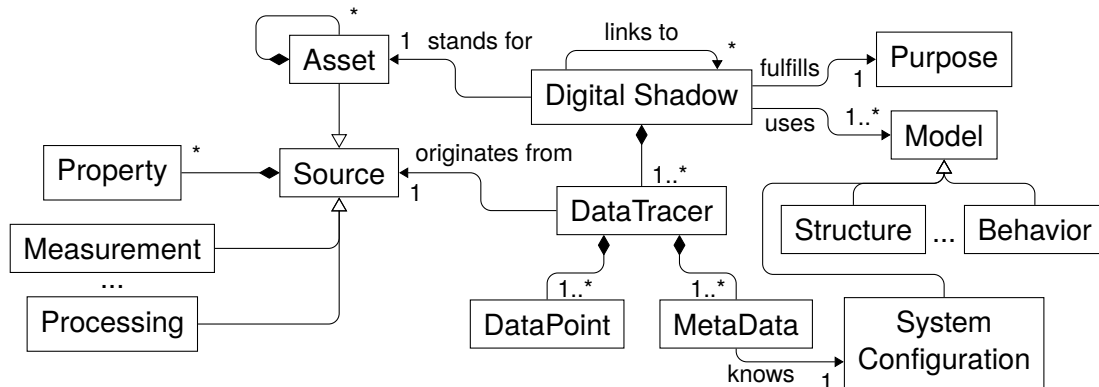


Figure 3.1.: Conceptual Model of a Digital Shadow according to [29][30]

According to the established definition and as illustrated in Figure 3.1, each Shadow Instance is characterised by a distinct Purpose. The *Purpose* serves to filter the data, ensuring efficiency by maintaining focus on relevant information. The *Model* is characterised by, e.g., the behaviour and structure of the Asset and is utilised as a construction plan. Guided by the Purpose, the Model should only represent the relevant aspects of the system. The Asset reflects the object that is to be mirrored by the Shadow Instance. It may include subsystems that are classified as Assets. It is important to mention that each Shadow Instance is linked to a single Asset, while an Asset can be reflected by multiple Shadow Instances. Establishing suitable attributes that accurately represent the properties of the Asset facilitates their utilisation as a data *Source* and therefore the creation of a Shadow Instance. Each property or set of properties represented by the Shadow Instance is obtained by tracing a specific Source utilising a connected *DataTracer*. A *DataPoint* represents a single value from a single DataTracer. Since each DataTrace can be associated with several DataPoints, they can, e.g., reflect a time series. To contextualise

DataTraces, they hold *MetaData* that describes them and holds information about the *System Configuration*. [30, p. 273ff]

Based on the conceptual model, the following section focuses on the workflow of creating a Shadow Instance.

3.2.2. Workflow for Creating a Shadow Instance

Figure 3.2 illustrates the process of generating a Shadow Instance based on multiple data inputs and predefined structures.

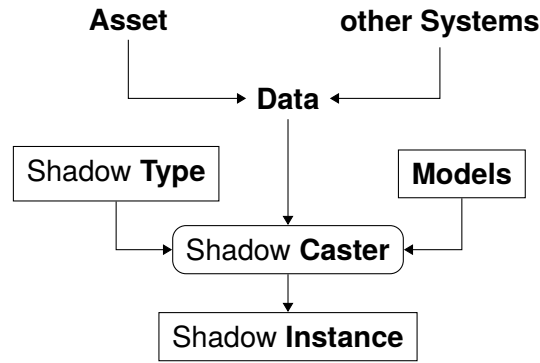


Figure 3.2.: Workflow of a Digital Shadow according to [29]

The *Asset*, as described in Section 3.2.1, represents the specific object for which a Shadow Instance is created. Additionally, *other Systems*, such as external simulations or data sources, contribute data that, while not directly linked to the *Asset*, may still influence its behaviour. Both the *Asset* and *other Systems* transmit data to the *Shadow Caster*, which is responsible for constructing the Shadow Instance. This process is guided by two key elements: the *Shadow Type* and further *Models*. The *Shadow Type* functions as a blueprint, defining the general structure of the Shadow Instance based on its intended purpose and ensuring consistency across similar entities. In contrast, *Models*, formulating domain knowledge, provide object-specific details essential for refining the generated Shadow Instance.

Together, these components enable the Shadow Caster to create an accurate Shadow Instance, tailored to the characteristics and environmental influences affecting the *Asset*. [29, p. 836]

The presented conceptual model has been employed successfully in a number of works, including Brecher et al. [13] and Schuh et al. [31].

This section provided a comprehensive overview of the creation of a Shadow Instance and the model of a Digital Shadow. As this thesis focuses on supporting Fault Detection and Diagnosis on ROS2 systems and thus facilitating the analysis of such systems, the subsequent sections explore multiple approaches to validate and analyse ROS2 systems.

3.3. Runtime Verification in ROS2 Systems

Runtime verification is a verification technique aimed at checking the correctness of a system during its execution. The core idea of runtime verification is to instrument a system with monitors that observe execution traces and detect violations or deviations from the expected behaviour. Therefore, runtime verification checks whether the functionality of a system meets the requirements and should not be confused with runtime validation, which focuses on whether those requirements are compatible with the system's intended purpose.

Huang et al. [32] introduces ROSRV, a runtime verification framework for ROS-based robotic systems, aiming to enhance safety and security by monitoring and controlling system interactions in real time. Through a specification language, users can define safety properties and security policies, which are then translated into automatically generated monitors that observe message flows and trigger corrective actions if necessary. To achieve this, ROSRV, designed for ROS1, replaces the ROS Master with a so-called RVMaster.

Ferrando et al. [33] and Saadat et al. [34] introduce ROSMonitoring and ROSMonitoring 2.0, respectively. Both share the common goal of enhancing runtime verification in ROS-based systems, ensuring that robotic applications adhere to predefined constraints during execution. They operate by monitoring system behaviour during runtime, rely on formal specifications to define correct behaviour, and aim to improve system reliability and safety. Additionally, both frameworks integrate automated monitoring mechanisms, reducing manual effort in verifying system compliance. Neither approach alters the core functionality of ROS but instead adds a verification layer that passively or actively ensures constraints are met. Furthermore, both systems have been evaluated in real-world robotic scenarios to analyse their scalability and overhead impact.

Despite these similarities, the two approaches differ in implementation and scope. ROSMonitoring 2.0 focuses on automating the generation of runtime monitors from natural language requirements, enabling developers to specify constraints in a structured format that is automatically transformed into executable monitoring components. These monitors passively observe system behaviour, identifying deviations from expected conditions but without direct intervention. In contrast, ROSMonitoring actively validates message exchanges, effectively enforcing constraints rather than merely detecting violations. Another distinction lies in flexibility and integration. While ROSMonitoring 2.0 follows a structured workflow with a dedicated pipeline for transforming requirements into monitors, ROSMonitoring enables users to define verification properties using diverse specification formalisms, making it more adaptable for differing use cases. Despite their strengths in constraint validation, neither approach provides a comprehensive Fault Detection and Diagnosis mechanism, as they focus on verifying rule compliance rather than analysing failure causes. While both enhance system robustness by ensuring correct operation within defined constraints, a more holistic model-driven approach would be required to systematically detect, diagnose, and mitigate faults in ROS2-based robotic systems.

In summary, ROSRV [32], ROSMonitoring [33] and ROSMonitoring 2.0 [34] focus on runtime verification and thus prevent hazardous behaviour based on rules. The definition of these rules is achieved through the utilisation of differing languages, necessitating deep expert knowledge. In contrast, this thesis introduces a meta-model that reduces the need for expert knowledge, facilitates Fault Detection and Diagnosis, and can be used with various algorithms, including those that perform runtime verification.

3.4. Analysis of ROS2 Systems

Given the present thesis emphasis on developing a meta-model that supports Fault Detection and Diagnosis by facilitating the analysis of the ROS2 system, the focus now shifts to examining prior research conducted on analysing ROS2 systems. Accordingly, this section also explores several implementation specific details that will be particularly relevant for the evaluation of the approach proposed in this thesis (see Chapter 7). The research leads to several works that focus on analysing ROS2 system performance, particularly in terms of latency measurements [35][36][37][38].

ros2_tracing [39] is a low-overhead tracing tool that employs several tracepoints within the ROS2 source code. These tracepoints provide detailed information about events occurring within a ROS2 system, such as the initialisation of nodes and publishers [40]. At the time of this work, *ros2_tracing* only enables offline analyses and is therefore unsuitable for the objectives of this thesis, as the monitoring should occur at runtime. However, the tracepoints are included within the ROS2 source code and enabled to be traced, and thus analysed, online. Independent of *ros2_tracing*.

Multiple previous works demonstrate the potential of *ros2_tracing*. For instance, Betz et al. [35] and Teper et al. [37] employed *ros2_tracing* to collect fine-grained runtime data and identify critical timing bottlenecks, while evaluating the impact of different system configurations, such as varying executor strategies. Bédard et al. [41] utilises *ros2_tracing* to analyse the message flow throughout a ROS2 system.

For the objectives of this thesis, it is imperative to mention that tracepoints only generate a latency overhead when actively traced. The impact of tracepoints related to publishing messages with differing sizes (1 KiB, 32 KiB, 64 KiB, 256 KiB) and frequencies (100 Hz, 500 Hz, 1000 Hz, 2000 Hz) was measured by Bédard, Lütkebohle, and Dagenais [39, p. 6516f]. The mean latency overhead quantifies to 3.3 μ s and therefore the created overhead „is acceptable for most target situations in ROS2.“ [39, p. 6516f]. Based on this evaluation, and also due to the high number of tracepoints, tracing those tracepoints within the ROS2 source code offers a promising approach for analysing ROS2 systems.

Maruyama, Kato, and Azumi [36] employed iRobot's *ros2-performance* [42] to measure and analyse latencies. Since the framework is primarily designed to evaluate single-process applications offline, it does not fit our requirements.

Another approach for analysing a ROS2 network is the utilisation of the *ros2-diagnostics* framework. The framework's functionality is restricted to the provision of tools for standardised messages and evaluation. However, the establishment of the integration interface on the Node side is a responsibility that falls to the user. Consequently, it does not fully meet the objectives of this thesis, as this constitutes an intervention into the codebase [43].

In summary, the most promising contribution is provided by *ros2_tracing* [39], as it enables a non-intrusive method to observe ROS2 systems by tracing specific tracepoints. Alternatives for online analysis, such as *ros2-diagnostics* [43], are intrusive and therefore necessitate modification of the codebase to observe a ROS2 system in question.

Although runtime verification and system analysis provide valuable insights into the behaviour and correctness of ROS2 systems during execution, these approaches either rely on deep expert knowledge or analyse only fractions of ROS2 systems. However, effective Fault Detection and Diagnosis in ROS2 systems necessitates an approach that considers the system as a whole. ROS2 system models offer such a comprehensive perspective.

3.5. Modelling ROS2 Systems

Recent research has demonstrated the significant role of models in leveraging Digital Twins and Digital Shadows. Nevertheless, as the objective is to create a Digital Shadow that facilitates Fault Detection and Diagnosis for generic ROS2 systems, this thesis aims to create a meta-model depicting ROS2 systems in general. In the literature, such models are seldom encountered in the context of ROS2.

One approach is presented by *MeROS* [44], a SysML-based meta-model developed for both ROS versions. MeROS systematically encapsulates core ROS concepts – including Nodes, Topics, Services, and Actions – by grouping related communication methods and components into abstract entities. This approach not only improves the clarity and compactness of system descriptions but also facilitates system documentation, defect detection, and team collaboration. Its design is driven by a comprehensive set of requirements derived from both literature and practical experiences in ROS-based projects. However, while MeROS excels in providing a unified model for the architecture and behaviour of ROS systems, it primarily targets the areas of system documentation and integration. Therefore, it provides guidance for ROS systems that are in the preliminary stages of their design process. Notably, it does not offer explicit constructs or extensions aimed at Fault Detection and Diagnosis. Furthermore, the created model refers

to ROS2 only and does not consider the underlying operating system, which may also be prone to faults.

Further approaches are designated for ROS1 only and lack completeness, as concepts such as Actions might not be represented. For instance, *ROSMOD* [45] is a model-driven development environment for distributed ROS systems, focusing on modelling, code generation, and deployment. While it supports runtime monitoring, it lacks direct Fault Detection and Diagnosis capabilities. In contrast, *RoBMEX* [46] is a domain-specific modelling framework for graphical mission planning and automatic code generation in ROS-based systems. It does not integrate runtime monitoring and is designed for mission specification rather than system diagnosis.

To date, the literature lacks a dedicated meta-model for ROS2 systems that is tailored to Fault Detection and Diagnosis, leaving a notable gap in the state of the art. This gap motivates the development of a meta-model for ROS2 systems that integrates mechanisms which support Fault Detection and Diagnosis, paving the way for more resilient and reliable rapid prototypes. Therefore, the subsequent chapter employs Model Based Systems Engineering to model various concepts and aspects of ROS2 systems.

4. Modelling ROS2 using Model Based Systems Engineering and SysML

For the creation of a Shadow Instance, an associated model is needed that serves as a blueprint for the Shadow Caster [29][30]. Since one requirement is adaptability to a broad range of ROS2 systems, this model will be represented by a mathematical meta-model that considers the fundamental concepts of ROS2 systems. This chapter serves as the basis for the meta-model, presenting and modelling various aspects of ROS2 systems, including certain concepts and their connections to the operating system.

The concept of Nodes is analysed in detail in Section 4.1 before delving into the communication within ROS2 systems in Section 4.2. The subsequent Section 4.3 describes ROS2 Containers – a concept in which Nodes are grouped to enable more efficient communication and optimise resource utilisation. Besides the ROS2 communication and thus the structure of a ROS2 network, Timers (described in Section 4.4), Executors (Section 4.5) and the operating system are to be considered to create an authentic Shadow Instance. To address the operating system, the significance of considering processes and their structure is described in Section 4.6.

Figure 4.1 depicts the structure and relationships between several of these elements that will be described throughout this chapter, additionally providing references to the respective sections.

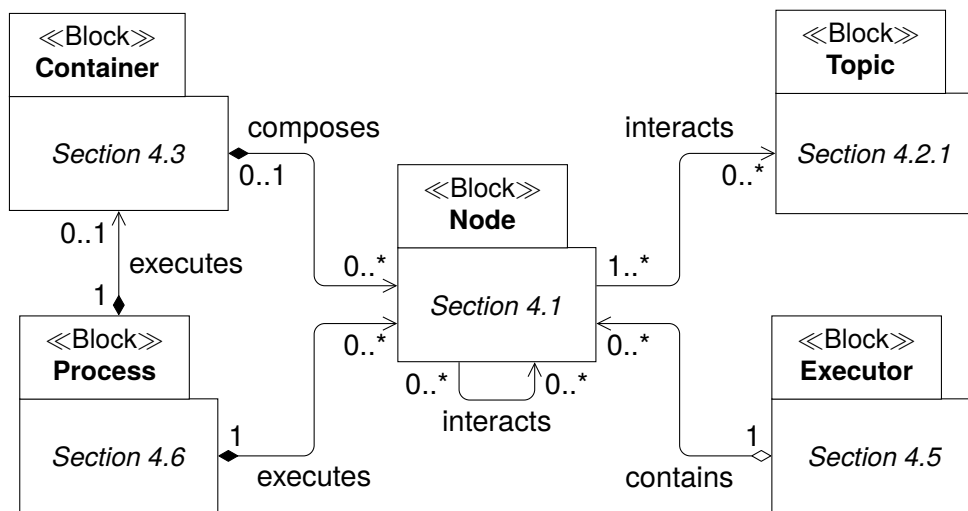


Figure 4.1.: BDD: Elements of ROS2 and their Respective Relations

4.1. The Concept of Nodes in ROS2

ROS2 employs a component-based structure where Nodes serve as autonomous execution units. They are provided by the user and execute specific tasks, such as reading sensor data, motion planning, or actuation control [47, p. 3f]. Their communication is presented in Section 4.2.

Figure 4.2 illustrates the concept of a Node and its relations to further concepts the Node manages, it also contains exemplary attributes. A Node may manage Publishers, Subscribers, Servers, Clients, and Timers, their concepts are described in the following sections. Each of them only exists in connection to a Node and cannot exist without a Node.

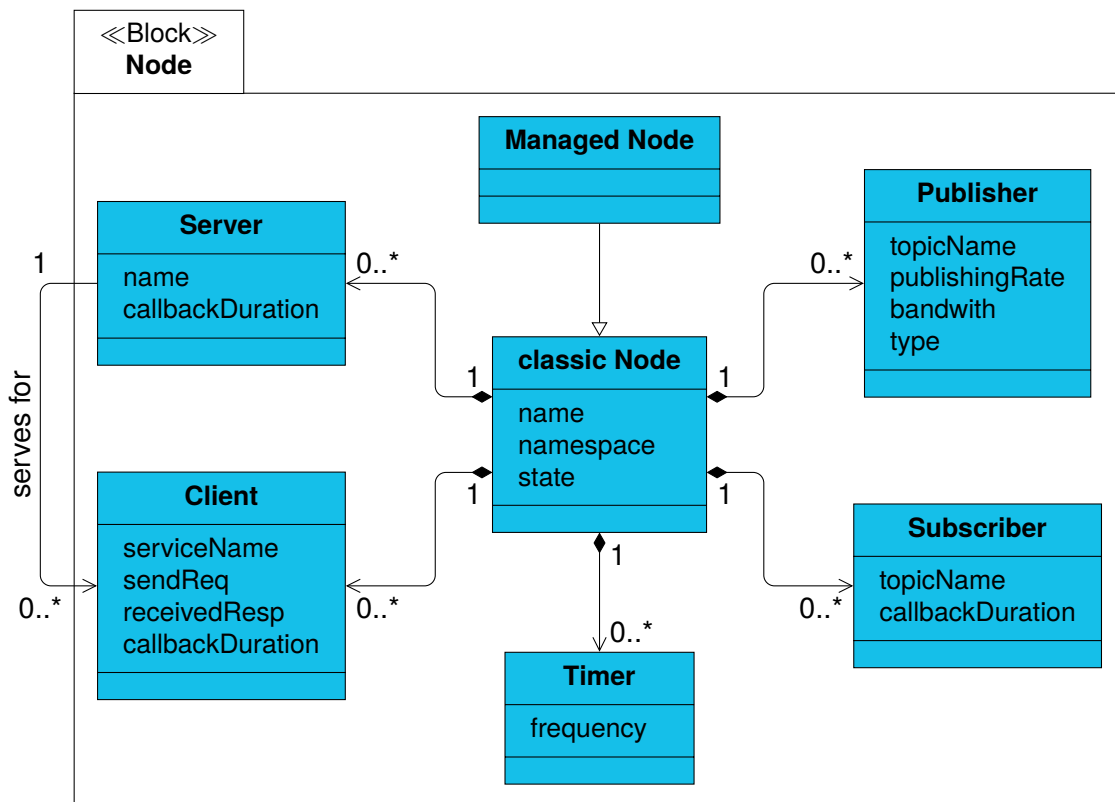


Figure 4.2.: BDD: ROS2 Node and Adjacent Elements of ROS2

Each Node in ROS2 has a *name* and is associated with a specific namespace. Both the name and the namespace remain constant during runtime. Although the ROS2 documentation necessitates a unique fully qualified name for each Node [48], this constraint is not enforced. However, for the approach of this thesis, the author assumes that the fully qualified name of a Node identifies it.

Namespaces provide a hierarchical structure for organising Nodes into groups. Each namespace follows a distinct naming structure and can contain multiple sub-namespaces. As illus-

trated in Figure 4.3, each sub-namespace is nested within exactly one parent namespace. The hierarchy is rooted in a global namespace, which itself is not subordinate to any other namespace. While every Node and Topic is associated with exactly one namespace, a single namespace may include multiple Nodes and Topics. Topics will be described within the subsequent Section 4.2.1.

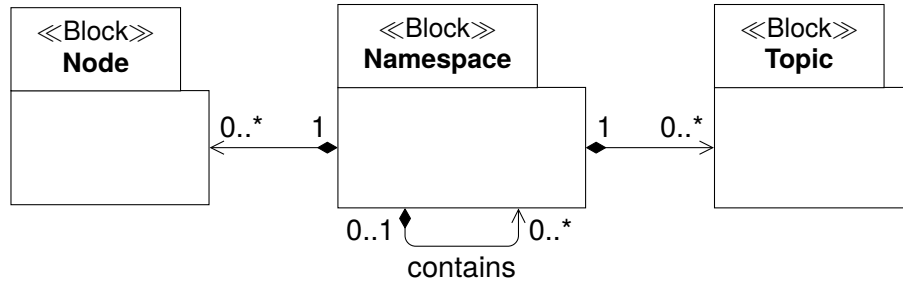


Figure 4.3.: PKG: Namespace Hierarchy in ROS2 Systems

Despite the flexibility of classic Nodes, their utilisation is not recommended due to their disadvantages in, e.g., CPU utilisation, latency, and unmanaged states [47, p. 6]. Instead, recommended architectural patterns are the concepts of Node Composition and Managed Nodes, which can also be employed in combination. *Node Composition* will be described in Section 4.3. The concept of *Managed Nodes* provides an architectural pattern for managing the Node's lifecycle. It is a managed execution model with the well-defined states A) „active“, B) „inactive“, C) „un-configured“, and D) „finalised“. This structured lifecycle enables finer control over the Node [47, p. 4][49]. As a classic Node only exists within the states A) „active“ and B) „inactive“, its states constitute a real subset of those offered by Managed Nodes.

Publishers, Subscribers, Servers, and Clients refer to the communication of Nodes and will be described in the subsequent section.

4.2. Communication Mechanisms of ROS2

ROS2 provides three methodologies of communication, namely A) Topics, B) Services, and C) Actions [47, p. 3]. Their concepts will be presented and modelled within this section.

4.2.1. The Concept of Topics in ROS2

Topics represent the predominant communication mechanism within ROS2 systems, exhibiting an asynchronous communication pattern. They are based on a Publisher-Subscriber methodology, acting as intermediaries between Nodes and establishing many-to-many communication.

As illustrated in Figure 4.4, a Node employs a specific *Publisher* to transmit messages of a defined *type* onto a designated Topic. Various properties, such as the *publishing rate* or the *bandwidth* utilised, may be of particular interest. Any Node subscribed to the Topic, utilising a corresponding *Subscriber*, fetches the message and executes an assigned callback function, whose runtime is referred to as the *callback duration* [47, p. 3]. It is important to mention that a Topic may distribute messages of multiple types and a Subscriber only fetches the messages that match its type.

This communication method is particularly beneficial due to the system's high modularity, as it enables developers to add or remove Nodes without requiring modifications to other Nodes [47, p. 3].

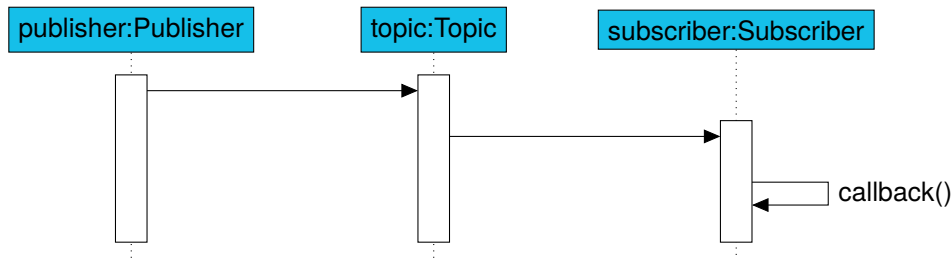


Figure 4.4.: SQ: ROS2 Communication via Topics

Attributes such as a Publisher's publishing rate are assigned to their origin, reflecting local properties. However, these individual values collectively affect the characteristics of the connected Topic. For instance, the Topic's overall message forwarding rate can be understood as the sum of the rates of all associated Publishers. This influence is depicted in Figure 4.5.

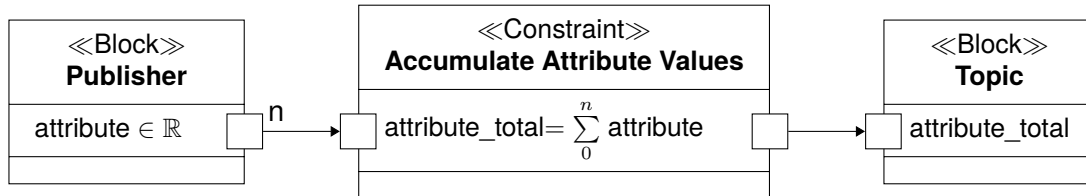


Figure 4.5.: PAR: Accumulate Publisher related Attributes to Topics

4.2.2. The Concept of Services in ROS2

Services represent a synchronous, one-to-one communication between Nodes as visualised in Figure 4.6. This concept adheres to a uniform, non-blocking request-response paradigm, wherein a Client sends a request to a Server that processes the requested task and returns a corresponding response upon completion [47, p. 4]. This induces that each request the Client sends is responded to by the Server. Storing the number of *sent requests* and *received responses* enables the validation of the specific Server-Client communication. It is important to

mention that it is up to the user whether the Client waits for the response to return or triggers a callback that is executed upon the return of the response. However, as both may trigger callbacks, they also hold a *callback duration* analogous to Subscribers.

Services are uniquely identified by their *name*. Even though it is not enforced, an ambiguous name leads to undefined behaviour in terms of which Service Server executes the request [50]. Therefore, this thesis assumes an unambiguous name.

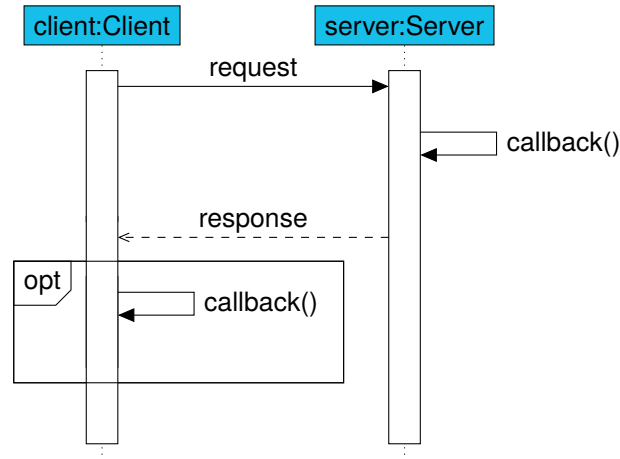


Figure 4.6.: SQ: ROS2 Server-Client Communication

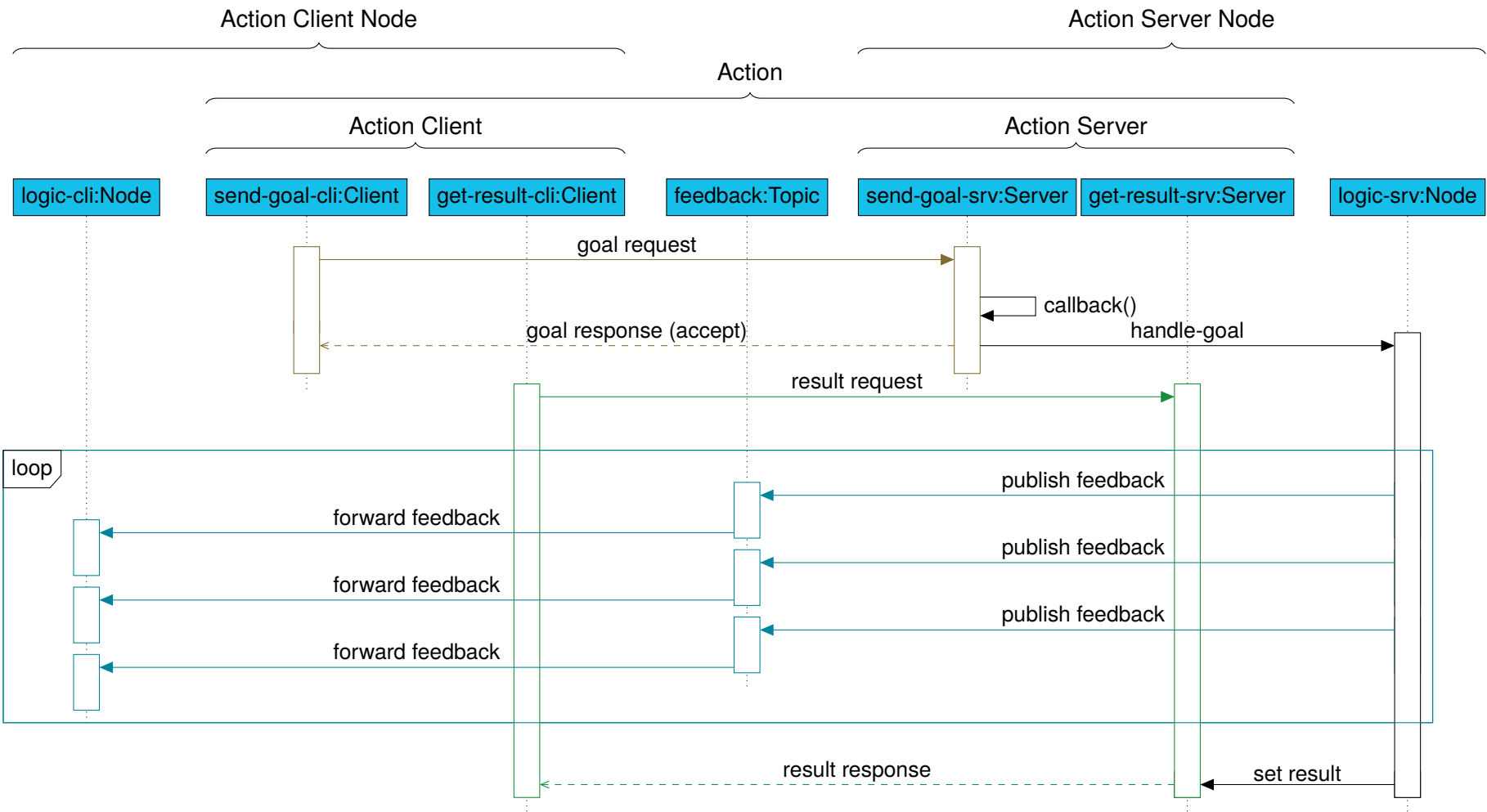


Figure 4.7.: SQ: ROS2 Communication via Actions based on [51]

4.2.3. The Concept of Actions in ROS2

Actions represent a more unique way of communication and are the optimal method in scenarios where tasks are requested and necessitate continuous feedback. They are predicated on the concepts of Topics and Services described previously. More specifically, each Action consists of the Services A) „send_goal“, B) „cancel_goal“, and C) „get_result“, and the Topics A) „feedback“ and B) „status“. The standard procedure of an Action is illustrated in Figure 4.7. To concentrate on the essentials of communication, only a subset of Services and Topics is represented. Whereas the Services „send_goal“ and „get_result“ are coloured brown and green, respectively. The feedback loop is coloured blue.

The Action Client of a Node sends a requested goal to the „send_goal“-Server of the Node that contains the corresponding Action Server. Upon acceptance, the Action Server triggers the user-defined logic and responds to the Client with an acceptance, prompting the Action Client to request the „get_result“-Server. Concurrently, the user-defined logic starts publishing continuous feedback on the designated feedback Topic that is subscribed to by the Node containing the Action Client. Upon completion of the goal, the Action Server responds via its „get_result“-Service. [51]

Throughout this process, the Action Server maintains a state machine that tracks the state for each request from each Action Client [51]. State changes are published onto the status Topic. A more detailed description of Actions is available at Open Source Robotics Foundation [51].

For clarity, in this thesis, „Servers“ and „Clients“ refer to the concept of Services, whereas „Action Servers“ and „Action Clients“ refer to the concept of Actions.

The present section provided a comprehensive overview of the various forms of communication that are employed by ROS2. The subsequent section elaborates on the concept of Containers.

4.3. The Concept of Containers in ROS2

As mentioned in Section 4.1, the concept of *Node Composition* reflects another recommended architectural pattern. It enables grouping several Nodes into a single Container for execution within a single process of the operating system. This approach facilitates intra-process communication, significantly reducing latency and CPU utilisation [47, p. 6f]. The concept of grouping Nodes into Containers serves as another basis for subsequent aggregations and is illustrated in Figure 4.8.

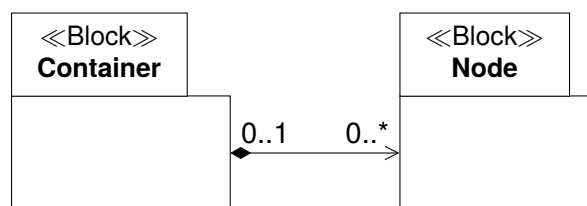


Figure 4.8.: PKG: Grouping Nodes by Containers

4.4. The Concept of Timers in ROS2

In ROS2, callbacks are not exclusively triggered upon the arrival of messages. A further possibility to trigger callbacks is through Timers. These *Timers* specifically initiate a designated callback execution at a specified frequency [37, p. 213][52]. Therefore, Timers also hold a *callback duration* analogous to Subscribers.

4.5. The Concept of Executors in ROS2

In ROS2, *Executors* are responsible for managing the execution of callbacks, „the smallest schedulable entity in ROS2“ [53, p. 2], and can therefore be referred to as high-level schedulers [39, p. 6512]. Given the existence of different types of callbacks – namely A) Timer-, B) Subscriber-, C) Service Server-, and D) Service Client callbacks – with differing priorities, the sequence of execution is determined by the priorities assigned to each callback type, with the highest priority assigned to A) descending to the lowest priority assigned to D) [52][53, p. 3]. *Wait sets* function to inform an Executor of the availability of events and messages queued. When the processing time of each callback exceeds the period of incoming events and messages, the queue is populated, and the respective Executor prioritises callbacks according to their priority. Executors may be under such a high workload that processing a callback can be delayed [41, p. 7ff], which motivates the observation of their utilisation. However, if the processing time does not exceed the period of incoming events and messages, callbacks are executed in a FIFO (first-in-first-out) manner. [52][53]

To counteract the population of the queue and thus enable a more prompt execution of callbacks, multithreaded Executors enable the concurrent execution of callbacks within a predetermined number of threads. They further enable the grouping of callbacks into callback groups to prevent hazardous behaviour that might occur through parallel execution. To prevent such hazardous behaviour, *Mutually Exclusive Callback Groups* disable parallel execution and execute callbacks one after the other. Conversely, *Reentrant Callback Groups* enable the parallel execution of callbacks, which is only limited to a specific number of threads. Nevertheless, multithreaded Executors cannot completely exclude the risk of populating the queue. [52]

Since each Node is assigned to a specific Executor, but every Executor can schedule callbacks of multiple Nodes, Nodes can be grouped by those Executors. This perspective is especially beneficial, since queues might be populated and therefore cause a certain delay in execution. Figure 4.9 illustrates this grouping, which forms another basis for aggregating a ROS2 system in the subsequent.

While the previous sections have focused on internal mechanisms and concepts of ROS2, a comprehensive system understanding also requires consideration of the underlying operating system processes. The following section therefore extends the perspective onto the underlying operating system.

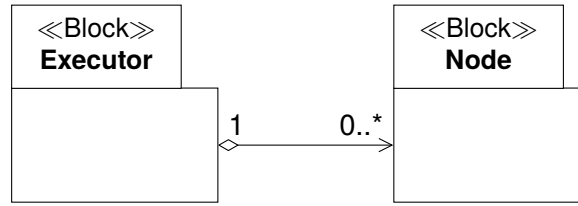


Figure 4.9.: PKG: ROS2 grouping by Executors

4.6. Considering Operating System Processes within ROS2 Systems

Given the objective of this thesis to model ROS2 systems rather than ROS2 itself, the model also incorporates aspects of the underlying operating system. Each Node runs inside a *process* that may execute multiple Nodes. These Nodes and thus processes may be started either directly or via *launch files*. The latter facilitates the coordinated launch of multiple ROS2 Nodes [54]. For the approach of this thesis, the author assumes that related Nodes are launched by an encompassing launch file. This assumption enables a meaningful grouping of Nodes within a ROS2 system.

In order to elaborate further on the system-level execution context, it is essential to consider the instantiation and management of processes. Each process, with the exception of the operating system's first process, is initiated by an entity that runs within another process, forming a hierarchical process structure as illustrated in Figure 4.10. Nodes inherently represent such entities. In order to maintain focus on the ROS2 system, multiple subtrees are later extracted from the resulting tree, which depicts the complete process structure of the operating system. These subtrees relate to the structure that is initiated by the launch files, ensuring that the analysis remains centred on the ROS2 software stack.

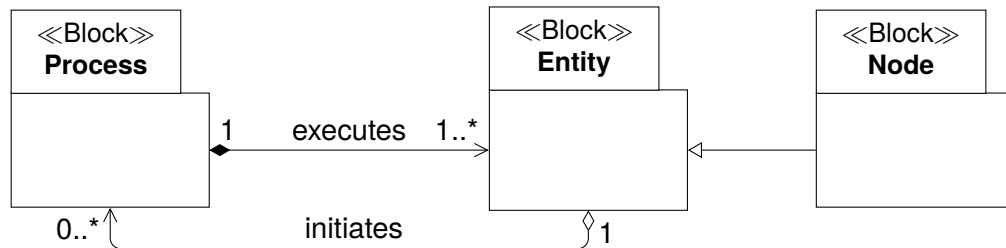


Figure 4.10.: PKG: Process Hierarchy in ROS2 Systems

In summary, this chapter applied Model Based Systems Engineering to model key aspects of ROS2 systems, with a particular focus on Nodes, their communication mechanisms and related concepts. In addition, the underlying operating system was considered, as it provides the execution environment for the ROS2 software stack. The following chapter builds upon this ROS2 model to derive a mathematical meta-model. This meta-model abstracts core concepts, aggregates system areas, and defines computational rules to support Fault Detection and Diagnosis by facilitating the analysis of a ROS2 system in question.

5. Deriving a Mathematical Meta-Model to represent the Fundamentals of ROS2 Systems

Building upon the ROS2 system model introduced in the previous chapter, this chapter addresses the gap identified in Chapters 1 and 3. This chapter introduces a mathematical meta-model that is capable of mirroring ROS2 systems in order to support Fault Detection and Diagnosis. Later on, it will be utilised as a blueprint for the Shadow Caster in order to generate Shadow Instances.

Throughout this chapter, multiple definitions describe the characteristics of the proposed meta-model. In order to reflect and evaluate the definitions, multiple examples are illustrated alongside. To ensure an unambiguous and consistent visual representation, Figure 5.1 illustrates multiple components of the directed Graph, which will be introduced in the following sections. Figures that are labelled with $\blacklozenge$ adhere to this visual standard within the constraints of the meta-model and may be refined in detail. However, to emphasise specific characteristics, certain figures intentionally deviate from the visual standard. These deviations are explicitly mentioned and labelled using $\blacklozenge$.

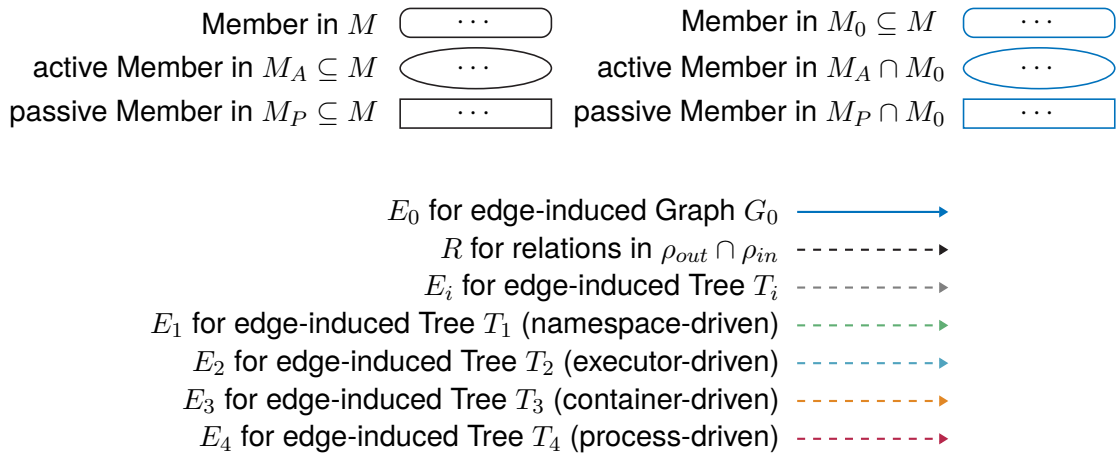


Figure 5.1.: Legend to represent Directed Graph G $\blacklozenge$

Previously, the author was a member of a research group that proposed a mathematical meta-model for a broad range of distributed robotic systems in [55]¹. The present work will extend the proposed meta-model by multiple definitions. Furthermore, due to the particular focus of this thesis, the mathematical meta-model is strongly influenced by the concepts of ROS2. Nevertheless, it is intentionally designed to be abstract and generic, with the objective of extending its applicability beyond ROS2 to a broader range of distributed robotic systems.

To elaborate, Section 5.1 introduces the fundamental, directed Graph that abstractly represents a ROS2 system. The subsequent Section 5.2 employs the directed Graph in order to create multiple aggregations, which facilitate the analysis of a ROS2 system in order to support Fault Detection and Diagnosis.

5.1. Establishing a Directed Graph for Modelling ROS2 Systems

In order to represent a ROS2 system abstractly, a directed Graph is introduced gradually within this section. Therefore, Section 5.1.1 establishes the fundamental, directed Graph that evolves through multiple versions in Section 5.1.2. The ROS2 communication Graph is defined in Section 5.1.3.

5.1.1. Creating the Fundamental ROS2 Graph

We propose the utilisation of a directed Graph to represent a ROS2 system (Definition 5.1.1). The Graph differentiates between active and passive Members. *Active Members* initiate and carry out tasks by executing operations or controlling the execution of logic within the ROS2 system [55, p. 2], e.g. Nodes. In contrast, *passive Members* neither initiate nor execute operations themselves. Instead, they establish data distribution or function as pure organisational entities [55, p. 2], e.g. Topics.

Additionally, the Graph consists of *communication Edges* and *organisation Edges*. This differentiation facilitates subsequent computations that require the Graph to be divided into various, edge-induced areas. These edge-induced areas will be defined throughout this chapter.

¹The paper was written concurrently with this thesis. Therefore, the contributions intersect. However, as the meta-model was extended and refined, the direct intersections of both models are rare.

Definition 5.1.1: Directed Graph to Represent ROS2 Systems

The ROS2 system is abstracted as a directed Graph:

$$G = (M, E),$$

where

- $M = M_A \cup M_P$ represents the set of Members, divided into active Members (M_A) and passive Members (M_P), with $M_A \cap M_P = \emptyset$,
- $E = E_0 \cup \bigcup_{i=1}^{N \in \mathbb{N}_{>0}} E_i$ represents the set of directed Edges[♣].

[♣] E_0 and E_i will be defined in Definitions 5.1.4 and 5.2.3, respectively.

To represent the properties of, e.g. Nodes, Topics, and Publishers, the Members and communication Edges are provided with an attributes function. Due to time-varying properties, e.g. the state of a Node or the publishing rate for a Publisher, the attributes function is provided with a time-dependent component.

Definition 5.1.2: Attribute Function to Assign Properties to Graph Entities

Let A be a tuple of attributes defined as:

$$A = A_0 \times A_1 \times \dots \times A_n \text{ with } n \in \mathbb{N}.$$

Then the attributes function θ refers to the Members and communication Edges, and is defined as:

$$\theta: (M \times \mathbb{T}) \cup (E_0 \times \mathbb{T}) \rightarrow A.$$

For all $k \in \{0, \dots, n\}$ and all $m \in M$, $e \in E_0$, the function θ satisfies the conditions:

$$\begin{aligned} (\exists t \in \mathbb{T}: \theta(m, t)_k \neq 0) &\Rightarrow (\forall t \in \mathbb{T}: \theta(e, t)_k = 0) \text{ and} \\ (\exists t \in \mathbb{T}: \theta(e, t)_k \neq 0) &\Rightarrow (\forall t \in \mathbb{T}: \theta(m, t)_k = 0). \end{aligned}$$

Time-dependence is enabled via the temporal domain $\mathbb{T} = [0, t]$, $t \in \mathbb{R}_{>0}$.

Reflection

As defined in Definition 5.1.1, the model introduces two types of Members to abstractly represent various entities in ROS2 systems. Firstly, active Members represent operating or controlling entities within the system and thus represent:

Nodes As they serve as autonomous execution units and encompass the logic for the executed operations.

Executors As they execute and control the execution of callbacks a Node encompasses.

Launch Files As they initiate the execution of Nodes.

In contrast, passive Members do not perform or initiate operations themselves, but establish data distribution or serve as pure organisational, non-executing, and non-initiating entities. Thus, they represent:

Topics As they establish data distribution between Publishers and Subscribers of Nodes.

Namespaces As they facilitate organising and structuring the ROS2 Graph in a hierarchical manner.

Containers As they group Nodes and thus enable a more efficient execution.

The presence of passive Members further enables the existence of virtual entities that exist for organisational reasons only. The utilisation of such virtual entities is intended for the subsequent aggregation of the Graph, which will be fulfilled by integrating Trees into the Graph.

As will be described in more detail subsequently, the Edges E_0 represent the communication within the ROS2 system, while E_i will be employed to model further structures within the Graph. The attributes function θ enables the representation of time-varying values, in order to enable differing values at differing timestamps. This time series facilitates the analysis of temporal trends or various accumulations at a subsequent point in time. As θ does not correspond to a particular spatial domain, both numerical and textual values are permitted. Additionally, the attributes of Members and communication Edges are disjoint. This is an important aspect for later computations.

With the fundamental directed Graph established, the following section versions the Graph to represent its dynamic nature.

5.1.2. Enabling the Evolution of the ROS2 Graph

In order to gain a comprehensive understanding of the ROS2 system, including its evolution over time, it is imperative for the Graph to adapt and evolve accordingly. This necessitates the extension of the Graph by Members and Edges, and an extension of the attributes function. Hence, Definition 5.1.3 adds a versioning to the Graph to represent its evolving nature.

Definition 5.1.3: Version the Directed Graph to Enable Evolving Nature

Let $G^v = (M^v, E^v)$ denote the state of a directed Graph at version $v \in \mathbb{N}_0$. Then, for all $v \geq 0$, the following conditions hold:

- $M^{v+1} = M^v \cup M^+$,
- $E^{v+1} = E^v \cup E^+$,

where

- M^+ : $M^v \cap M^+ = \emptyset$ denotes the set of newly added Members M ,
- E^+ : $E^v \cap E^+ = \emptyset$ denotes the set of newly added Edges E .

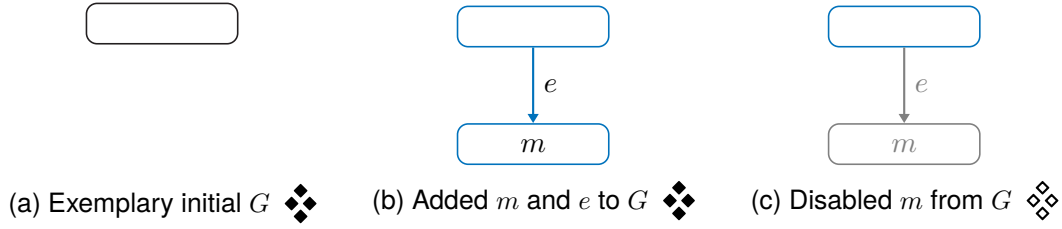
The attributes function θ extends accordingly.

However, in the following, whenever a Graph G is mentioned, it implicitly refers to the current version $G^{v_{curr}}$, unless stated otherwise.

Reflection

To represent the dynamic nature of ROS2 systems, Definition 5.1.3 creates a set of directed Graphs that is versioned each time Members or Edges are added. However, neither Members nor Edges are ever removed in order to also represent parts of the ROS2 system that had an influence on the ROS2 system earlier on. Instead, later on, an attribute is to be employed in order to indicate their state.

It is imperative to acknowledge the significance of this aspect, particularly in light of the potential for Fault Detection and Diagnosis algorithms to encompass a comprehensive range of data, including historical relationships. In order to illustrate this, Figure 5.2 represents multiple versions of the Graph. Note that Figures 5.2b and 5.2c refer to the same topology of the Graph, but m and e are coloured gray in the latter one. This is because m is removed and thus its state is set to „finalised“, also disabling e . Note that the upper entity transitions from black to blue in Figures 5.2a and 5.2b. The rationale for this is described subsequently.


 Figure 5.2.: Adding and Removing Entities to Exemplary Graph G

With the evolving nature of the directed Graph established, the subsequent section focuses on the communication and thus defines the ROS2 communication Graph.

5.1.3. Representing the Communication Structure of ROS2 Systems

To facilitate Fault Detection and Diagnosis and represent the communication structure of the ROS2 system, the Edges E_0 are employed to model the communication between Members.

Communication within ROS2 is based on the concepts of Topics and Services. The latter represents the direct communication between Nodes and thus active Members, whereas Topics provide a communication mechanism between active and passive Members. To accurately model the information flow within the ROS2 system, different types of directed communication Edges are established: *sending* Edges to represent Service interactions, and *publishing* and *subscribing* Edges to represent Topic-based communication. A special case is the direct communication of an active Member with itself, which will be discussed in the subsequent.

Definition 5.1.4: ROS2 Communication based on Kohl et al. [55, p. 3]

The communication Edges inside the directed Graph G are defined as

$$E_0 = E_{\uparrow} \cup E_{\nabla} \cup E_{\wedge},$$

where

- $E_{\uparrow} \subseteq (M_A \times M_A) \times \{0\} \times j$, with $j \in \mathbb{N}_0$ represents sending Edges,
- $E_{\nabla} \subseteq (M_A \times M_P) \times \{0\} \times j$, with $j \in \mathbb{N}_0$ represents publishing Edges,
- $E_{\wedge} \subseteq (M_P \times M_A) \times \{0\} \times j$, with $j \in \mathbb{N}_0$ represents subscribing Edges,
- $E_{\circ} \subseteq E_{\uparrow}$ with $E_{\circ} = \{((m, n), 0, j) \in E_{\uparrow} \mid m = n\}$ represents a special form of communication that carries zero payload and is employed periodically.

With the communication Edges E_0 established, the edge-induced, directed Subgraph $G_0 \subseteq G$ defined in the following represents the communication structure of the ROS2 system in question and is denoted as the *ROS2 communication Graph*. Analogous to the communication Edges E_0 , the Members M_0 are denoted as *communication Members*.

Definition 5.1.5: The ROS2 Communication Graph [55, p. 3]

The edge-induced, directed Subgraph G_0 that represents the ROS2 communication Graph is defined as:

$$G_0 = (M_0, E_0),$$

where $\forall m, n \in M_0 \subseteq M: (m, n) \in E_0 \vee (n, m) \in E_0$.

Reflection

Definition 5.1.4 introduces multiple types of communication Edges that aim to represent the various methods of communication ROS2 offers. These communication Edges unexceptionally reach from the source of information flow to its destination. This facilitates the trace of information flow, as for $s, d \in M_0, ((s, d), 0, j \in \mathbb{N}_0) \in E_0$ each s denotes the source and d the destination of information flow.

Figure 5.3 illustrates the communication between active and passive Members, and refers to the concept of Topics. It is established by publishing Edges and subscribing Edges, representing Publishers and Subscribers, respectively. This is valid, given that each Publisher or Subscriber belongs to a specific Node (active Member) and is associated with a specific Topic (passive Member). Consequently, passive Members in relation with a publishing or subscribing Edge refer to the concept of Topics (Section 4.2.1).



Figure 5.3.: Representing the Concept of ROS2 Topics with the Meta-Model ❖

Given that each sending Edge $E_{\uparrow}$ connects two active Members, the concept of Services (Section 4.2.2) is constructed by employing $E_{\uparrow}$. However, as the Edges of the Graph are directed and indicate the direction of the information flow, every Service is represented by exactly two sending Edges, relating to request and response. Figure 5.4 illustrates this type of communication, with m_1 referring to a Service Client and m_2 referring to the respective Service Server. Linking the number of sent messages to their respective sending Edges facilitates the analysis

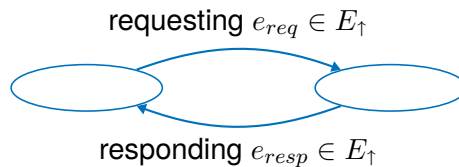


Figure 5.4.: Representing the Concept of ROS2 Services with the Meta-Model ❖

of whether each Service Client request received a corresponding Service Server response.

The concept of Actions is constructed by employing multiple Services and Topics (Section 4.2.3). Due to the existence of three Services and two Topics that are encompassed by the concept of Actions, each Action Server - Action Client pair is represented by six sending, two publishing and two subscribing Edges. Figure 5.5 exemplarily represents the concept of ROS2 Actions with the mathematical meta-model, whereas m_1 refers to an Action Client and m_2 refers to the respective Action Server.

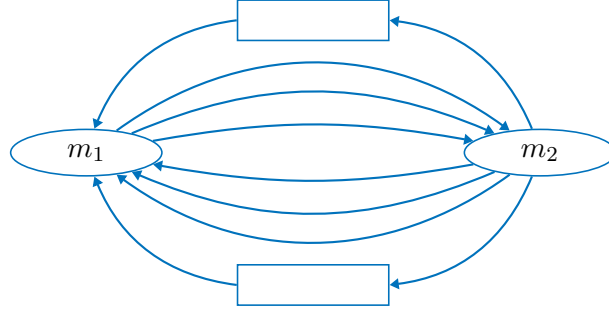


Figure 5.5.: Representing the Concept of ROS2 Actions with the Meta-Model ❖

Beneath the concept of Actions, there are further scenarios where two Members are connected by multiple Edges E_0 , e.g. a Node encompassing multiple Publishers or Subscribers to the same Topic. To enable such characteristics while maintaining unambiguity, each communication Edge $((s, d), 0, j \in \mathbb{N}_0) \in E_0$, with $s, d \in M_0$ is uniquely identified by its source s , destination d and index j . This unambiguity is mandatory, as it enables the unambiguous assignment of attribute values to communication Edges. It is important to note that a multigraph [56, p. 153] would also enable multiple Edges between two Members and thus enable the representation of the exact same topology. However, multigraphs do not keep track of the distinct identity of individual relationships represented by those Edges [56, p. 153].

Even though Topics and Services form the two fundamental mechanisms of communication within ROS2, they do not represent all correlations within a ROS2 system. To elaborate, Timers are not based on communication. However, Timers form one of four callback triggers within ROS2 and are the only callback trigger that is not inherently based on communication. Therefore, Timers are represented by a special case of sending Edges, reaching from an active Member to itself. This virtual self-communication is illustrated in Figure 5.6.



Figure 5.6.: Timers in G_0 represented by E_O ❖

Modelling Timers as virtual communication Edges, where a Node communicates with itself, may at first appear artificial, given that no actual data transmission occurs. However, in ROS2, Timers are registered as callback sources analogous to Subscribers. From a scheduling and

profiling perspective, Timer callbacks are handled by the same Executor infrastructure as communication callbacks and thus compete for the same computational resources. Therefore, this abstraction ensures that Timer-triggered executions are seamlessly integrated into the ROS2 communication Graph, enabling a structurally coherent analysis of ROS2 systems.

Definition 5.1.5 denotes the ROS2 communication Graph that is edge-induced by the communication Edges E_0 . Its existence is imperative as it represents the entire communication structure of the ROS2 system without additional, organising Edges. The ROS2 communication Graph is necessitated in the subsequent Definition 5.2.1.

The present section introduced a directed Graph to represent ROS2 systems. It categorised the entities of a ROS2 system into active and passive Members, established their communication, enabled its evolution, and added a temporal domain to represent time-dependent behaviour. Based on this Graph, the subsequent section introduces multiple types of aggregation to facilitate Fault Detection and Diagnosis.

5.2. Aggregating ROS2 Systems to Support Fault Detection and Diagnosis

To support Fault Detection and Diagnosis by facilitating the analysis of the ROS2 system in question, this section provides multiple forms of aggregation in order to organise the entities of the ROS2 communication Graph. These aggregations will be employed to total values and enable a broad view onto parts of ROS2 systems.

5.2.1. Creating Relations to Aggregate Member- and Edge-Specific Attributes

In order to represent the ROS2 system in question with a high degree of detail, multiple characteristics are assigned to the communication Edges rather than to a Member directly. For instance, the forwarding rate of a Topic is determined by accumulating the publishing rates of the adjacent publishing Edges. To accumulate those Edge-specific values and thus enable a structured analysis of local Edge properties, a relation is introduced that reaches from communication Members to communication Edges. To this end, the following definition introduces the sets of outgoing and incoming adjacent communication Edges with respect to a specific Member.

Definition 5.2.1: Creating Relations between Members M_0 and Edges E_0

Let $G_0 = (M_0, E_0)$ be a directed Graph as defined in Definition 5.1.5. We define the relation $R \subseteq M_0 \times E_0$, which lets us define ρ_{out} and ρ_{in} to denote outgoing and incoming adjacent Edges E_0 for each Member M_0 :

- $\rho_{out}(m) = \{((m, n), 0, j) \in E_0 \mid (m, ((m, n), 0, j)) \in R \wedge n \in M_0\}$ with $j \in \mathbb{N}$,
- $\rho_{in}(m) = \{((n, m), 0, j) \in E_0 \mid (m, ((n, m), 0, j)) \in R \wedge n \in M_0\}$ with $j \in \mathbb{N}$.

Building on the relation between Members M_0 and communication Edges, Definition 5.2.2 introduces a method for aggregating Edge-specific attribute values. This offers a more comprehensive perspective onto specific parts of the ROS2 system. Rather than undertaking an individual analysis of each Edge, it employs a summation of the attribute values associated with adjacent Edges to a particular Member. It is important to note that this aggregation is only enabled for attributes that are of numerical value.

Definition 5.2.2: Total Adjacent Edge-Related Attribute Values

For assessing the attributes of a Member and its adjacent Edges in T_i we denote the attribute functions $\Theta_{out} : M_i \times \mathbb{T} \rightarrow A$ and $\Theta_{in} : M_i \times \mathbb{T} \rightarrow A$ with $n \in \mathbb{N}$.

Θ_{out} and Θ_{in} are defined as follows:

- $\Theta_{out}(m, t) = \theta(m, t) + \sum_{e \in \rho_{out}(m)} (\theta(e, t))$, where $\theta(m, t), \theta(e, t) \in \mathbb{R}$,
- $\Theta_{in}(m, t) = \theta(m, t) + \sum_{e \in \rho_{in}(m)} (\theta(e, t))$, where $\theta(m, t), \theta(e, t) \in \mathbb{R}$.

Reflection

To evaluate the defined relations, two exemplary characteristics are introduced.

Firstly, publishing Edges exhibit a publishing rate, as they represent Publishers. As described in Section 4.2.1 and illustrated in Figure 4.5, these Publishers are connected to a specific Topic whose forwarding rate corresponds to the total of all belonging publishing rates. Instead of analysing each publishing rate directly to accumulate their values manually, the total forwarding rate of the Topic should be available directly.

Secondly, subscribing Edges and sending Edges trigger specific callbacks, which are executed subsequently. As each callback is executed for a certain time, its durations can be regarded as relative callback durations (by referring them to a time interval) that are assigned to the triggering communication Edge. Since each Node might encompass multiple callbacks, the total utilisation of all belonging callbacks might be of interest.

Definition 5.2.1 introduces the sets of outgoing or incoming communication Edges E_0 , denoted by ρ_{out} and ρ_{in} , respectively, related to a specific Member. The distinction between ρ_{out} and

ρ_{in} is imperative to prevent degenerated behaviour by maintaining unambiguity. In order to prove this statement, Figure 5.7 exemplarily illustrates two active Members that are connected by two sending Edges $E_{\uparrow}$. Ignoring the direction of Edges leads to distorted results, as totalling the attribute values of $\rho(m_1)$ component-wise would incorrectly include the relative callback durations caused by e_2 and executed by m_2 .

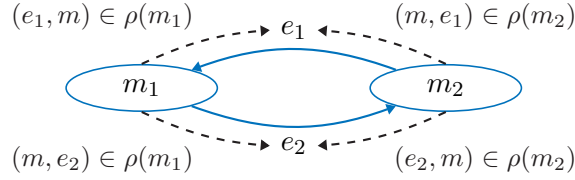


Figure 5.7.: Proof to Consider Edge Direction in order to Prevent Ambiguity ❖

Based on the proof that the sets ρ_{out} and ρ_{in} are justified in their existence, an exemplary topology is given in Figure 5.8. It illustrates relations R defined in Definition 5.2.1 reaching from an active Member to adjacent communication Edges. The communication Edges, that ρ_{out} and ρ_{in} relate to, are coloured in brown and green, respectively. In conjunction with the examples described previously, the brown Edge corresponds to a publishing Edge E_{Υ} , while the green Edges correspond to one subscribing Edge E_{λ} and one sending Edge $E_{\uparrow}$ each. A mixture is exemplarily given by the Timer $E_{\odot}$. It is evident that the Timer $E_{\odot}$ is associated as an outgoing and an incoming Edge.

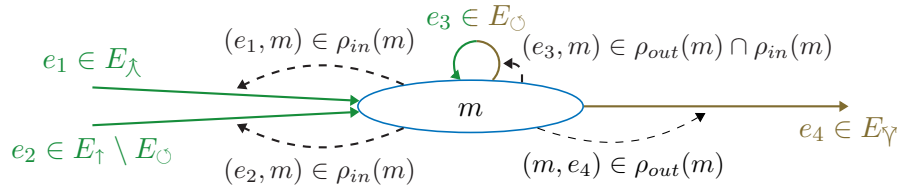


Figure 5.8.: Relation to Accumulate E_0 -Related Attributes ❖

With the topology established, Figure 5.9 illustrates the summation of Edge-related attribute values as denoted in Definition 5.2.2. It presents the superior attribute functions Θ_{in} and accumulates the values of the adjacent communication Edges e_1 to e_3 . As the characteristics of Members and Edges are disjoint, the corresponding values inside Θ_{out} and Θ_{in} are well-defined. This means that $\theta(e, t)$ has no value if a $\theta(m, t)$ contains one and thus adds 0, while $\theta(m, t)$ adds its attribute value and vice versa.

Exemplarily, A_{n-1} refers to the relative callback durations from the above example and thus accumulates within Θ_{in} . Analogously, the accumulated publishing rate of Publishers publishing to a specific Topic accumulates within the superior attributes function Θ_{in} of the associated

	$\theta(m, t)$	+	$\theta(e_1, t)$	+	$\theta(e_2, t)$	+	$\theta(e_3, t)$	+	$\Theta_{in}(m, t)$
A_0	10	+	0	+	0	+	0	+	10
A_1	13	+	0	+	0	+	0	+	13
...	...	+	...	+	...	+	...	+	...
A_{n-1}	0	+	0.3	+	0.05	+	0.3	+	0.65
A_n	0	+	20	+	10	+	1	+	31
	$\underbrace{\theta(m, t)}_{\theta(m, t)} + \underbrace{\sum_{e \in \rho_{in}(m)} (\theta(e, t))}_{\sum_{e \in \rho_{in}(m)} (\theta(e, t))} = \underbrace{\Theta_{in}(m, t)}_{\Theta_{in}(m, t)}$								

Figure 5.9.: Totaling Edge and Member-related attribute values

Topic. However, it is important to note that this fine-grained approach also enables further analyses, e.g. the total publishing rate a Node initiates.

The Definitions 5.2.1 and 5.2.2 thus facilitate the analysis of the ROS2 system in question by accumulating attribute values without losing the high degree of detail.

5.2.2. Creating Organisational Trees

Multiple concepts and structures of ROS2 systems follow the structure of a Tree, e.g. the namespace of a Node described in Section 4.1. Further concepts create a set of Trees that might be joined (by a virtual and thus passive Member) to create a superior Tree encompassing all Trees of a given set, e.g. the concept of Executors (described in Section 4.5). To represent those Trees within the directed Graph, the following definition denotes families of organisation Edges E_i , to extend G and construct edge-induced, rooted Trees.

Definition 5.2.3: Rooted Tree for Various Perspectives [55, p. 3]

Let $0 < i < N$ with $N \in \mathbb{N}_{>0}$, then E_i is defined as

$$E_i = (M \times M) \times \{i\}.$$

Whereas the edge-induced, directed Subgraph $T_i = (M_i, E'_i)$ forms a directed, rooted Tree, with

- $E'_i = \{(m, n) \mid ((m, n), i) \in E_i\}$, with $m, n \in M$,
- $M_i = \{m, n \in M \mid (m, n) \in E'_i \cup (n, m) \in E'_i\}$.

The root is represented by

$$\exists! r \in M_i \forall m \in M_i: (m, r) \notin E_i.$$

Reflection

Definition 5.2.3 enables the construction of edge-induced Trees T_i by creating additional relations between Members. Those hierarchies aggregate areas of the ROS2 system and will be utilised in the subsequent to total the attribute values of Members. Since the root of each Tree is defined as the unique Member without incoming adjacent Edges, the resulting Tree is well-defined. Each adjacent outgoing Edge connects a specific Member to one of its children, thus directing to the leaves of the respective Tree. To elaborate, multiple exemplary Trees are illustrated in the following. For the sake of clarity, only the edge-induced Trees are visualised.

The first exemplary, edge-induced Tree that can be created within the ROS2 Graph is outlined by the namespaces described in Section 4.1 and represented as a model in Figure 4.3. As namespaces organise Members and neither execute nor control any operations, they refer to the concept of passive Members. There are no constraints to the leaves of the Tree as they may either be Nodes or Topics and thus active or passive Members, respectively. An exemplarily, namespace-driven Tree is visualised in Figure 5.10. To clarify the example, the Members are filled with exemplary names. It is important to note that although multiple Members may have the same name, they are differentiated by their respective namespaces. Thus, the fully qualified name of the bottom outer right active Member is „turtle2/turtlesim“, while the second active Member called „turtlesim“ is identified as „turtle1/turtlesim“.

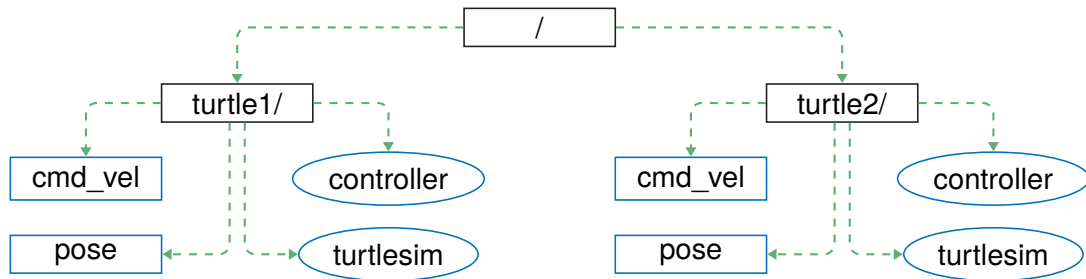


Figure 5.10.: Exemplary Namespace-Driven Tree ❖

The concept of Executors, as described in Section 4.5 and represented as a model in Figure 4.9, organises Nodes into groups and thus creates a set of Trees, whereas the root relates to an Executor and the leaves to the Nodes. The only inner Member² of each Tree refers to its root, representing the Executor itself. As illustrated in Figure 5.11, the set of Trees is joined by employing a passive Member as a virtual root in order to connect to the root of each Subtree for creating a superior Tree.

A further hierarchy constructs a process-driven Tree, illustrated in Figure 5.12. It is of particular interest, as it facilitates the assignment of Nodes to an operating system. To this end, a passive Member is created to establish a further level within this hierarchy. It connects to the roots of

²inner Member: Analogous to a classic „inner node“ but called „inner Member“ to avoid confusion.

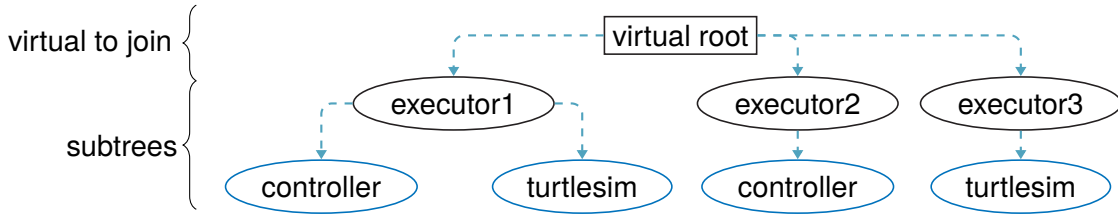


Figure 5.11.: Exemplary and Joined Executor-Driven Tree ❖

each (operating system specific) process-driven Tree³. Thus, it enables the analysis of active Members that belong to a specific operating system. Process-driven trees are elaborated in Appendix A.

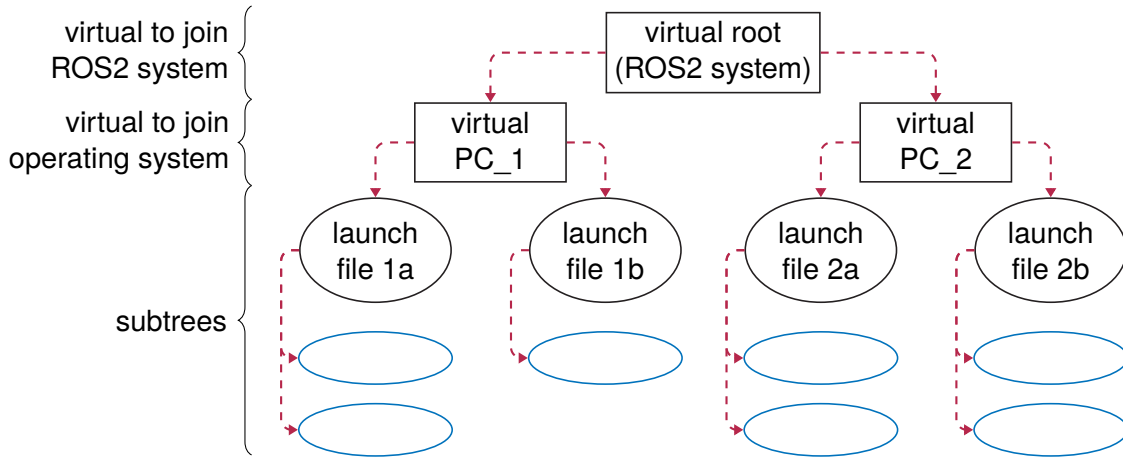


Figure 5.12.: Exemplary Process-Driven Tree (of the ROS2 system) ❖

The concept of Node Composition shapes a Container-driven Tree and is described within Appendix A. It is important to mention that not every Node has to be part of a Container and thus not every Node has to be part of a Container-driven Tree.

In summary, the creation of edge-induced Trees T_i enables further perspectives onto the ROS2 system in question by grouping Members within a hierarchical manner. In order to derive the maximum benefit from this grouping, the creation of a top-down approach is introduced in the following.

Aggregate Areas of a ROS2 System by Employing a Top-Down approach

Based on the previously defined Trees, a top-down approach is introduced to facilitate the analysis of ROS2 systems by totalling up the attribute values of multiple Members instead of analysing each one individually.

³To elaborate how operating system specific, process-driven Trees are created, see Appendix A.

Definition 5.2.4: Top-Down Approach to Analyse Directed, Rooted Trees T_i

Let $T_i = (M_i, E'_i)$ be a directed, rooted Tree as in Definition 5.2.3. Let $C_i(p) = \{m \in M_i \mid (p, m) \in E'_i\}$ denote the children of $p \in M_i$. Then the accumulated attribute functions $\Psi_{i,out}: M_i \times \mathbb{T} \rightarrow A$ and $\Psi_{i,in}: M_i \times \mathbb{T} \rightarrow A$ are recursively defined as

$$\Psi_{i,out}(p, t) = \sum_{c \in C_i(p)} (\Psi_{i,out}(c, t) + \Theta_{out}(c, t)),$$

$$\Psi_{i,in}(p, t) = \sum_{c \in C_i(p)} (\Psi_{i,in}(c, t) + \Theta_{in}(c, t)),$$

where the sum is evaluated component-wise on A , $t \in \mathbb{T}$ and $\Psi_{i,out}(p) = \Psi_{i,in}(p) = \vec{0}$, for all p with $C_i(p) = \emptyset$.

Reflection

Definition 5.2.4 introduces a top-down approach for analysing directed, rooted Trees T_i , enabling a comprehensive view of the system's behaviour without analysing each Member individually. This top-down approach propagates and accumulates edge-related properties along the hierarchical structure of T_i through a recursive traversal similar to a Depth-First Search. In this process, each Member totals up attribute values from its descendants by accumulating the recursively aggregated attribute values of its direct children. Those attribute values are represented through the attribute functions $\Psi_{i,out}$ and $\Psi_{i,in}$. To elaborate, Figure 5.13 illustrates a more extensive, exemplary single step of the totalling. Note the exemplary calculation that elaborates the summation.

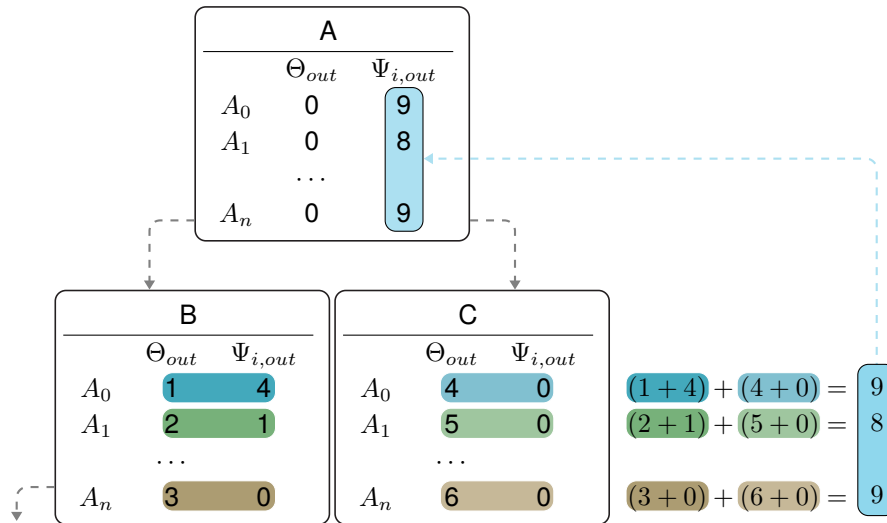


Figure 5.13.: Exemplary Single Step of Totalling $\diamond$

A broader example of this procedure is illustrated in Figure 5.14 for $\Psi_{i,out}$. It further illustrates that $p \in M_i$ can refer to any Member that is part of the edge-induced Tree T_i . This property is illustrated by the $\Psi_{3,out}$ column, which contains the respective values. Therefore, p is employed to increase the resolution onto specific parts of the system.

The entities with wide rounded corners refer to active Members.

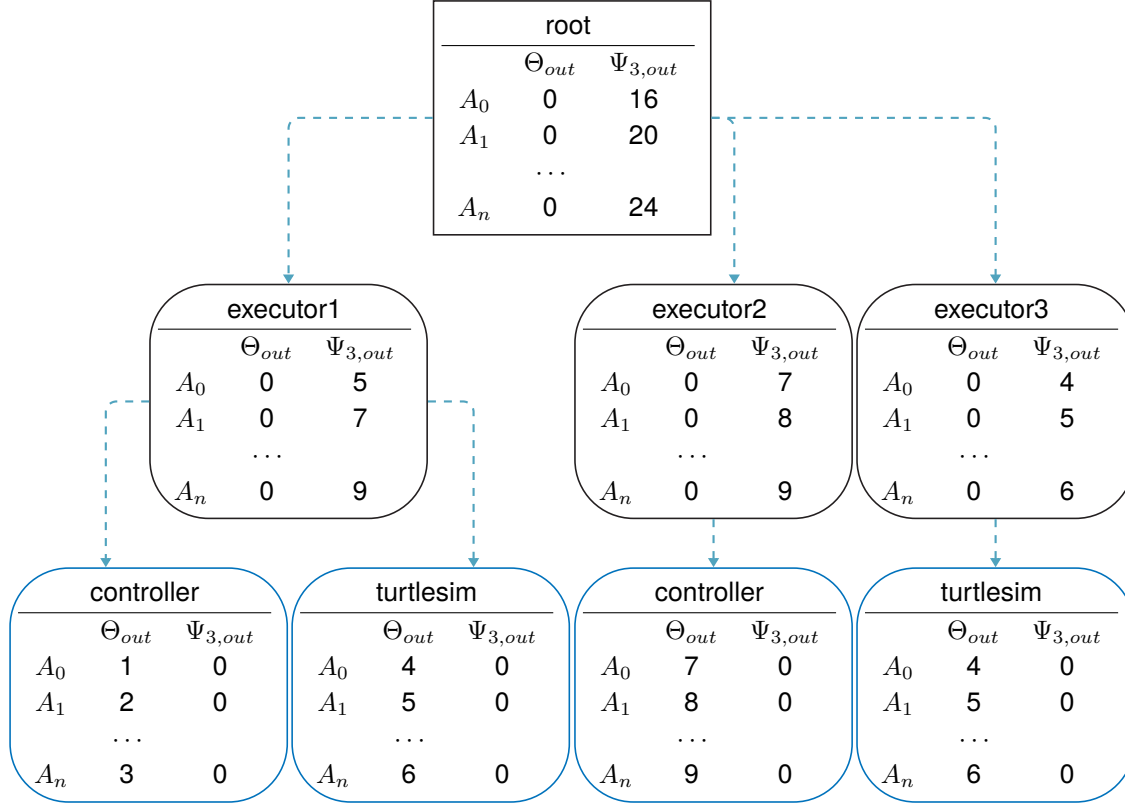


Figure 5.14.: Exemplary Top-Down Aggregation of the Executor-Driven Tree for $\Psi_{3,out}$ 

As a result, this mechanism enables a holistic view by abstracting Members into groups and totalling their attribute values, offering a broad analysis of the ROS2 system.

5.2.3. Creating Subtrees and Adding Binary Operations for Finer Aggregation

As the in Definition 5.2.3 described directed, rooted Trees organise and group Members of the ROS2 system, they enable a comprehensive view onto parts of the ROS2 system. In order to accumulate more specific parts, binary operations might be applied to the Members of respective Trees. For an even finer granularly selection, the following definition introduces the creation of Subtrees. Note that a Tree is also enabled to be its own Subtree.

Definition 5.2.5: Extracting Subtrees for a Fine-Grained Perspective

Given a directed, rooted Tree $T_i = (M_i, E'_i)$ and a Member $m \in M_i$, the directed Subtree rooted at m is defined as

$$T_{i,m} = (M_{i,m}, E'_{i,m}),$$

with

- $M_{i,m} = \{m\} \cup \{u \in M_i \mid m \rightsquigarrow u\}^\clubsuit$ and
- $E'_{i,m} = \{(v, w) \in E'_i \mid v, w \in M_{i,m}\}.$

$\clubsuit a \rightsquigarrow b$: there exists a directed path from a to b [57, p. 787]

With the existence of Subtrees, the following definition denotes multiple binary operations to be applied on Subtrees.

Definition 5.2.6: Binary Operations on Trees

Let $T_i = (M_i, E_i)$ and $T_j = (M_j, E_j)$ be directed, rooted Trees like defined in Definition 5.2.3 and let $m \in M_i$ and $n \in M_j$. Then the corresponding Subtrees $T_{i,m} = (M_{i,m}, E_{i,m})$ and $T_{j,n} = (M_{j,n}, E_{j,n})$ are rooted at m and n as defined in Definition 5.2.5.

Hence, the following operations on Member sets of Subtrees are defined:

- Intersection: $M_{(i,m) \cap (j,n)} = M_{i,m} \cap M_{j,n},$
- Union: $M_{(i,m) \cup (j,n)} = M_{i,m} \cup M_{j,n},$
- Difference: $M_{(i,m) \setminus (j,n)} = M_{i,m} \setminus M_{j,n}.$

With the created sets of Members, the following definition denotes how the attribute values of collected Members are totalled.

Definition 5.2.7: Accumulation of Binary Aggregated Subsets

Let $\otimes$ be the abstract binary operation to represent one of the concrete operations highlighted in Definition 5.2.6:

$$\otimes \in \{\cap, \cup, \setminus\}$$

and $t \in \mathbb{T}$. Then the attribute functions $\Upsilon_{out}: M_{(i,m) \otimes (j,n)} \times \mathbb{T} \rightarrow A$ and $\Upsilon_{in}: M_{(i,m) \otimes (j,n)} \times \mathbb{T} \rightarrow A$ are defined as

$$\begin{aligned} \Upsilon_{out}(M_{(i,m) \otimes (j,n)}, t) &= \sum_{w \in M_{(i,m) \otimes (j,n)}} \Theta_{out}(w, t) \text{ and} \\ \Upsilon_{in}(M_{(i,m) \otimes (j,n)}, t) &= \sum_{w \in M_{(i,m) \otimes (j,n)}} \Theta_{in}(w, t). \end{aligned}$$

Reflection

In order to facilitate the analysis of specific parts of a ROS2 system, Definition 5.2.5 introduces Subtrees to select specific sections. Based on those Subtrees, Definition 5.2.6 denotes three binary operations to analyse Trees with more detail by creating the A) intersection, B) union, and C) difference. Definition 5.2.7 totals the Member attribute values of the given set. As the attribute functions Θ_{out} and Θ_{in} consider either outgoing or incoming Edges, respectively, the corresponding attribute functions Υ_{out} and Υ_{in} inherit those characteristics and thus also the characteristics of unambiguity. All binary operations are exemplarily illustrated in Figure 5.15, whereas the gray dashed lines represent the Tree T_i while the purple dashed lines represent the Tree T_j . The union (I) is induced by $\Upsilon(M_{(i,m_1) \cup (i,m_2)}, t)$. The intersection (II) is induced by $\Upsilon(M_{(i,m_1) \cap (j,m_4)}, t)$. The difference (III) is induced by $\Upsilon(M_{(i,m_2) \setminus (j,m_3)}, t)$.

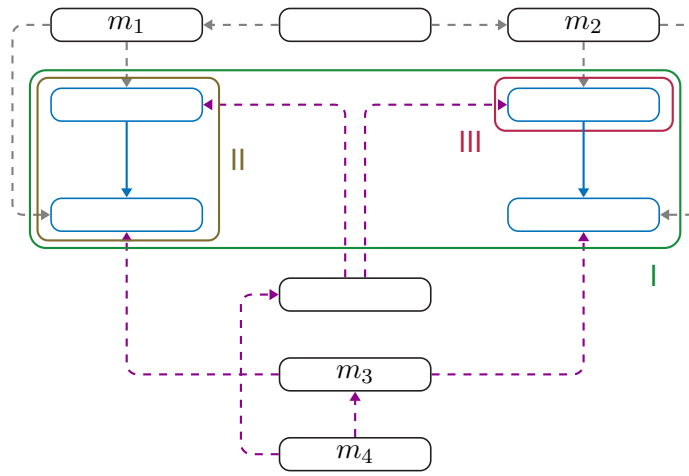


Figure 5.15.: Exemplary Binary Operations $\diamond\diamond$

5.3. Discussing the Mathematical Meta-Model: Capabilities and Challenges

This chapter introduced a mathematical meta-model that abstracts the introduced ROS2 model in order to represent the fundamentals of ROS2 systems by providing active Members and passive Members to reflect various entities of ROS2 systems. As the Members of the Graph are abstract and not dedicated exclusively to, e.g., Nodes or Topics, they enable the integration of ROS2 system specific entities or information, e.g., battery level information. Such data can be integrated into the model by creating and maintaining a respective instance, without being a fundamental component of ROS2 systems.

Furthermore, the chapter introduced multiple communication Edges to represent the overall communication structure of ROS2 systems alongside the concept of timers. The latter are represented as virtual communication Edges and thus a part of the ROS2 communication Graph. To facilitate Fault Detection and Diagnosis, the introduced mathematical meta-model enables the construction of edge-induced Trees and thus creates additional perspectives onto the ROS2

system in question. Finally, three types of aggregation are introduced to facilitate the analysis of system areas. It is imperative to acknowledge the significance of those aggregations, as they enable a broad analysis of the ROS2 system in question without necessitating expert knowledge. For instance, the root of an arbitrary Tree can be observed, employing Ψ_{out} or Ψ_{in} , in order to gather a blurred but broad analysis of the ROS2 system which might be refined on suspicious behaviour. Moreover, the introduced meta-model enables the allocation of all attributes to their respective origins, thereby enabling a precise and detailed analysis.

Alongside the forms of aggregation, the mathematical meta-model also enables time-series by employing various attribute values through the temporal domain. For constant attributes, the attribute values remain constant for each timestamp. However, varying attribute values represent a time-series that may be analysed at a later point in time. Importantly, this property enables the analysis of time-dependent behaviour and statistical evaluation by calculating, e.g., the variance or the arithmetic mean.

Additionally, as there are no restrictions to the attributes, the content of the messages is also representable by the mathematical meta-model, enabling their analysis.

The time-series are separated from the structural information that are represented by the directed Graph and the edge-induced Trees. This separation enables a flexible analysis of the ROS2 system, as the time-series can be evaluated independently of the structural information.

Despite the variety of concepts that the mathematical meta-model is able to represent, and the multiple types of aggregation that facilitate the analysis of a ROS2 system in question, it does not adequately capture callback groups. As illustrated in Figure 5.16, callback groups are subordinate groups of an Executor and represent organisational entities and thus passive Members. They necessitate a relation that reaches from a passive Member, for which applies $m_P \notin G_0$, to a sending or subscribing Edge. Even though Definition 5.2.1 defines similar

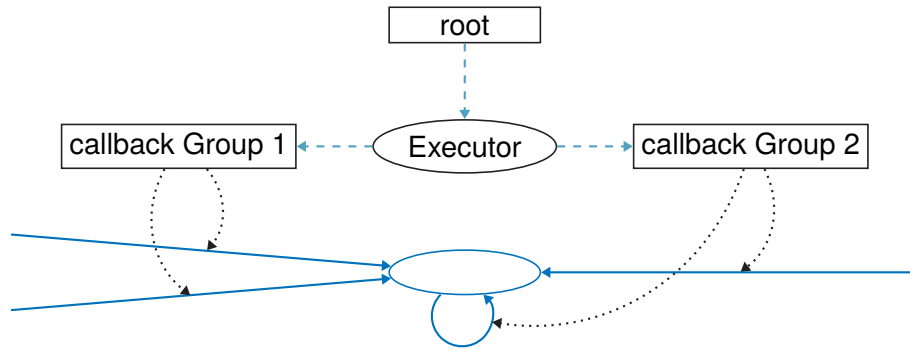


Figure 5.16.: Aggregating Values by Callback Groups $\diamond$

relations, they reach from an arbitrary communication Member to its adjacent communication Edges. This excludes relations from Members that are not part of the ROS2 communication Graph, as well as relations from Members M_0 to non-adjacent Edges. The Edges that would be necessitated but are not established are visualised as dotted lines within Figure 5.16.

In summary, this chapter abstracts the introduced ROS2 model to derive a mathematical meta-model that supports Fault Detection and Diagnosis.

6. Employing the Mathematical Meta-Model to create a ROS2 System Digital Shadow in Support of Fault Detection and Diagnosis

Based on the mathematical meta-model, the present chapter introduces a Digital Shadow for ROS2 systems that supports Fault Detection and Diagnosis. For this purpose, multiple requirements are described within Section 6.1 before Section 6.2 refines the conceptual model for Digital Shadows, outlined by Becker et al. [30] (Section 3.2.1), to the objective of this thesis. Section 6.3 introduces a logical model for the ROS2 Digital Shadow.

6.1. Requirements for a Shadow Caster Targeting ROS2 Systems

Prior to the refinement of the conceptual model in the subsequent section, this section presents multiple requirements that the Shadow Caster shall fulfil to meet the objectives of this thesis.

First and foremost, it is imperative to enable the representation of the mathematical meta-model, since this will enable to capture the structure of the ROS2 system in question, whilst also enabling the integration of ROS2 system specific information.

Further requirements are associated with communication. In this context, the Shadow Caster is intended to operate in an event-driven manner in two key aspects. Firstly, the collected data is limited to the necessitated data – e.g., structural data and data that has been explicitly requested by clients. Secondly, clients should only receive data that they have explicitly requested.

These requests are intended to provide insights without requiring expert knowledge about the ROS2 system. In order to meet this objective, standard requests, which reflect the attribute functions defined within the mathematical model, are made available. Given the potentially large number of additional, possibly system specific analyses, the responsibility for custom requests lies with the client. Consequently, the Shadow Caster must provide a mechanism for these custom requests.

A further distinction is made regarding the execution frequency of requests. Clients should be enabled to perform both single-time requests and continuous requests, where the latter are executed at a given frequency. To minimise the number of transmitted messages, event-based messages – such as status changes of Nodes – are also to be supported.

6.2. Refining the Conceptual Model for Digital Shadows to ROS2 Systems

Based on the conceptual model for Digital Shadows described in Section 3.2.1, this work refines and applies the conceptual model to the context of ROS2 systems.

This refined conceptual model is illustrated in Figure 6.1. For this thesis, the *Asset* is represented by the ROS2 system. Inheriting from the *Source*, the ROS2 system also serves as the Source the *DataTracers* originate from. The *DataTracers* form the Digital Shadow by tracing structural information to form the ROS2 Graph and capturing continuous information about the entities within the ROS2 Graph. For clarity, the *DataTracers* are introduced in Tables 6.1 and 6.2 and described in the subsequent. The *Model* is represented by the introduced mathematical meta-model. It serves as a blueprint for the Digital Shadow and is guided by the *Purpose* of supporting Fault Detection and Diagnosis within ROS2 systems.

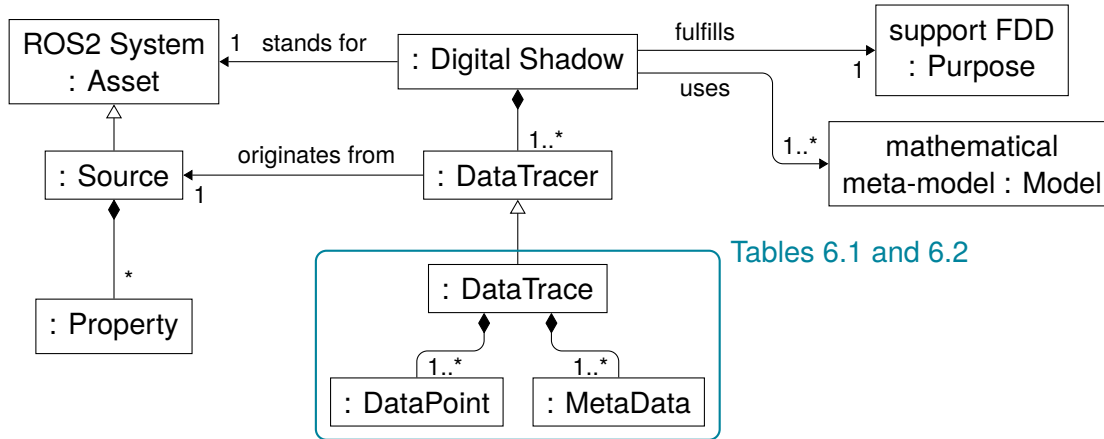


Figure 6.1.: Conceptual Model for Digital Shadow Targeting ROS2 Systems based on Becker et al. [30]

As listed in Table 6.1, there are multiple *DataTracers* to create the structure of a Digital Shadow for a ROS2 system. The *DataTracers* strictly orient at the mathematical meta-model. Even though it might be confusing that the *DataPoints* of active and passive Members are this generic, they enable a variety of information to be stored. This is imperative, as it enables the unrestricted creation of Members and thus facilitates the extension of the ROS2 system model. For instance, it enables the construction of multiple, also to the present unknown, Trees within the ROS2 Graph. With exception of the Tree-forming Edges E_i , which do not contain attributes, each element is identified by a unique identifier (*UID*), enabling the unambiguous selection and assignment to its continuous information. As the mathematical meta-model does not enable attributes for Tree-forming Edges E_i and they are uniquely identified by their adjacent Members and index i , a UID is not required.

Table 6.1.: DataTracers to Construct Structural ROS2 Graph

DataTracer	DataPoint	MetaData
Active Member	$\{\text{specificInfo}\}$	UID
Passive Member	$\{\text{specificInfo}\}$	UID
Publisher Init	activeMember.UID, passiveMember.UID, msgType	UID
Subscriber Init	passiveMember.UID, activeMember.UID, msgType	UID
Server Init	activeMember.UID, serverName, msgType	UID
Client Init	activeMember.UID, serverName, msgType	UID
Timer Init	activeMember.UID, frequency	UID
Edges E_i Init	*member.UID, *member.UID, i	—

All attributes of the mathematical meta-model are provided by a time-dependent component. Consequently, all attribute values are identified as continuous values. Within the conceptual model, the continuous attributes relate to continuous information. In order to accumulate continuous information about the ROS2 system in question, the DataTracers in Table 6.2 describe generic forms of data acquisition capable of capturing a wide range of runtime information. This includes utilisations, service calls, state changes, logs, etc.. Its generic design ensures adaptability and applicability across various requirements and use cases. It also enables the integration of non-ROS2 system information. Each attribute value is assigned to a specific entity via its UID, while the attribute and timestamp describe the content and time of measurement.

Table 6.2.: DataTracers to Capture Continuous Information

DataTracer	DataPoint	MetaData
Attributes Value	value (numerical)	attribute, timestamp, *.UID
Attributes Value	value (textual)	attribute, timestamp, *.UID

Discussing the Refined Conceptual Model for the Digital Shadow Targeting ROS2 Systems

The present section refined the conceptual model for Digital Shadows proposed by Becker et al. [30]. In contrast to the utilisation of, e.g., structural or behavioural diagrams, as suggested by Becker et al. [30], this methodology employs the defined mathematical meta-model. The rationale for this is the objective of the Digital Shadow to represent a broad range of ROS2 systems rather than specific implementations. Moreover, as the mathematical meta-model is abstract, it enables the integration of system specific information that is beyond the concepts of ROS2. For instance, battery level information.

The DataTracers of the conceptual model strictly adhere to the mathematical meta-model to ensure consistency. Therefore, they are designed to be generic in order to capture a wide range of runtime information while facilitating the integration of system specific information. The strict separation of structural and continuous values supports efficient storage, as optimal tools can be applied for each type in subsequent stages. Moreover, as will be detailed later, this facilitates a precise and non-redundant evaluation of values through clear queries and structured analysis. This is further supported through the utilisation of unique identifiers, enabling a uniform and unambiguous analysis.

The subsequent section builds upon this refined conceptual model to derive a corresponding logical model.

6.3. Logical Model to Generate Shadow Instances

Based on the refined conceptual model for a Digital Shadow targeting ROS2 systems, this section introduces a logical model to generate a concrete Shadow Instance. It is based on both the conceptual model and the workflow for creating Shadow Instances (Section 3.2.2).

The core components of the logical model are introduced alongside their roles and interactions in Sections 6.3.1 and 6.3.2, and **emphasized** throughout the thesis. Subsequently, multiple exemplary processes are described in Section 6.3.3. A detailed explanation on how the logical model meets the Digital Shadows requirements is discussed in Section 6.3.4.

6.3.1. Representing Shadow Instances

The Shadow Instance relies on three primary databases (Figure 6.2) to represent the ROS2 system. A **graph database** captures the structural relations within the ROS2 system and thus represents the ROS2 Graph. The primary keys it defines serve as UIDs to distinguish each element.

Continuous information is stored within a **time-series database** and a **log database**, storing numerical and textual information, respectively.

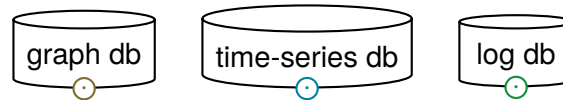


Figure 6.2.: Representing the Shadow Instance with three Databases

The communication with databases is illustrated through coloured $\odot$ and $\otimes$, representing the direction of communication¹.

¹For instance: $\odot \leftarrow \otimes$ or $\odot \leftrightarrow \odot$.

The Shadow Instance originates from the DataTracers, is shaped by the mathematical meta-model, and generated by the Shadow Caster.

6.3.2. Creating the Shadow Caster

The Shadow Caster generates the Shadow Instance with information originating from the ROS2 system. For the sake of clarity, the components and their relations are illustrated in Figure 6.3.

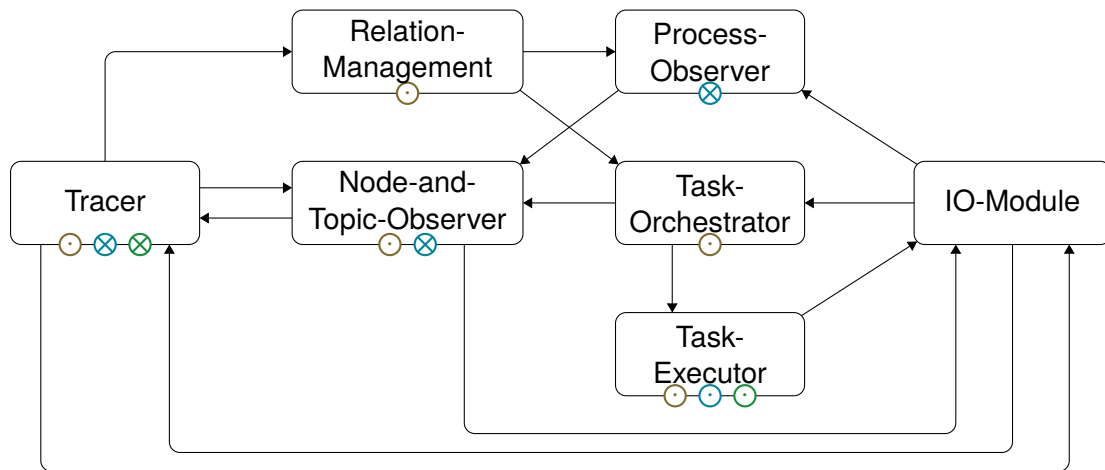


Figure 6.3.: Logical Model of Shadow Caster Targeting ROS2 Systems

The following components are responsible for collecting and processing data from various sub-sources.

Tracer

As first part of the Shadow Caster, the **Tracer** acts as a central endpoint for incoming data from multiple subsources. It collects, preprocesses, and forwards data to further Shadow Caster modules. For instance, it forwards Node and Topic information, and information about organisation Edges that serve the aggregation of the ROS2 system, to the **Relation-Management** and all types of information that belong to the ROS2 communication Graph to the **Node-and-Topic-Observer**. Furthermore, it populates the **time-series database** and the **log database** with continuous information. To this end, it obtains the UID of each entity either from the **Node-and-Topic-Observer** – when instantiating entities of the ROS2 communication Graph – or directly from the **graph database**. This enables the **Tracer** to accurately assign incoming values to their corresponding UID.

Additionally, it can be employed to initiate the integration of external information that is not covered by the fundamental ROS2 system. To this end, it directly creates an entity within the ROS2 Graph and thus within the **graph database**. In return, the **Tracer** receives

the UID of the entity to populate the **time-series database**.

To also consider the message content of a ROS2 system, the **Tracer** is enabled to directly forward ROS2 messages to the **IO-Module** and store them within the **log database**. Even though structural information are mandatory to generate a holistic perspective onto the ROS2 system, continuous information are not always of interest and may be enabled or disabled during runtime, thereby minimising generated overhead to a bare, necessitated minimum. This toggling process is instructed by the **IO-Module**.

Relation-Management

The **Relation-Management** exclusively receives information by the **Tracer**. It is responsible for establishing additional relationships that are beyond the ROS2 communication Graph. Therefore, it queries the **graph database** to construct and manage the Trees introduced in Section 5.2.2. As all organisation Edges within the ROS2 Graph are passed through or initiated by the **Relation-Management**, it sends a trigger to the **Task-Orchestrator**, that updates its aggregations. When process-related information is involved, it also forwards these entities alongside the belonging UIDs to the **Process-Observer** to initiate further monitoring.

Process-Observer

The **Process-Observer** monitors runtime information about all processes associated with the ROS2 system the Shadow Caster is operating on. Therefore, it is a special case that serves both as a data Source for the Asset and as a component of the Shadow Caster, as it also directly contributes data to the **time-series database**. While the monitoring of process related information may be enabled or disabled by the **IO-Module**, the **Process-Observer** always keeps track of the process alive states. Upon detecting terminated processes, it infers the death of associated instances and forwards this information, alongside the belonging UID and timestamp, to the **Node-and-Topic-Observer**.

Node-and-Topic-Observer

The **Node-and-Topic-Observer** fulfills two primary functions. Firstly, managing Nodes, Topics, and their communicational relations within the **graph database** by employing the information received from the **Tracer** and the **Process-Observer**. It returns the UID of instantiated entities to the **Tracer** and populates, e.g., state changes to the **time-series database**.

Secondly, it forwards requested events, such as state changes, to the **IO-Module**. The number of requested events may be extended or retracted by the **Task-Orchestrator**.

IO-Module

The **IO-Module** acts as the interface to a requesting client. It accepts client requests and forwards them to further components of the Shadow Caster – specifically the **Task-Orchestrator** – to fulfill client requests. Vice versa, it forwards the requested information to the respective client.

Task-Orchestrator

The **Task-Orchestrator** is employed by the **IO-Module** for all client requests and captures what each client requested. For requests that aim at system areas, it queries the **graph database** to extract the requested entities. However, it forwards a single UID or the collected UIDs alongside a requested task to the **Task-Executor**. A special case are event-driven requests that relate to the ROS2 communication Graph (like state changes of Nodes), as such events are managed by the **Node-and-Topic-Observer**. Therefore the **Task-Orchestrator** forwards the collected UIDs to the **Node-and-Topic-Observer**.

Additionally, the **Task-Orchestrator** is informed by the **Relation-Management** about updates within the system topology. On updates, it requeries the **graph database** to update the requested entities and thus notifies the **Task-Executor** and the **Node-and-Topic-Observer**, which adapt their tasks accordingly.

Task-Executor

The **Task-Executor** receives specific tasks and belonging UIDs by the **Task-Orchestrator**. It is enabled to perform four types of tasks, namely A) *single-time standard requests*, B) *continuous standard requests*, C) *single-time custom requests*, and D) *continuous custom requests*. It is important to mention, that A) and B) are implemented tasks that encompass standard requests. For instance, all types of summations that were defined within the mathematical meta-model or information requests about entities within the ROS2 Graph. Conversely, the tasks C) and D) relate to custom tasks that are requested by a client and employ the query language of a database to gain information. Thus they may be utilised to, e.g., analyse an entire time-series or search for the shortest path between two entities. It is important to note that those queries are read-only to prevent the manipulation of the database. Additionally, only non empty database responses are forwarded to the requesting client, who is thus enabled to create a custom, event-driven request that improves system efficiency by minimising data transmission.

However, single-time standard- and single-time custom requests are performed a single time and their result is forwarded to the **IO-Module**.

Continuous standard- and continuous custom-requests are performed at a specific time interval and their result is forwarded to the **IO-Module**.

Integrating Domain Knowledge into the Shadow Caster

The so far developed logical model is enabled to reconstruct a ROS2 system without requiring expert knowledge. However, it does not enable the integration of domain knowledge in form of additional models. In this context, domain knowledge may refer to information about existing Nodes, Edges, and reference attribute values for various instances. This knowledge is integrated through a configuration mechanism in form of textualised Requirement-Diagrams.

Figure 6.4 illustrates the entire, extended logical model. All modules marked with the configuration symbol  can be configured accordingly. Note the new module **Evaluator**, that employs the **graph-** and **time-series database** alongside the **configuration** to assess how close the system matches the specified configuration. Initiated by the IO-Module it identifies significant deviations and quantifies their magnitude. Thereby this model supports Fault Detection and Diagnosis especially in terms of Fault Detection.

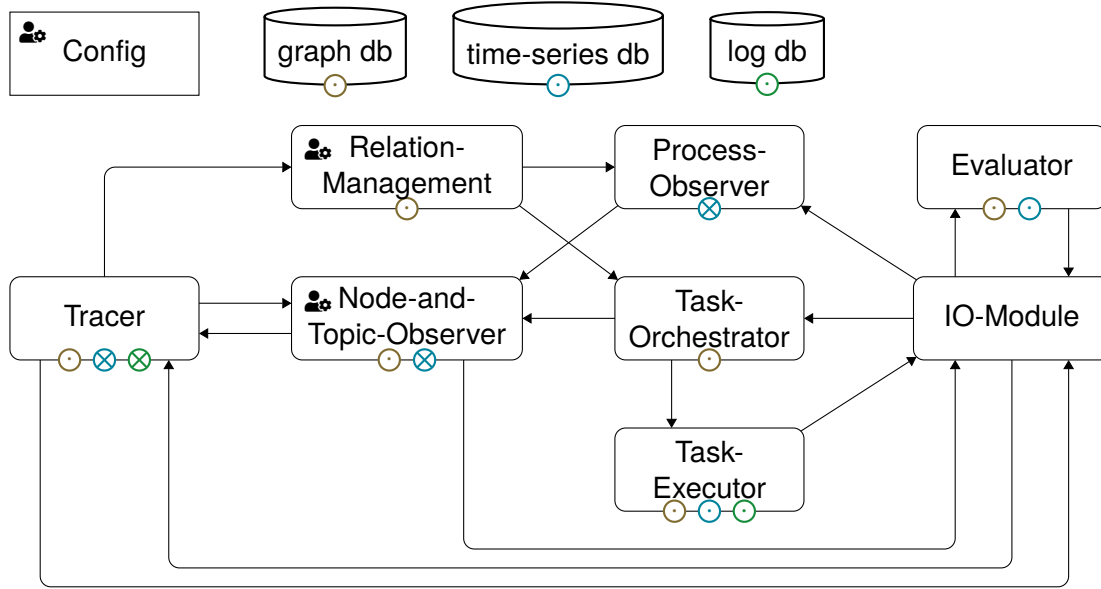


Figure 6.4.: Extending the Logical Model to enable the Integration of Domain Knowledge

The present section elaborated the components of the logical model for the Shadow Caster.

6.3.3. Exemplary Interactions with the Shadow Caster

To elaborate and strengthen the illustration of the logical model, this section provides various exemplary scenarios that might occur during runtime.

Launching a Node

To properly represent the ROS2 system, the launch of Nodes needs to be processed correctly. Therefore, the first example considers the launch of a Node.

Firstly, the **Tracer** receives the data about a new Node that has been launched. This information includes, e.g., the name, namespace, and process of the Node alongside a timestamp. It preprocesses the received data and forwards the gained information to the **Relation-Management** and the **Node-and-Topic-Observer**.

The **Relation-Management** processes the namespace and thus identifies the position of the Node within the namespace-driven Tree. Additionally, it analyses the position of the Node within the process-driven Tree employing process information. The gathered information of both Trees is queried to the **graph database** in return for the UIDs. It is imperative to note that the **Relation-Management** only infiltrates quantitative information and only creates non-existing entities.

As the gathered information contains process-specific data, it forwards exactly this data alongside the corresponding UIDs to the **Process-Observer**.

The **Process-Observer** monitors the alive state of the processes and, when requested, also populates process-specific information to the **time-series database**. If a process disappears, it notifies the **Node-and-Topic-Observer**.

The **Node-and-Topic-Observer** receives the information from the **Tracer** and thus populates the **graph database**. It only adds qualitative information to the **graph database** and also only creates non-existing entities. Additionally, the **Node-and-Topic-Observer** populates the state of the Node to the **time-series database**.

As the Node is launched initially, the **Node-and-Topic-Observer** sends its information alongside the UID to the **Tracer**. This enables the **Tracer** to populate the **time-series database** with continuous information about the launched Node.

Upon receiving a notification regarding a status change of the Node, the **Node-and-Topic-Observer** verifies the relevance of the information and proceeds to update the **graph database** accordingly. In any case, the status change is populated to the **time-series database** with its associated timestamp.

Requesting Single-Time Information

The second example illustrates the process of requesting single-time attribute values, whereas an area of the ROS2 system is requested. The **IO-Module** forwards the request to the **Task-Orchestrator**, which queries the **graph database** for the requested area. In return, the **Task-Orchestrator** receives the UIDs of the involved entities. Subsequently, it forwards the collected UIDs to the **Task-Executor**, which queries the **time-series** or **log database** to retrieve the requested information. If the attribute values originate from the **time-series database**, the attribute values of the information are totalled and the result is returned to the **IO-Module**, which forwards the totalled attribute value to the requesting client. If the attribute values originate from the **log database**, they are compressed and forwarded to the **IO-Module**.

If the request is of a continuous nature, the same behaviour would be observed, with the exception that the **Task-Executor** would query the **time-series database** at regular intervals and forward the result accordingly.

Integrating an External Data Source

As the Digital Shadow is limited to the information that is provided by the ROS2 system, it provides the option of integrating external information. This is achieved by employing the **Tracer**, which creates a new entity within the **graph database** and thus assigns the source to a specific UID. Subsequently, the **Tracer** populates the **time-series database** with the information provided by the external data source.

6.3.4. Discussing the Logical Model for the Shadow Caster

The present section refined the conceptual model and introduced the logical model for the Digital Shadow targeting ROS2 systems to facilitate Fault Detection and Diagnosis. It is based on the refined conceptual model and the workflow for creating Shadow Instances [30]. Due to the numerous tasks of receiving, organising, and providing the given information, the logical model is modular and thus consists of multiple components, whereas each is dedicated with a specific task. This modularity facilitates the concurrent execution of those tasks and enhances scalability. However, it also increases communication overhead and system complexity, especially due to asynchronous communication. The former can be addressed within the implementation. To target latter one each component is assigned with one specific, unique task. For instance, the information that the **Node-and-Topic-Observer** and the **Relation-Management** populate the **graph database** with, do not intersect. The only intersection is driven by the population of the **time-series database** that is carried out by the **Tracer**, the **Node-and-Topic-Observer**, and the **Process-Observer**, as they are enabled to populate the same time-series. However, as their contributions to the **time-series database** are disjointed and thus differ by their assignment, this risk exists but is unlikely to occur. Outsourcing the **Process-Observer** would solve this drawback, but require another instance, beside the introduced logical model, to take over its essential task. In principle, however, it is possible.

By employing three types of databases, the logical model is enabled to fully represent the mathematical meta-model. Thereby strictly separating structural and continuous information, exactly as the mathematical meta-model implied. The **graph database** is employed to represent the ROS2 Graph and thus the structural information, while the **time-series database** and the **log database** are utilised to store numerical and textual information, respectively. This separation is crucial to ensure efficient storage of information, as optimal tools can be selected for each type. Moreover, it facilitates a precise and non-redundant evaluation of values through clear queries and structured analysis, thereby considering temporal analyses and the analysis of system areas.

The logical model provides multiple standard analyses that can be applied to the time-series, e.g. the summations defined within the mathematical meta-model. However, since the procedures for analysing the recorded information are far-reaching and may even be dedicated to

a specific ROS2 system, the logical model enables the execution of external analyses, strictly spoken database queries. Related to the **time-series database**, the queries may analyse a temporal sequence, while the **log database** may be queried for specific keywords or types of logs. Additionally, the **graph database** enables the analyses of the ROS2 systems topology.

As the attribute values are assigned to a dedicated UID, attribute, and timestamp, all continuous information are of an equal rank. Therefore, the logical model enables a seamless integration of external and ROS2 system specific information.

Attribute values that belong to entities that are executed on further operating systems and belong to the same ROS2 system are also integrated through the **Tracer**. Thus, a single source of truth is created, preventing redundancy and facilitating holistic analyses, as all information is stored within the same respective databases.

The utilisation of UIDs for all entities within the Shadow Instance enables a uniform and unambiguous analysis.

Besides the topological and continuous information about the entities within the ROS2 system, the logical model also considers message contents. It is imperative to consider this objective, especially in perspective of Fault Detection and Diagnosis, as sensor failure may cause serious difficulties during runtime, which are independent of faults within the software stack. Due to the extensive range of ROS2 messages and especially the possibility of custom messages, it is impractical to preprocess their content through the Shadow Caster. Therefore, the content is directly forwarded to the **IO-Module**. To facilitate the subsequent assignment of message content to specific circumstances, the message contents can be stored in the **log database**. However, due to communication overhead, this approach becomes unsuitable when the volume or size of messages increases. Especially as rosbags [58] already present a method of storing message content.

Based on the introduced logical model, a proof-of-concept is implemented and evaluated in the subsequent section.

7. Evaluation

In order to evaluate the introduced mathematical meta-model and assess the impact of the Digital Shadow on ROS2 platforms, this chapter addresses various aspects, ranging from the feasibility of generating system specific models to utilisation tests under varying conditions. To this end, a proof-of-concept, which is based on the logical model presented in Section 6.3, is implemented¹.

Section 7.1 briefly describes the frameworks and tools that are employed to implement the introduced approach. To align the evaluation with the objectives defined in Chapter 1, Sections 7.2 to 7.4 address the objectives referring to the introduced meta-model. To elaborate, the meta-model aims to be applicable to a wide range of ROS2 systems, while enabling the integration of system specific information and creating holistic perspectives on the ROS2 system. Additionally, it is intended to accurately represent the dynamic nature of the running system. Section 7.2 illustrates the exemplary generation of a ROS2 system specific Shadow Instance. Section 7.3 describes challenges of the mathematical meta-model that predominantly originate from the employed tools, before constraints are derived from these challenges in Section 7.4.

The subsequent Sections 7.5 and 7.6 address the objectives referring to the introduced Digital Shadow, which aims at minimising expert knowledge and the incurred performance overhead while being non-intrusive in order to keep the software stack and codebase clean. Therefore, Section 7.5 explores the performance of the Digital Shadow with respect to the generation of ROS2 system models. The overhead incurred due to processing requests, which gather information about the generated ROS2 system model, is explored in Section 7.6.

To further validate the applicability to real-world scenarios, Section 7.7 explores the generation of a ROS2 system model within a real-world robot scenario.

In order to emphasise the feasibility of the mathematical meta-model, this chapter aligns with the legend defined in Chapter 5. However, Members are now illustrated as visualised in Figure 7.1. Note that Members of the ROS2 communication Graph are underlined. All figures that adhere to the updated legend are labelled with . Analogous to ,  marks figures that adhere, but deviate from the legend in order to highlight specific characteristics.

¹The implementation can be found at: <https://github.com/KARTOFF8xE/STREAM-DSM>.

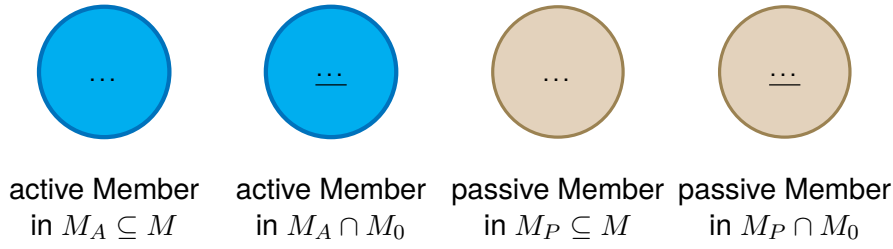


Figure 7.1.: Partially Updated Legend ♣

7.1. Employed Tools and Implementation

This section gives a brief overview of the tools employed in the proof-of-concept. Due to the focus of this thesis, it is implemented for ROS2.

ROS2 Version

In order to generate a ROS2 system specific model that represents the communication structure of the system, the tracepoints within the ROS2 source code are employed. As new tracepoints are exclusively added to the development version of ROS2 – rolling –, this version is utilised for the proof-of-concept. Nevertheless, it is important to mention that the implementation works independently of the ROS2 version, as long as the employed tracepoints exist. The utilised tracepoints are briefly described in Table B.1.

The Shadow Caster

To generate an accurate Shadow Instance and prove the functionality of the logical model introduced in Section 6.3.2, the Digital Shadow strictly adheres to this logical model. The **Tracer** module of the Shadow Caster is represented by a *structural tracer* and a *continuous tracer*, tracing structural and continuous data of the ROS2 system in question, respectively. This distinction enhances the strict separation of structure and value, which thereby continues consistently throughout the approach of this thesis.

The structural and continuous tracer employ the Unix-exclusive *LTTng* (Linux Trace Toolkit next generation) framework [59] as the default tracing backend due to its low overhead capabilities in addition to its functionality for tracing both userspace and kernel events [59][60]. Additionally, LTTng supports enabling and disabling tracepoints at runtime, thereby contributing to the objective of aggregating only necessitated data.

Babeltrace2 [61] is selected as the parsing toolkit for recorded tracepoint events, as it is the toolkit recommended by LTTng for parsing recorded trace data [59]. Tracepoint events are events that are triggered when an enabled tracepoint is reached and thus traced.

The proof-of-concept only contains the basic functionality that is necessitated to evaluate the concepts introduced in this thesis, with focus on the generation of platform-specific system models. For instance, the configurations that enable the integration of expert knowl-

edge in the form of textual Requirement-Diagrams are not implemented. Inherently, the **Evaluator** module is not implemented.

The Shadow Instance

To represent the Shadow Instance, the graph database *neo4j* [62] is employed to store the structural information of the model, as it is considered one of the most widely used graph databases in the world [63, p. 377], which underlines its relevance and suitability for modelling structural information in practical applications.

Continuous numerical values are stored within the time-series database *influxDB* [64], which is chosen due to its low space consumption and high throughput capabilities [65, p. 13], thereby contributing to the objective of generating a lightweight solution.

The log database, for storing textual values, is not implemented. However, it is not necessitated to explore the viability of the proposed mathematical meta-model and the logical model of the Digital Shadow targeting ROS2 systems. This is because it fulfills an analogous function to the numerical time-series database with the exception that it additionally aims at the capability of storing exchanged ROS2 messages.

In the present section, the combination of Shadow Caster and Shadow Instance is denoted as the *Shadow Implementation*.

Within the Shadow Implementation, the continuous values „publishing rate“ and „CPU utilisation“ are implemented to represent one ROS2 specific attribute and one operating system specific attribute. Thus one Edge-related attribute and one Member-related attribute are represented. The „publishing rate“ is determined by the respective tracepoint, whereby the respective triggered events are analysed and assigned to a respective Publisher and thus publishing Edge. Therefore, the number of respective occurrences per second is computed to a respective publishing rate. The CPU utilisation is calculated by employing the information provided within the */proc* directory.

Additionally, the namespace- and process-driven, edge-induced Trees are implemented in order to evaluate their feasibility.

Based on the tools and frameworks employed, the subsequent sections explore the capabilities of the implementation in relation to the concepts proposed in this thesis. First and foremost, the following section focuses on evaluating the feasibility of the mathematical meta-model.

7.2. Feasibility of the Mathematical Meta-Model

The feasibility of the mathematical meta-model is imperative for achieving the objectives of this thesis. Therefore, this section gradually explores its feasibility by generating an exemplary ROS2 Graph in Section 7.2.1. Subsequently, the implementation of aggregating the ROS2

system is described (Section 7.2.2) before delving into the representation of continuous values in Section 7.2.3. The potential for integrating system specific information is discussed in Section 7.2.4.

7.2.1. Generating an Exemplary ROS2 Graph

To demonstrate an application of the mathematical meta-model, an exemplary ROS2 Graph is generated for the official Action Server/Client tutorial [66] of ROS2, as it offers a clear yet comprehensive insight into the communication within a ROS2 system. To this end, it is described gradually within this section. To ensure a clear understanding, the order stated does not correspond to the order in which the initialisation actually takes place.

1) Instantiating Nodes

As a first step, two active Members, referring to the two initialised Nodes, are instantiated by the respective tracepoint events, which contain the Nodes respective namespace, name and handle. The process ID is gathered through contextual information. Figure 7.2 illustrates the so far generated ROS2 Graph. Note that the instantiated active Members are not part of the ROS2 communication Graph, as they are not connected to any communication Edges.



Figure 7.2.: Instantiating Active Members that Represent Nodes ❖

2) Instantiating Timers

Timers are created by two tracepoint events, whereby one instantiates the Timer and thereby establishes a correlation between the Timer's handle and frequency. Concurrently, the second event establishes a link between the respective Node handle and Timer handle. Note that the Action Client is connected to at least one communication Edge and therefore part of the ROS2 communication Graph (Figure 7.5).

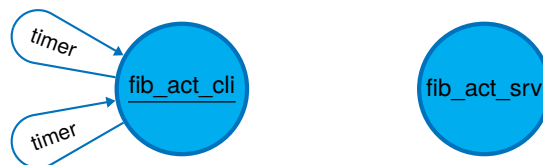


Figure 7.3.: Adding Timers to the Generated ROS2 Graph ❖

3) Instantiating Services

To represent Services, six sending Edges are added between the active Members, referring to the request and response of the individual Services „send_goal“, „cancel_goal“, and „get_result“. The Service Server/Client communication is instantiated by two tracepoint events.

Both contain the name of the associated Service Server in addition to the respective Node handle. The tracepoints are differentiated by their purpose, whereby one induces the instantiation of a Service Client and thus additionally provides a respective Client handle, while the second induces the instantiation of a Service Server and thus provides a respective Server handle. Figure 7.4 illustrates the updated ROS2 Graph containing the newly added sending Edges. Note that even though there are multiple sending Edges in both directions, each Edge is individually identified by the name of the Service Server/Client combination it relates to. This aspect is important for later analyses.

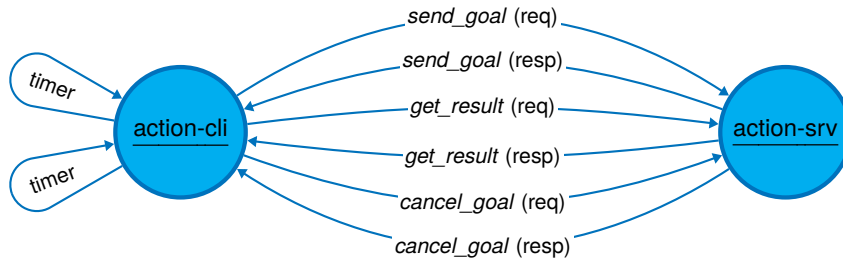


Figure 7.4.: Adding Sending Edges that Represent Server-Client Communication ❖

4) Instantiating Topics

A variety of Topics, relating to passive Members, is added to the existing Graph. However, Topics are not created explicitly, they are initialised alongside the instantiation of a Publisher or Subscriber. The respective tracepoint events contain the handle of the initialising Node, the handle of the respective Publisher or Subscriber, and the name of the respective Topic. The updated ROS2 Graph is illustrated within Figure 7.5. Note that only the upper two Topics refer to the concept of Actions. The lower two Topics are fundamental Topics any ROS2 Node publishes and subscribes to.

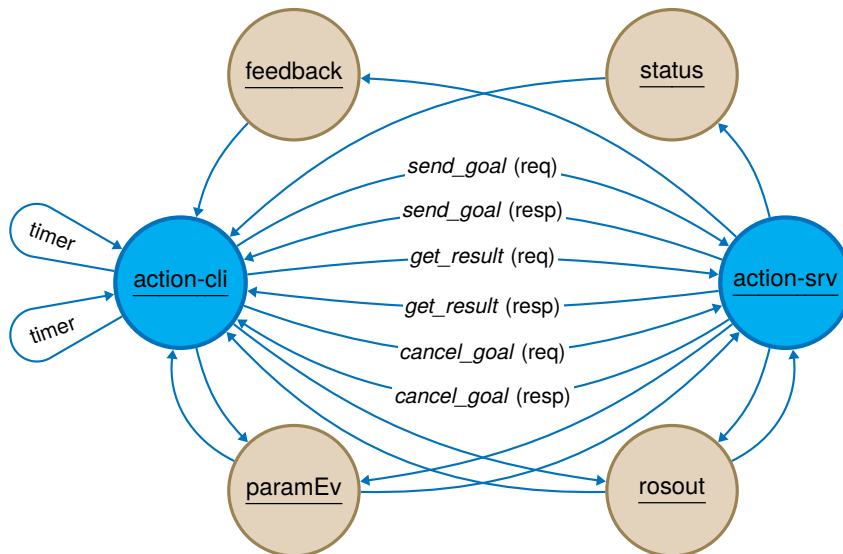


Figure 7.5.: Extending the Example by Topics Represented as Passive Members ❖

The ROS2 communication Graph representing the communication structure of the official Action Server/Client tutorial is thus fully represented. However, as mentioned in Section 7.1, the process- and namespace-driven Trees are integrated into the Graph's structure to organise the ROS2 Graph (see Section 5.2.2). For the sake of clarity, the communication Edges are not explicitly named in further figures.

5) Integrating the Process-Driven Tree

The process-driven Tree is integrated into the existing ROS2 communication Graph by inserting additional Members and organising Edges that represent the various affiliated processes and their relationships. It is constructed by joining the roots of several Subtrees. Each Subtree consists of multiple branches, which are constructed by recursively traversing the parent processes of the respective process instance that encompasses a Node. It is trimmed above the least process called „ros2“. As a result, each branch starts with the respective process instance that encompasses a Node and ends at the least process called „ros2“. Subsequent to the traversal and trimming, the thus created branch of the Tree is added to the respective Subtree. Within Figure 7.6, the active Members called „ros2“ relate to the instances that triggered the Action Server and Action Client, respectively². Note the passive Member called „host“, which relates to the virtual and thus passive Member that joins all Subtrees that originate from the same operating system.

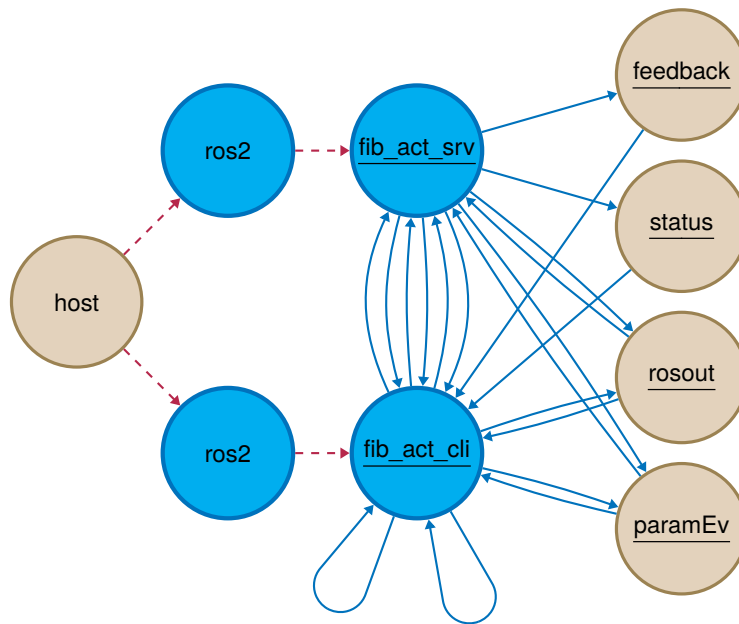


Figure 7.6.: Extending the ROS2 Graph by its Respective Process-Driven Tree ❖

6) Integrating the Namespace-Driven Tree

The namespace-driven Tree is constructed based on the name structure of each communication Member. For instance, the fully qualified name of the „feedback“-Topic relates to „/action/fibonacci/feedback“, which is split into its components „/“, „action/“, „fibonacci/“, and „feedback“. The first component refers to the root of the namespace-driven Tree, which is

²Both were started via a *ros2 run* command.

represented by a passive Member called „/“. The second component is represented by a passive Member called „action“, which is connected to the root via an organising Edge. The third component is represented by a passive Member called „fibonacci“, which is connected to the second component via an organising Edge. Finally, the fourth component, called „feedback“, represents the Topic as a passive Member and is connected to the third component via a further organising Edge. This process is repeated for all communication Members, thereby creating a Tree structure that represents the namespace structure of the ROS2 system. The exemplary, resulting namespace-driven Tree is added to the so far constructed Graph, visualised in Figure 7.7.

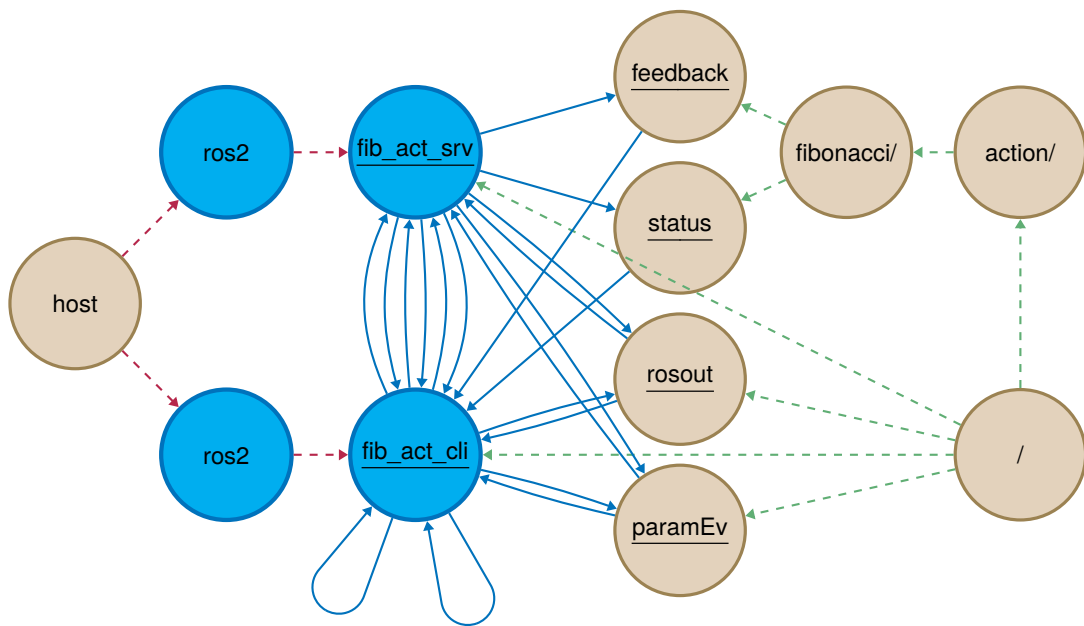


Figure 7.7.: Extending the ROS2 Graph by its Respective Namespace-Driven Tree ❖

This section demonstrated the feasibility of the mathematical meta-model for representing the fundamentals of ROS2 systems. Specifically, it generated a system specific model that reflects the communication structure created in the Action Server/Client tutorial. Additionally, the respective process- and namespace-driven Trees have been integrated into the ROS2 Graph. With the evidence that the implementation is capable of representing the fundamental structure of ROS2 systems, the subsequent section describes the implementation of the aggregations and summations (see Section 5.2).

7.2.2. Aggregating ROS2 Systems

The aggregations defined in Section 5.2 are intended to facilitate Fault Detection and Diagnosis by enabling a broad but blurred view of the ROS2 system in question. Thus, their feasibility is imperative to the objectives of this thesis, as they enable a holistic view onto a ROS2 system.

All aggregations are implemented by first querying the graph database in order to retrieve the respective Members and Edges. Subsequently, as each aggregation leads to a totalling of the respective attribute values, the time-series database is queried to directly total the respective attribute values and return the computed result.

An exemplary aggregation and its influence on the involved systems is represented in Section 7.6.2.

With the computations defined in the mathematical meta-model implemented, the following section explores the integration of time-dependent and thus continuous values.

7.2.3. Continuous Values in the ROS2 Digital Shadow

As described previously, the strict separation of structure and continuous values is maintained throughout the entire approach of this thesis. Within the ROS2 Graph, each Member and communication Edge is provided with a UID, enabling an unambiguous assignment of attribute value to a respective entity. For instance, Figure 7.8 illustrates the attribute values that refer to the publishing rates that are assigned to publishing Edges in order to illustrate the communication via Topics during an exemplary execution of the Action Server/Client tutorial (Figure 7.7). The sequence of the Action Server returning its feedback to the Action Client is illustrated. Concurrently with the start of the feedback loop, the Action Server publishes the respective state of the Action procedure onto the „status“-Topic. In total, it publishes nine feedback messages to the respective „feedback“-Topic. Following the last message, the Action Server updates the state of the Action procedure by publishing onto the „status“-Topic again. Note also the six messages that the Action Client published onto the „paramEv“-Topic. Those messages were sent due to the initialisation of the Action Client.

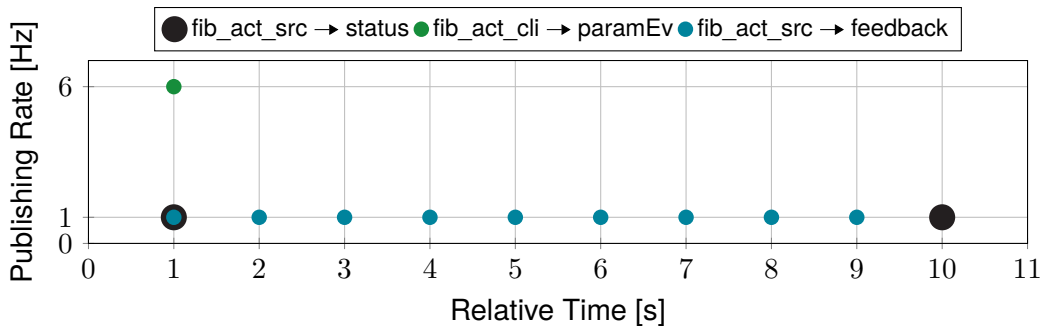


Figure 7.8.: Exemplary Publishing Rate of all Instances in the Generated ROS2 System Model

This strict separation of continuous values and structure, and the assignment via an UID, contributes to the objectives of this thesis, as it enables the selection of optimised solutions for both dimensions, i.e. storing structural values in a graph database and storing continuous, numerical values within a time-series database.

With the evidence that the differentiation between structure and value is feasible, the subsequent section discusses the viability of integrating system specific information.

7.2.4. Integrating System Specific Information into the Model

Due to its generic design, the mathematical meta-model can incorporate platform-specific elements into the generated ROS2 system model. This is achieved by treating externally integrated Members as equivalent to native ROS2 components. Therefore, externally integrated Members are also distinguished by the UID, which further enables a clear assignment to the continuous values stored within the time-series database, allowing for a seamless and coherent system representation.

This section demonstrated the generation of an exemplary ROS2 Graph, gave a brief insight into the implementation of the aggregations and discussed the potential of integrating system specific information. Despite the promising results, the creation of a specific ROS2 system model is not without challenges. The subsequent section elaborates the challenges that arise when generating a system specific model based on the implemented proof-of-concept.

7.3. Challenges while Generating System Specific Models

The structure of the ROS2 communication Graph is exclusively generated by employing the tracepoints of the ROS2 source code. However, although the tracepoints appear to enable a faithful generation of the ROS2 Graph, there are challenges that may cause the model to degenerate.

7.3.1. Reassigning Communication Edges across Multiple Restarts

As described in Section 5.1.2, the ROS2 Graph is only populated and never reduced. Therefore, its Members and Edges are to be reassigned at the restart of the referring entity. For instance, if a Node terminates, its respective active Member and adjacent Edges are not removed and thus the topology of the ROS2 Graph is not changed. If the Node is restarted, it has to be reassigned to the respective active Member that it was represented by in the ROS2 Graph. Analogously, the instantiated Publishers, Subscribers, Service Servers, etc. are to be reassigned to their respective communication Edges. This is imperative, as information, like continuous values, is related to the specific entity throughout the entire runtime of the system.

While this reassociation is trivial for communication Edges that are uniquely identified by their source, destination, type, and – when existing – name³, this becomes challenging for communication Edges that cannot be separated by those properties.

To elaborate, consider a Node that manages five Publishers to the same Topic. Three of those Publishers are instantiated at the initialisation of the Node. The further Publishers are instantiated during runtime.

Reassigning the former three Publishers to their representations within the ROS2 Graph is trivial, as those Publishers are initialised within a given order and thus countable. This count is illustrated in Figure 7.9a. However, there is no unambiguous property that differentiates the latter two Publishers, as there is no guarantee regarding their instantiation order (Figure 7.9b). Thus, the reassignment of those Publishers to their respective publishing Edges presents a challenge.

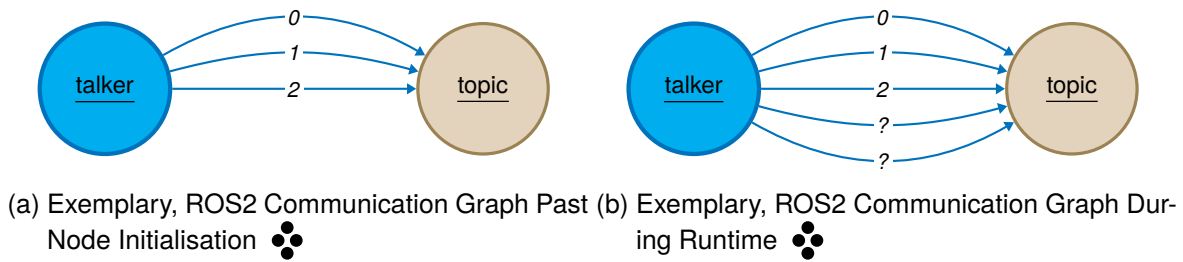


Figure 7.9.: Challenge with Reassigning Communication Edges across Multiple Restarts

7.3.2. Differentiating between Multiple Message Types of a Single Topic

A second challenge arises from the ambiguity of Topic types (see Section 4.2.1). Since the tracepoint events that reflect the instantiation of Publishers or Subscribers for a given Topic do not provide any information about the respective message type they are transmitting or receiving, no differentiation between these types can be made.

For instance, consider a Topic that forwards messages of various types. As the generated ROS2 Graph contains no information about the message types, the resulting communication structure becomes degenerated. This is because a Node that subscribes to a Topic may not receive all messages published onto it. This challenge is illustrated in Figure 7.10, where the publishing and subscribing Edges are coloured brown and green to represent different message types being published to the same Topic.

7.3.3. Generating the Process-Driven, Edge-Induced Tree

The construction of the process-driven Tree induces a challenge that originates from the usage of Node Composition, where multiple Nodes are containerised and thus grouped within a single

³For instance, the name of a Service Server that is related to a sending Edge.

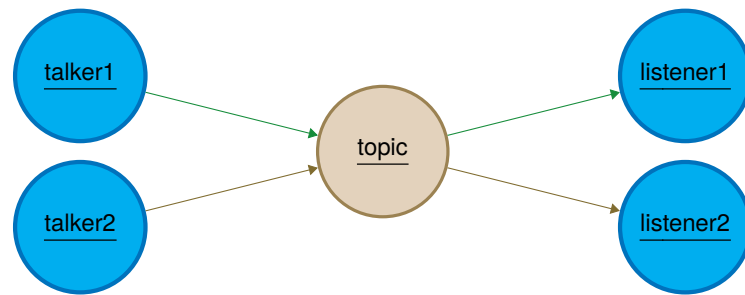


Figure 7.10.: Challenge of Differentiating Multiple Message Types on a Single Topic 

process to enhance system performance (see Section 4.3). To elaborate, consider two Nodes as part of a Container and thus two Nodes that share the same process (Figure 7.11). If „node1“ initiates the creation of a third Node („child“), the parent process of „child“ refers to both Nodes of the Container. In conclusion, „child“ is assigned to two parent processes and the Tree is malformed.

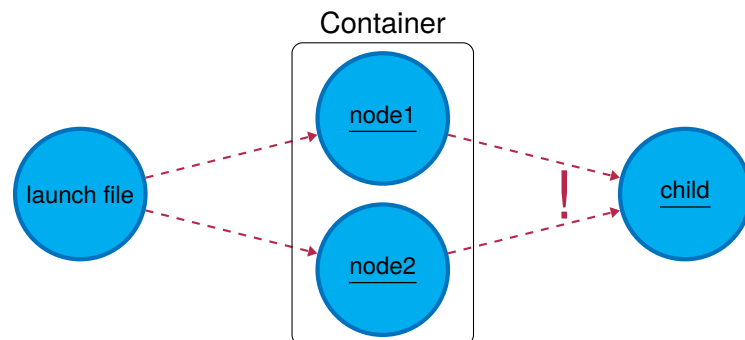


Figure 7.11.: Challenge while Constructing the Process-Driven, Edge-Induced Tree 

7.4. Defining Constraints on ROS2 Systems to Ensure Consistency and Correctness of the Generated Model

In consideration of the constraints arising from the tools that are employed, it is imperative to define constraints for ROS2 systems in order to ensure the validity of the generated model. Therefore, the following constraints are defined.

The Combination of Name and Namespace of Nodes has to be Unique

Nodes are integrated into the model by employing the tracepoints of the ROS2 source code. The respective tracepoint events only provide the name and namespace of the Node to uniquely identify it across restarts. Therefore, the combination of the Node's name and namespace has to uniquely identify the Node.

Service Names have to be Unique

Similar to Nodes, Services are integrated into the model by employing the tracepoints of the ROS2 source code. The respective tracepoint events only provide the name of the Service to uniquely identify it across restarts. Thus, the Service is identified by its name only. In conclusion, the name of a Service has to be unique.

Topics are Required to have a Unique Type

Topics are implicitly instantiated alongside the instantiation of Publishers or Subscribers. However, as the respective tracepoint events provide no information on the type of the respective Publisher or Subscriber, the generated communication structure becomes degenerated if a Topic distributes messages of multiple types. In conclusion, each Topic has to be dedicated to a single message type.

The Operating System has to be Unix based

Tracepoints are implemented into the ROS2 source code and traced using LTTng's user-space tracing infrastructure, which is available for Unix only [59]. As a result, equivalent tracing functionality is not natively supported on non-Unix platforms without significant modifications or custom instrumentation.

The Nodes have to be Written in C++

The tracepoints of the ROS2 source code are distributed across the multiple layers [67] of the ROS2 middleware [40]. To elaborate, various tracepoints are included within a single layer only and are therefore not triggered if the respective layer is not passed. For instance, the tracepoint that is employed to link a Timer to its respective Node is embedded within the `roscpp`⁴ only. In conclusion, Timers that are not instantiated utilising the `roscpp` cannot be added to the ROS2 Graph. An undiscovered Timer leads to an incomplete generation of the ROS2 Graph.

Composed Nodes are Not Enabled to Initiate Further Nodes

Within the concept of Node Composition, multiple Nodes share the same process in order to increase system performance. However, if one of the respective Nodes instantiates a further Node or instance, the parent process of the newly created Node cannot be determined unambiguously. However, linking it to all Nodes that belong to the respective process and thus Container, degenerates the process-driven Tree, as the newly created Node refers to multiple parents. Therefore, not the entire meta-model, but this type of edge-induced Tree is not feasible for respective ROS2 systems, as it may degenerate.

One Edge per Direction between Members allowed at Runtime

As described previously, the reassignment of multiple communication Edges poses a particular challenge when the Edges are initialised at runtime, meaning past the initialisation

⁴The `roscpp` is the official C++ client library for ROS 2.

phase of the respective Node. Due to this uncertainty, each Edge that is initialised at runtime has to be uniquely identified by its source, destination, type, and – when existing – name.

In summary, although the introduced mathematical meta-model is theoretically feasible (see Section 5.3), the practical implementation of the Shadow Caster demonstrates that the model is limited due to several constraints that predominantly arise from the tools employed. While some constraints can typically be addressed smoothly, e.g. by employing fully qualified Node-, Service-, and Topic-names, further constraints may be difficult to resolve due to possibly employed software stacks that are not based on the rclcpp or guidelines referring to the operating system. It is imperative to mention that blocking the instantiation of similar communication Edges at runtime hardly limits the structure of the ROS2 system and therefore represents a strong constraint.

However, if the constraints are met, the approach of this thesis offers a feasible methodology for automatically generating generic ROS2 system models that can be analysed from multiple perspectives. It is also demonstrated that expert knowledge or the manipulation of the software stack is not necessitated due to the utilisation of the tracepoints that are embedded within the ROS2 source code.

In addition to the feasibility of the meta-model, the generated overhead is also of interest. Consequently, the subsequent section evaluates various implementation-specific aspects with focus on the introduced overhead onto the operating system that executes the ROS2 system in question.

7.5. Generating ROS2 System Models Using the Shadow Caster: Performance Tests

In order to evaluate the overhead introduced onto the operating system executing the ROS2 system in question, the Shadow Implementation is analysed through multiple tests. The test setup used and the procedures performed are elaborated within Section 7.5.1, while the evaluation takes place within Sections 7.5.2 to 7.5.4.

7.5.1. The Test-Setup

Given that ROS2 systems can exhibit a wide variety of topologies, a set of tests is conducted to evaluate the incurred overhead under several configurations. The constructed topologies consist of multiple Publisher-Subscriber pairs, with each pair communicating independently. For instance, „/talker_0“ exclusively publishes a message to „/topic_0“, which is then exclusively received by „/listener_0“. This setup allows for a systematic analysis on how the number of

running Nodes affects resource consumption and the incurred overhead.

In addition to varying the number of Node pairs, the publishing frequency was also modified in order to investigate its impact on performance metrics. This dual-parameter approach enables a fine-grained assessment of the induced overhead.

For an extensive analysis, 55 different test configurations are defined by the combination of 11 different publishing frequencies (0 Hz, 10 Hz, 20 Hz, ..., 100 Hz) and 5 different numbers of Node pairs (0, 10, 20, 30, 40). Each configuration was run twice, once alongside the monitoring solution developed in this thesis⁵ and once without it⁶. Leading to 55 test-pairs and 110 executed tests, with each test run lasting 10 min followed by a 5 min pause. Within subsequent figures, the number of Node pairs is denoted as „n“.

The CPU and RAM utilisations are recorded using *cAdvisor* [68] and *process-exporter* [69]. Additionally, the end-to-end latencies are extracted directly from the exchanged messages in order to provide a detailed view of the effects on the communication performance.

All experiments are executed sequentially in isolated Docker Containers [70] in order to ensure a reproducible and clean testing environment. An exception is made for the two databases, which are executed within their own respective Docker Containers.

As recommended by Bédard, Lütkebohle, and Dagenais [39], all power-saving features (dynamic frequency scaling, C-states) are disabled and the UDP buffers were increased to 64 MB. All tests are conducted on a laptop operating in performance mode. Its hardware and software specifications are listed in Table 7.1.

Table 7.1.: Specifications of the Testsystem

Component	Specification
Operating System	Ubuntu 24.04
Kernel	6.8.1-1015-realtime
CPU	Intel® Core™ i7-8750H @ 6 x 4.10 GHz
RAM	16 GB DDR4 @ 2400 MHz

It is important to mention that the performance highly depends on the implementation and tools chosen. Therefore, results may vary for, e.g., other databases.

7.5.2. Latencies under Several Conditions

The end-to-end latencies are recorded for each message exchanged between each Publisher-Subscriber pair. All latencies are measured in nanoseconds and the median is computed for each test run. Subsequently, the differences of each respective monitored and non-monitored

⁵In the following called „monitored“.

⁶In the following called „non-monitored“.

test run are calculated in order to analyse the overhead introduced by the Shadow Implementation.

To validate the quality of the measured latencies, the *Mann-Whitney U Test* was employed, as it does not necessitate normal distribution [71, p. 1]. The test was applied to compare the latency distributions between monitored and non-monitored configurations. According to the Mann-Whitney U Test, all results are statistically significant (Figure C.1), suggesting meaningful differences between the measured test results.

The calculated differences are visualised in Figure 7.12. Note that configurations where no message exchange occurred are omitted. Although the measured results are significant, the data does not reveal any clear influence or trend, as the noise from the executing system does not allow for a proper measurement. Therefore, no statement – regarding the effect of the Shadow Implementation on the latency of the ROS2 system – can be made.

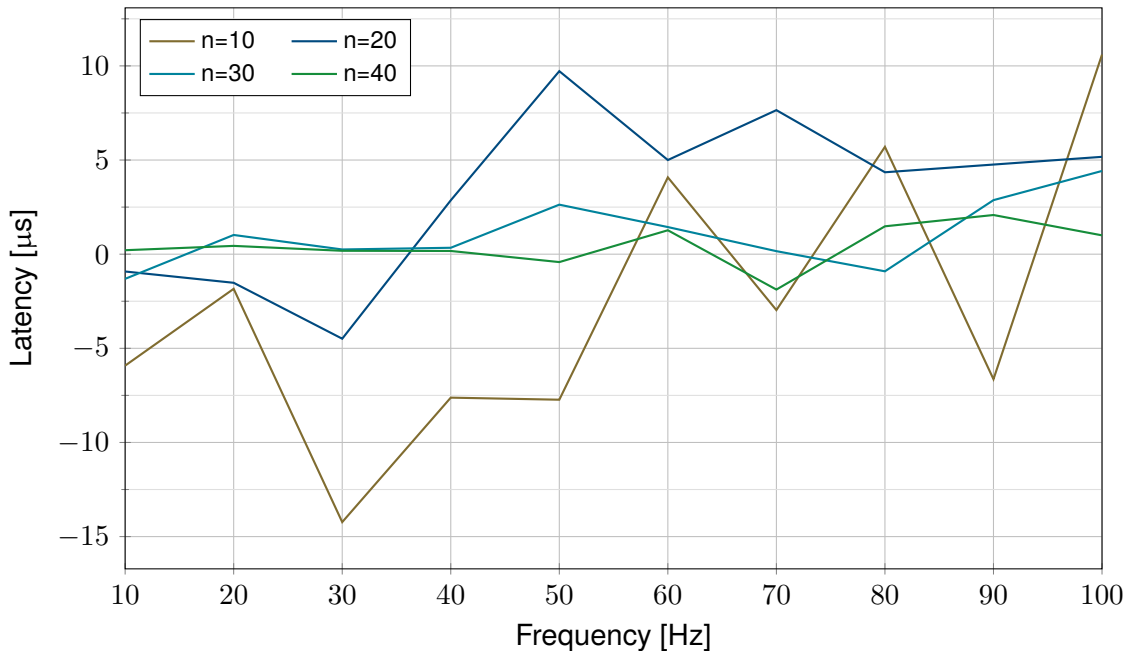


Figure 7.12.: Latency Overhead Originating from Differing Test Configurations

7.5.3. CPU Utilisation under Several Conditions

A second metric refers to the CPU utilisation introduced through the Shadow Implementation. Therefore, the CPU utilisations of all instances are monitored, their median calculated and visualised within Figure 7.13. Note the CPU utilisation of the entire Shadow Implementation in comparison to the CPU utilisation incurred by the ROS2 system itself.

It is observed that the CPU utilisation of the Shadow Implementation increases in proportion to the number of Publisher-Subscriber pairs and remains relatively constant for increasing frequencies greater than zero. The predominant reason for the increasing CPU utilisation of the

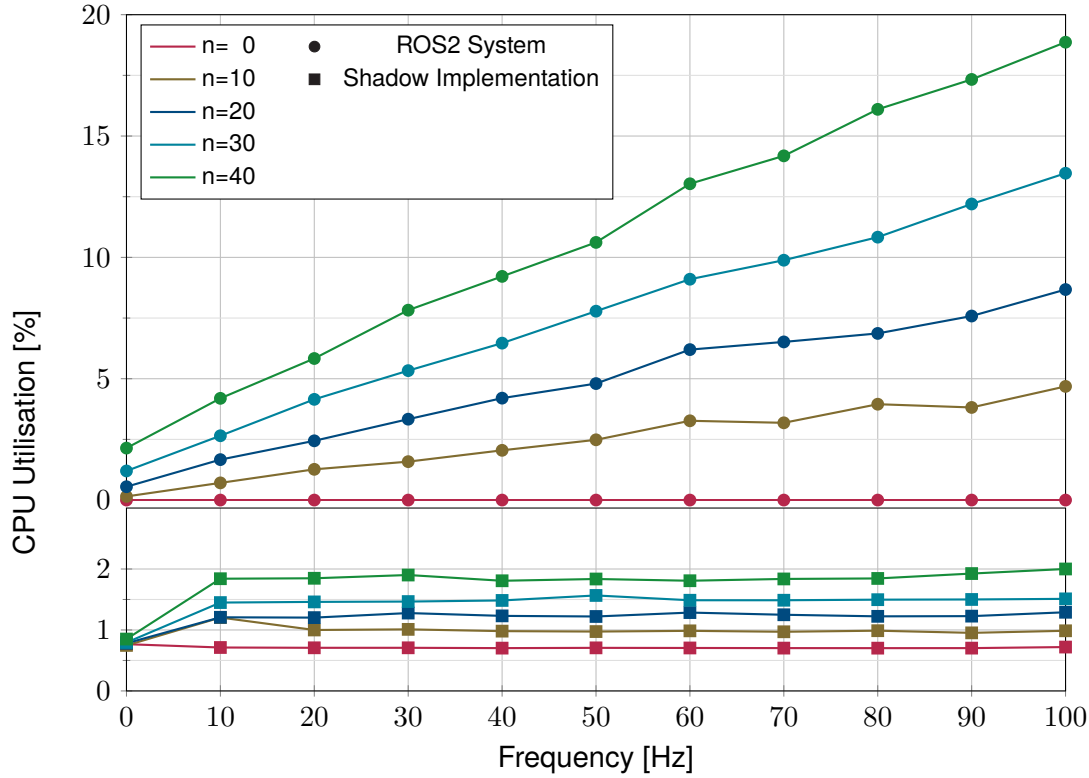


Figure 7.13.: CPU Utilisation of the Shadow Implementation Compared to the ROS2 System

Shadow Implementation is found at the neo4j database (see Figure C.2), which stores the ROS2 Graph of the respective system. The rationale for this trend is the number of queries processed by the database, which increases in proportion to the number of entities that are to be represented within the ROS2 Graph and thus the number of Publisher-Subscriber pairs. The queries are executed to integrate the respective Members and Edges into the ROS2 Graph. It is evident that the influxDB also exhibits a minor increase, which is also attributed to the increasing number of queries. However, influxDB is a time-series database that is designed to handle high-frequency data throughputs more efficiently. Consequently, influxDB exhibits a smaller overhead in these scenarios than neo4j. The Shadow Caster's impact on the CPU utilisation also increases with the number of Publisher-Subscriber pairs (see Figure C.2). This small increase is also attributable to the increasing number of Entities created and thus managed. The performance of the Shadow Caster will be further discussed in Section 7.6.3. Figure C.4 illustrates the relative CPU utilisation of the Shadow Implementation with respect to the ROS2 System.

7.5.4. RAM Utilisation under Several Conditions

To analyse the introduced RAM overhead, the RAM utilisations are measured during the performance tests. It is observed that the RAM usage does not change significantly with increasing

frequency (see Figure C.3). Accordingly, the values are grouped by the number of Publisher-Subscriber pairs and the respective arithmetic mean is calculated. The results are visualised in Figure 7.14. It is evident that the RAM utilisation of the Shadow Implementation increases in proportion to the number of Publisher-Subscriber pairs.

The primary rationale for this trend is that the Shadow Instance stores an increasing amount of information that is proportional to the number of Publisher-Subscriber pairs and thus caches information within the RAM. However, the RAM usage of the ROS2 System increases at a noticeably higher rate and surpasses the Shadow Implementation at 30 Members. Conversely, the RAM usage of the Shadow Caster remains below 25 MB and is therefore negligible in comparison to the RAM used by the ROS2 System and Shadow Instance. The performance of the Shadow Caster will be further discussed in Section 7.6.3. A more detailed visualisation of the RAM usage is illustrated in Figure C.3. Figure C.5 illustrates the relative RAM utilisation of the Shadow Caster and the Shadow Instance with respect to the ROS2 System.

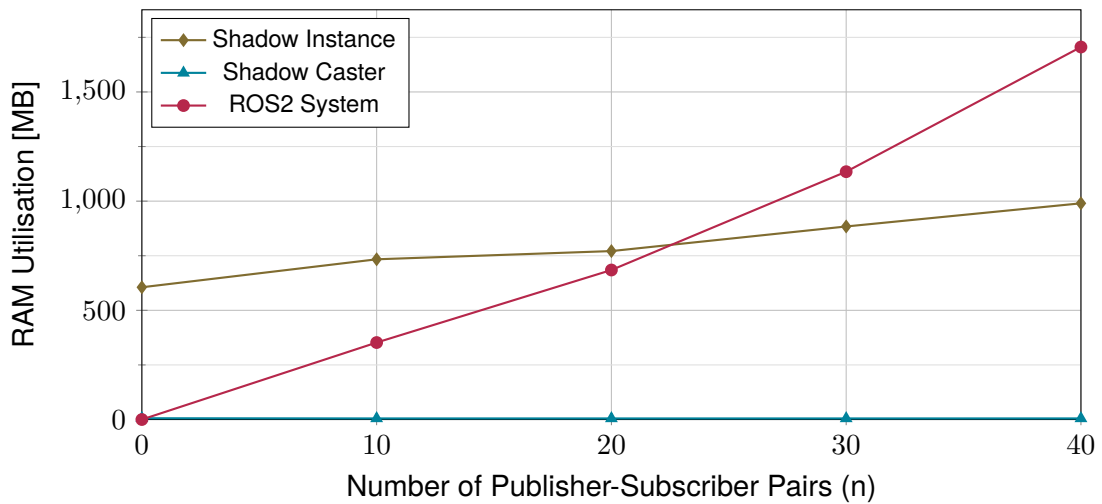


Figure 7.14.: RAM Utilisation of the Shadow Implementation Compared to the ROS2 System

To summarise, the performance tests demonstrated that the Shadow Implementation introduces an overhead that increases with the number of Publisher-Subscriber pairs. However, in comparison to the ROS2 system, especially the CPU related overhead stays relatively low, particularly for larger ROS2 systems. The incurred overhead mainly originates from the Shadow Instance, which is represented by the databases. Consequently, the Shadow Instance provides a definitive foundation for enhancing efficiency. Nevertheless, it is important to note that the Nodes used for testing did not perform any computationally intensive tasks, such as image processing or similar complex operations. Therefore, the relative performance impact of the Shadow Implementation is expected to be significantly lower in more demanding scenarios, as the computational workload of the Nodes would surpass the overhead introduced by the Shadow Implementation significantly.

Even though the performance tests covered a high number of configurations, enabling clear conclusions to be drawn, the previous tests only cover the utilisations during the generation of ROS2 system models. However, the Shadow Caster additionally serves as an endpoint that enables the gathering of recorded information in order to perform an online Fault Detection and Diagnosis. Therefore, the following section examines various aspects related to requests handled by the Shadow Caster.

7.6. Requesting Information of the ROS2 System

Due to the high number of possible requesting scenarios, this section analyses two fundamental metrics, which relate to the property of serving as an endpoint. Firstly, the latency at which events are forwarded is described, explored and analysed in Section 7.6.1. Secondly, the CPU utilisation incurred by requesting information is analysed in Section 7.6.2.

7.6.1. Analysing the Event Propagation Latency

In order to accelerate feedback cycles, the latency at which requesting clients are notified about events is of interest. For instance, if a Node is started, the Shadow Caster has to process the respective tracepoint event before the gathered information can be forwarded to the client. This period, between the start of a Node and a requesting client receiving the respective information, is denoted as the *Event Propagation Latency*.

For a detailed analysis, two configurations are explored that differ in the usage of the LTTng tracer. For a clear understanding, both implementations are described briefly.

Rotate Trace Files

Trace files can only be parsed if the respective trace file is finalised. Therefore, finalising a trace file at the end of system execution is not feasible for modelling systems at runtime. To counteract this constraint, the concept of *Rotation* enables a batch-wise analysis of trace events. To elaborate, multiple trace files are created one after the other in order to analyse them even if the recording session is still running. Thus, each trace file only represents a specific time interval, which is defined by the Rotation period. As a result, the finalised files can be parsed and analysed periodically. Therefore, this methodology induces a delay, as the recorded events are processed after the Rotation period has elapsed. Within the implementation, a Rotation period of 1 s is chosen.

LTTng-Live

Conversely to Rotations, *LTTng-Live* provides a live session that transmits events to an endpoint that can be observed. As the events are transmitted directly to the endpoint, this approach enables the immediate processing of tracepoint events.

Figure 7.15 illustrates the statistical distribution of the Event Propagation Latency across 3500 measurements per implementation and thus test series. It is observed that the median Event Propagation Latency employing Rotations is 59.26% higher than employing LTTng-Live.

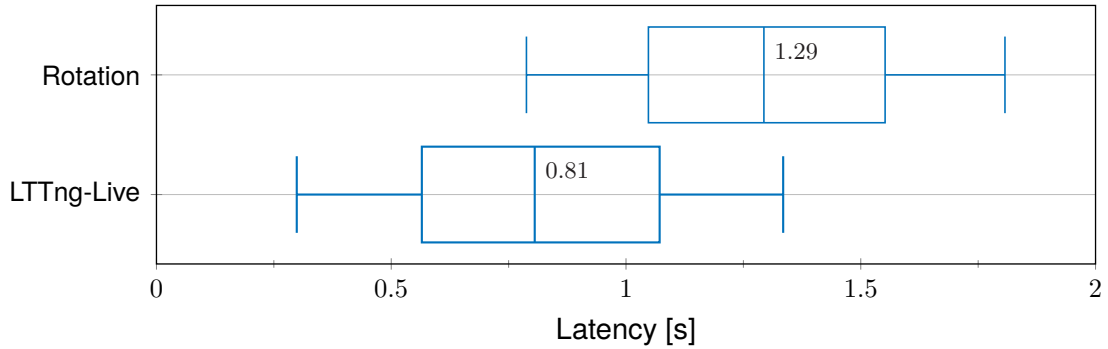


Figure 7.15.: Event Propagation Latency for Rotating Trace Files and LTTng-Live

Nevertheless, as both approaches refer to differing implementations, the CPU utilisation is monitored to compare the induced overhead and it is evident that LTTng-Live incurs 8772% more overhead (during the executed Event Propagation Latency tests) than the employed Rotations (see Figure C.6).

As the Shadow Implementation aims to be a lightweight solution, the decision falls upon employing Rotations.

Besides the Event Propagation Latency, the overhead incurred by requesting the Shadow Caster is also of interest, as it provides further insight into the performance of the Shadow Implementation.

7.6.2. Analysing the Overhead of Information Requests

In order to evaluate the influence of a client request on the overhead introduced by the Shadow Caster, this section explores the CPU utilisation incurred through processing these requests.

To introduce a more complex example, two Nodes, publishing to the same Topic with a frequency of 100 Hz each, are considered⁷.

Within the test, a client is started with the aim to periodically explore the forwarding rate of the Topic by employing the relations introduced within the mathematical meta-model (Θ_{in} in Definition 5.2.2). According to the request of the client, the forwarding rate is returned periodically. To handle the request, the Shadow Caster firstly queries the neo4j database in order to receive the UIDs of the respective incoming publishing Edges. Subsequently, it sends periodic requests to influxDB at a frequency of 1 Hz, requesting the aggregated value and returning the corresponding result to the client.

⁷As described previously, publishing frequencies are assigned to the corresponding publishing Edges.

The effects on the CPU utilisations are visualised in Figure 7.16. It is observed that there is no significant effect on the CPU utilisation of the Shadow Caster. This aspect is discussed in detail in the subsequent section. Similarly, the neo4j database also exhibits no measurable effect. However, the CPU utilisation of influxDB doubles due to the frequent number of requests it has to process.

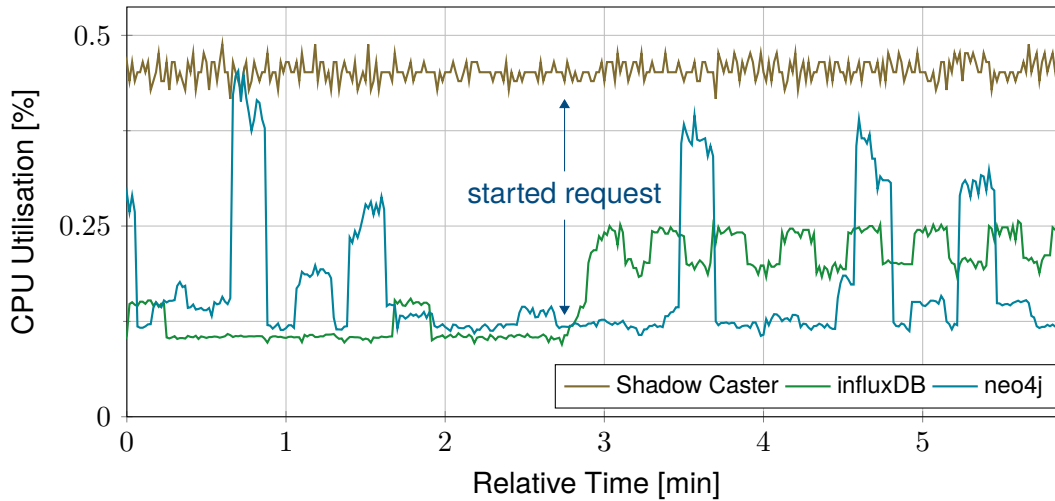


Figure 7.16.: Influence of Requests on the CPU Utilisation of the Shadow Implementation

Additionally to the CPU utilisation, the RAM utilisation is measured (Figure C.7). It is observed that there is no measurable effect on the Shadow Instance and just a small effect on the Shadow Caster. The latter originates from the instantiation of a shared memory ring-buffer, which is employed in order to transmit messages to the client.

In summary, the results provided a rough overview of the performance of the implemented Shadow Caster in two different perspectives. The following section examines the performance of the Shadow Caster in more detail.

7.6.3. The Performance of the Shadow Caster

Within the proof-of-concept, the Shadow Caster refers to the core component of the implementation, as it is responsible for generating the ROS2 Graph and managing the communication with multiple requesting clients. Therefore, an analysis of the Shadow Caster's performance is of high interest.

Throughout the tests presented in this chapter, the CPU and RAM utilisations of the Shadow Caster were recorded. It is observed that the majority of measurements indicate no significant trends or patterns, as CPU and RAM utilisation remain relatively constant. This can be explained by the internal communication mechanism of the Shadow Caster. To elaborate, the Shadow Caster is implemented as a multi-threaded application designed to handle multiple tracepoint

events or client requests concurrently. However, it does not employ an event-based communication mechanism, which would enable more efficient inter-thread communication and, consequently, enhance CPU utilisation. Instead, the communication is based on periodically polling message queues, resulting in generated overhead, even when there are no tasks to perform. Therefore, the overhead that is created due to the respective circumstances is diminished by the noise of the overhead that originates from the polling and is therefore not measurable. The polling approach was chosen for simplicity of implementation and to maintain focus on the proof-of-concept.

This section analysed the overhead that originates from requests that explore the generated model. To this end, the incurred resource utilisations of the Shadow Caster and Shadow Instance were analysed.

7.7. Applying the Created Concept to a Real-World Scenario

To demonstrate the feasibility of the Shadow Implementation outside the laboratory, it is deployed onto a real-world robotic system called „rosee“. The system is based on a „Husarion Lynx“ platform [72], which is equipped with multiple sensors (Table 7.2):

Table 7.2.: Sensors that are Assembled at rosee for the Real-World Scenario

Model	Quantity	Website
Stereolabs ZED-2i	4	[73]
FIXPOSITION Vision-RTK	1	[74]

Additionally, the system is equipped with a „Neousys Nuvo-9160GC“ [75], a computer that is designed for industrial applications. Its specifications are listed in Table 7.3.

Table 7.3.: Specifications of rosee’s Host Computer

Component	Specification
Operating System	Ubuntu 24.04
Kernel	6.8.0-62-generic
CPU	12th Gen Intel® Core™ i7-12700TE @ 4.6 GHz
RAM	32 GB
ROS2 Version	Jazzy Jalisco

To explore the feasibility of the Shadow Implementation, the ROS2 system is started in three stages, demonstrating the extensibility of the generated model at runtime.

Firstly, the base of the robot⁸ is launched. The resulting system specific Graph is illustrated in

⁸The *base* represents the core of the system and is, e.g., responsible for the movement of the robot.

Figure 7.17. As for all respective following figures, publishing Edges are coloured light green, while subscribing Edges are coloured in a dark green. Sending Edges are coloured red with a refinement for self-sending Edges (representing Timers), which are coloured orange. Organising Edges are coloured gray.

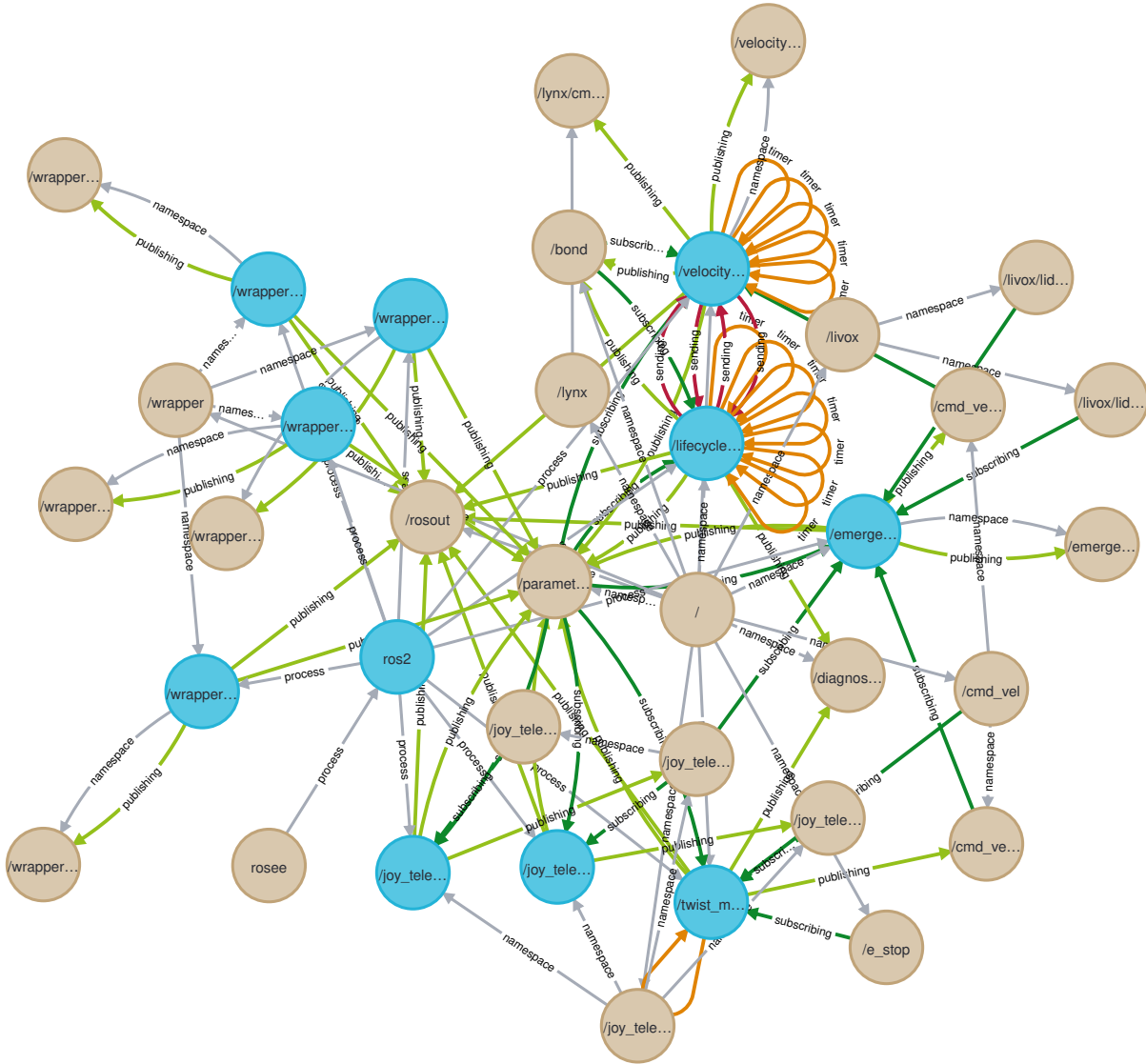


Figure 7.17.: Real-World Scenario: Generated ROS2 Graph (Stage 1) 

For the second stage, the two ZEDs and the Vision-RTK2 are launched (see Figure 7.18). Note the two large groups of Members that refer to the respective ZEDs and the small new group that refers to the Members that represent the Vision-RTK2. Subsequent, the remaining two ZEDs are started. The generated ROS2 Graph topology is visualised in Figure D.1, alongside a list of all Nodes that are started across the entire process.

By launching the system across multiple stages, the iterative expansion of the Graph is demonstrated. Additionally, the size of the Graphs should also be noted, as it demonstrates the generation of large, non-laboratory system models.

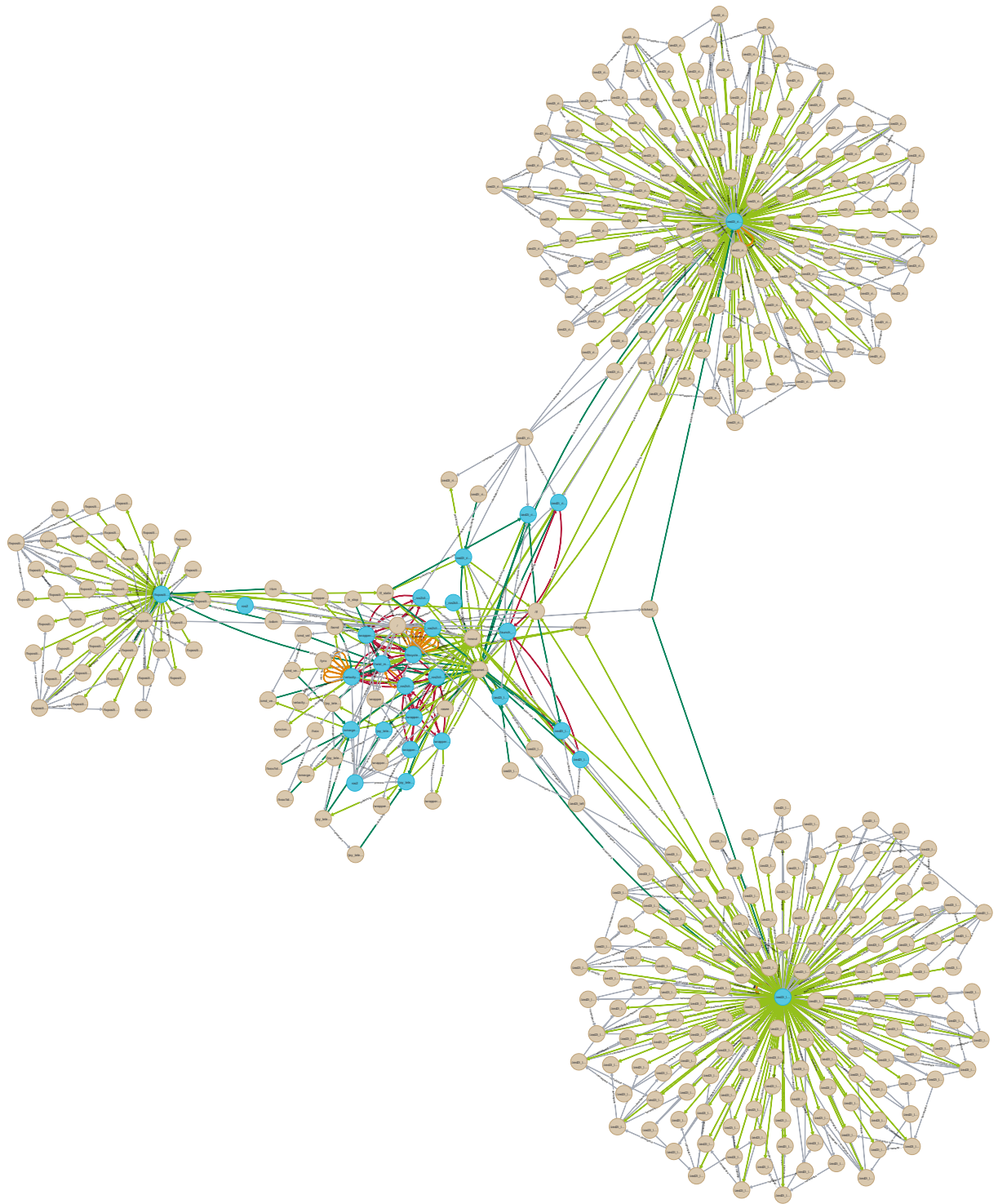


Figure 7.18.: Real-World Scenario: Generated ROS2 Graph (Stage 2) 

7.8. Summary

This section summarises the results and findings with regard to the objectives formulated in Chapter 1. The findings obtained from the evaluation of the proposed approach are thus briefly revisited and contextualised in the following:

Generic Meta-Model for ROS2 Systems

The introduced mathematical meta-model abstracts the fundamental concepts of ROS2 systems and is sufficiently generic to represent a wide range of ROS2 platforms. Additionally, its design also enables the integration of platform-specific information. Nevertheless, multiple constraints originate from the means of implementation that limit the applicability.

Holistic Perspectives onto ROS2 Systems

By introducing multiple aggregation mechanisms in a hierarchical manner, the approach enables both fine-grained and holistic analysis. This supports model-based Fault Detection and Diagnosis and enables the system to be viewed from various relevant perspectives.

Runtime Representation

Both meta-model and implementation support dynamic, automated modelling at runtime.

Lightweight Digital Shadow

The evaluation demonstrates that the relative resource consumption (CPU and RAM) decreases with increasing size of the ROS2 systems and is largely independent of publishing frequencies. Therefore, the solution is scalable. However, there is still room for optimisation, especially with regard to the Shadow Instance. The event-based approach is partially feasible, as respective tracepoints can only be enabled or disabled entirely, leading to the conclusion that a metric is enabled or disabled system-wide for the monitored system.

Non-Intrusiveness

The approach is non-intrusive, as it relies on tracepoints within the ROS2 source code and does not require any modification of the existing software stack or codebase.

Minimising Expert Knowledge

The automated approach reduces the need for manual modelling and expert knowledge, as it relies on the tracepoints of the ROS2 source code and employs them in order to generate system specific models.

In conclusion, the evaluation demonstrated that the proposed approach enables the automated, holistic, and efficient modelling of ROS2 systems at runtime within the defined constraints. The objectives set out in Chapter 1 are addressed and evaluated, providing a solid foundation for further research and development in model-based system analysis and Digital Shadows for robotic applications.

8. Conclusion and Future Work

This thesis introduced an abstract, mathematical meta-model alongside the logical model for a Digital Shadow in order to enable the automated generation of ROS2 system models at runtime. Thereby, the approach addresses key challenges in the domain of rapid prototyping by providing a holistic, extensible, and non-intrusive system representation that supports Fault Detection and Diagnosis by facilitating the analysis of a ROS2 system in question.

The evaluation demonstrated, that the meta-model is capable of representing a wide range of ROS2 systems, including their communication structure, organisational hierarchies, and runtime attributes. Nevertheless, it also revealed constraints due to the means of implementation. Although these constraints limit the applicability to ROS2 systems, they are necessitated in order to ensure the correctness of the generated model. To encounter these constraints, further research needs to explore additional means of implementation in order to enhance the feasibility of the introduced meta-model.

The Digital Shadow implemented as a proof-of-concept has proven to generate system specific runtime models without necessitating expert knowledge or changes in the existing software stack. The relative overhead induced (CPU and RAM) decreases with the increasing size of the ROS2 system and increasing message frequencies. Therefore, the Digital Shadow proves to be a scalable solution. Despite the already low resource consumption of the implementation, it still has potential for improvement, especially regarding the overhead incurred by the Shadow Instance that is represented by the databases employed. In future work, embedded database solutions will be explored to further enhance performance, as they are expected to significantly reduce the overhead and thus gain efficiency by negating the need for resource-intensive services. Additionally, the implementation is to be refactored to employ event-driven mechanisms. Paving the way to a more lightweight Digital Shadow that is capable of operating on resource-constrained systems.

As the proof-of-concept only encompasses basic functionality, future work is required to extend the Digital Shadow's capabilities.:

Consider the Development of a Domain-Specific Language

Custom queries are difficult to handle, as the entire response, that still has to be parsed, is returned to the client. The development of a Domain-Specific Language (DSL) for custom queries would facilitate advanced analyses by facilitating requests.

Integration of Further Metrics

To explore further capabilities of the concept, the integration of further metrics is to be considered. For instance, the utilisations of Executors or callback durations would gain deep insights into the scheduling of the system. This may further include the exploration of additional data sources.

Extend the Implementation to represent Multi-Host ROS2 systems

The implementation is only capable of representing single-host ROS2 systems. Nevertheless, the model is capable of representing multi-host ROS2 systems. To evaluate the feasibility, the implementation is to be extended to support the generation of ROS2 system models that extend across multiple hosts.

Enable the Integration of Expert Knowledge

As only basic functionality is implemented, the integration of expert knowledge is not yet supported. The implementation of configuration mechanisms would enable the incorporation of expert knowledge in form of textual Requirement-Diagrams. To employ these configurations, the **Evaluator** module is to be implemented to enable more sophisticated Fault Detection and Diagnosis.

In addition to development itself, three key areas require further research:

Evaluate in Development Process

Although the proof-of-concept was tested in a real-world scenario, additional validation is necessitated to explore the added value of the approach in various contexts. This also involves the inclusion of the generated model directly into the development process to assess its impact on the development and debugging of respective systems.

Evaluate further Situations and Metrics

While multiple analyses were conducted within the evaluation, there are further investigations that are to be made. For instance, as the databases are queried via the network, a network utilisation is generated and thus its overhead is to be evaluated. Similarly, as the databases are writing and reading data from the disk, these interactions are to be monitored alongside the general disk-storage that is used.

Applicability to Further Robotic Systems

Even though the meta-model is strongly influenced by ROS2, it is designed to be abstract and thus not limited to ROS2 systems. In order to evaluate the general applicability of the approach, further distributed robotic system, such as YARP [76] or PX4 [77] are to be explored with regard to their suitability to the introduced meta-model. Depending on the suitability, the approach is to be updated and evaluated by an implementation.

Especially within the described constraints, the implementation of the proposed approach proved to be sufficient and is convincingly demonstrated regarding both feasibility and scalability. Despite the challenges and the need for future work, this thesis presents a promising solution

and establishes a generic, extensible foundation for automated runtime modelling that enables comprehensive analysis of ROS2 systems during operation. The methods and tools developed pave the way for more resilient, reliable, and maintainable robotic applications and offer a starting point for future research in model-based system analysis employing the concept of Digital Shadows.

Bibliography

- [1] Juan Angel Gonzalez-Aguirre et al. “Service robots: Trends and technology”. In: *Applied Sciences* 11.22 (2021), p. 10702.
- [2] Steven D Tripp and Barbara Bichelmeyer. “Rapid prototyping: An alternative instructional design strategy”. In: *Educational technology research and development* 38.1 (1990), pp. 31–44.
- [3] Tamara Taylor and Thomas A Standish. “Initial thoughts on rapid prototyping techniques”. In: *ACM SIGSOFT Software engineering notes* 7.5 (1982), pp. 160–166.
- [4] Jey Won, Kathryn DeLaurentis, and Constantinos Mavroidis. “Rapid prototyping of robotic systems”. In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065)*. Vol. 4. 2000, pp. 3077–3082.
- [5] Kao-Shing Hwang et al. “Rapid prototyping platform for robotics applications”. In: *IEEE Transactions on Education* 54.2 (2010), pp. 236–246.
- [6] Shunki Shibuya et al. “Seamless Rapid Prototyping with Docker Container for Mobile Robot Development”. In: *2022 61st Annual Conference of the Society of Instrument and Control Engineers (SICE)*. 2022, pp. 1063–1068. doi: 10.23919/SICE56594.2022.9905781.
- [7] João-Vitor Zacchi, Edoardo Clementi, and Núria Mata. “APIKS: A Modular ROS2 Framework for Rapid Prototyping and Validation of Automated Driving Systems”. In: *arXiv preprint arXiv:2502.20507* (2025).
- [8] Ricardo Caldas et al. “Runtime Verification and Field-based Testing for ROS-based Robotic Systems”. In: *IEEE Transactions on Software Engineering* (2024).
- [9] Pablo Ghiglino and Guillermo Sarabia. “The ring-buffer ROS2 executor: a novel approach for real-time ROS2 Space applications”. In: *2023 IEEE Space Computing Conference (SCC)*. 2023, pp. 80–85.
- [10] Ana Luísa Ramos, José Vasconcelos Ferreira, and Jaume Barceló. “Model-based systems engineering: An emerging approach for modern systems”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42.1 (2011), pp. 101–111.

- [11] Venkat Venkatasubramanian et al. "A review of process fault detection and diagnosis: Part I: Quantitative model-based methods". In: *Computers & chemical engineering* 27.3 (2003), pp. 293–311.
- [12] Johannes Georg Kohl. "Using data stream mining techniques to localize and isolate faults in the ROS2 network". 2025. unpublished.
- [13] Christian Brecher et al. "Gaining IIoT insights by leveraging ontology-based modelling of raw data and digital shadows". In: *2021 4th IEEE international conference on industrial cyber-physical systems (ICPS)*. 2021, pp. 231–236.
- [14] Azad M Madni and Michael Sievers. "Model-based systems engineering: Motivation, current status, and research opportunities". In: *Systems Engineering* 21.3 (2018), pp. 172–190.
- [15] Sanford Friedenthal, Alan Moore, and Rick Steiner. *A practical guide to SysML: the systems modeling language*. 3. Morgan Kaufmann, 2014. ISBN: 9780128008003.
- [16] Azad M Madni and Shatad Purohit. "Augmenting MBSE with Digital Twin Technology: Implementation, Analysis, Preliminary Results, and Findings". In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 2021, pp. 2340–2346.
- [17] Azad M Madni and Shatad Purohit. "Economic analysis of model-based systems engineering". In: *Systems* 7.1 (2019), p. 12.
- [18] Aditya Akundi and Viviana Lopez. "A review on application of model based systems engineering to manufacturing and production engineering systems". In: *Procedia Computer Science* 185 (2021), pp. 101–108.
- [19] Matthew Hause et al. "The SysML modelling language". In: *Fifteenth European systems engineering conference*. Vol. 9. 2006, pp. 1–12.
- [20] Juan Liu et al. "Construction method of shop-floor digital twin based on MBSE". In: *Journal of Manufacturing Systems* 60 (2021), pp. 93–118.
- [21] Michael Grieves and John Vickers. "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems". In: *Transdisciplinary perspectives on complex systems: New findings and approaches* (2017), pp. 85–113.
- [22] Werner Kritzinger et al. "Digital Twin in manufacturing: A categorical literature review and classification". In: *Ifac-PapersOnline* 51.11 (2018), pp. 1016–1022.
- [23] Fei Tao et al. "Digital twin modeling". In: *Journal of Manufacturing Systems* 64 (2022), pp. 372–389.
- [24] Antar Mazumder et al. "Towards next generation digital twin in robotics: Trends, scopes, challenges, and future". In: *Heliyon* 9.2 (2023).
- [25] Xi Zhang et al. "An effective MBSE approach for constructing industrial robot digital twin system". In: *Robotics and Computer-Integrated Manufacturing* 80 (2023), p. 102455. ISSN: 0736-5845. DOI: <https://doi.org/10.1016/j.rcim.2022.102455>. URL: <https://www.sciencedirect.com/science/article/pii/S0736584522001375>.

-
- [26] Sebastian Kropatschek et al. “Designing a Digital Shadow for Efficient, Low-Delay Analysis of Production Quality Risk”. In: *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2022, pp. 1–8. doi: 10.1109/ETFA52439.2022.9921582.
 - [27] Gianmarco Pisanelli et al. “ROSIE: A ROS Adapter for a Modular Digital Twinning Framework”. In: *2022 31st IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*. 2022, pp. 1297–1304. doi: 10.1109/RO-MAN53752.2022.9900842.
 - [28] Pascal Bibow et al. “Model-driven development of a digital twin for injection molding”. In: *International Conference on Advanced Information Systems Engineering*. 2020, pp. 85–100.
 - [29] Christian Brecher et al. “An ecosystem for digital shadows in manufacturing”. In: *Procedia CIRP* 104 (2021), pp. 833–838.
 - [30] Fabian Becker et al. “A conceptual model for digital shadows in industry and its application”. In: *Conceptual modelling: 40th International Conference, ER 2021, Virtual Event, October 18–21, 2021, Proceedings 40*. 2021, pp. 271–281.
 - [31] Günther Schuh et al. “Building Digital Shadows for Production Control”. In: *IFIP International Conference on Advances in Production Management Systems*. 2022, pp. 110–117.
 - [32] Jeff Huang et al. “ROSRV: Runtime verification for robots”. In: *Runtime Verification: 5th International Conference, RV 2014, Toronto, ON, Canada, September 22-25, 2014. Proceedings 5*. 2014, pp. 247–254.
 - [33] Angelo Ferrando et al. “ROSMonitoring: a runtime verification framework for ROS”. In: *Towards Autonomous Robotic Systems: 21st Annual Conference, TAROS 2020, Nottingham, UK, September 16, 2020, Proceedings 21*. 2020, pp. 387–399.
 - [34] Maryam Ghaffari Saadat et al. “ROSMonitoring 2.0: Extending ROS Runtime Verification to Services and Ordered Topics”. In: *arXiv preprint arXiv:2411.14367* (2024).
 - [35] Tobias Betz et al. “How Fast is My Software? Latency Evaluation for a ROS 2 Autonomous Driving Software”. In: *2023 IEEE Intelligent Vehicles Symposium (IV)*. 2023, pp. 1–6. doi: 10.1109/IV55152.2023.10186585.
 - [36] Yuya Maruyama, Shinpei Kato, and Takuya Azumi. “Exploring the performance of ROS2”. In: *Proceedings of the 13th international conference on embedded software*. 2016, pp. 1–10.
 - [37] Harun Teper et al. “End-to-end timing analysis and optimization of multi-executor ROS 2 systems”. In: *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2024, pp. 212–224.
 - [38] Harun Teper et al. “End-to-end timing analysis in ROS2”. In: *2022 IEEE Real-Time Systems Symposium (RTSS)*. 2022, pp. 53–65.
-

- [39] Christophe Bédard, Ingo Lütkebohle, and Michel Dagenais. “ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2”. In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 6511–6518. doi: 10.1109/LRA.2022.3174346.
- [40] ROS 2. *Design of ROS 2 Tracing*. Accessed: 2025-03-18.
- [41] Christophe Bédard et al. “Message flow analysis with complex causal links for distributed ROS 2 systems”. In: *Robotics and Autonomous Systems* 161 (2023), p. 104361. ISSN: 0921-8890. doi: <https://doi.org/10.1016/j.robot.2022.104361>. URL: <https://www.sciencedirect.com/science/article/pii/S0921889022002500>.
- [42] IRobot. *ros2-performance*. Accessed: 2025-03-18.
- [43] ROS. *diagnostics*. Accessed: 2025-03-18. URL: <https://github.com/ros/diagnostics>.
- [44] Tomasz Winiarski. “Meros: Sysml-based metamodel for ros-based systems”. In: *IEEE Access* 11 (2023), pp. 82802–82815.
- [45] Pranav Srinivas Kumar et al. “ROSMOD: a toolsuite for modeling, generating, deploying, and managing distributed real-time component-based software using ROS”. In: *Electronics* 5.3 (2016), p. 53.
- [46] Matheus Ladeira, Yassine Ouhammou, and Emmanuel Grolleau. “RoBMEX: ROS-based modelling framework for end-users and experts”. In: *Journal of Systems Architecture* 117 (2021), p. 102089.
- [47] Steven Macenski et al. “Robot operating system 2: Design, architecture, and uses in the wild”. In: *Science robotics* 7.66 (2022), eabm6074.
- [48] William Woodall. *Topic and Service name mapping to DDS*. Accessed: 2025-05-07. 2016. URL: https://design.ros2.org/articles/topic_and_service_names.html.
- [49] Open Robotics. *ROS2 Node Lifecycle Design*. Accessed: 2025-03-19. 2024. URL: https://design.ros2.org/articles/node_lifecycle.html.
- [50] Open Source Robotics Foundation. *Services*. Accessed: 2025-05-02. 2025.
- [51] Open Source Robotics Foundation. *Actions*. Accessed: 2025-04-16. 2019.
- [52] Open Robotics. *ROS2 Executors*. Accessed: 2025-03-17. 2025. URL: <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Executors.html>.
- [53] Lukas Johannes Dust et al. “Experimental evaluation of callback behavior in ROS 2 executors”. In: *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2023, pp. 1–8.
- [54] Open Source Robotics Foundation. *Launch*. Accessed: 2025-04-16. 2025.

-
- [55] Johannes Kohl et al. *Dynamic System Model Generation for Online Fault Detection and Diagnosis of Robotic Systems*. 2025. arXiv: 2507.01550 [cs.RO]. URL: <https://arxiv.org/abs/2507.01550>.
 - [56] Claude Berge. “Les problemes de coloration en théorie des graphes”. In: *Annales de l'ISUP*. Vol. 9. 2. 1960, pp. 123–160.
 - [57] John B Kelly and LM Kelly. “Paths and circuits in critical graphs”. In: *American Journal of Mathematics* 76.4 (1954), pp. 786–792.
 - [58] Open Source Robotics Foundation. *Launch*. Accessed: 2025-05-08. 2025.
 - [59] EfficiOS Inc. *LTTng Documentation (v2.13)*. Accessed: 2025-06-16. 2023. URL: <https://lttng.org/docs/v2.13/>.
 - [60] Mathieu Desnoyers and Michel R Dagenais. “The lttng tracer: A low impact performance and behavior monitor for gnu/linux”. In: *OLS (Ottawa Linux Symposium)*. Vol. 2006. Cite-seer. 2006, pp. 209–224.
 - [61] EfficiOS Inc. *Babeltrace 2 Documentation*. Accessed: 2025-06-16. 2023. URL: <https://babeltrace.org/docs/v2.0/libbabeltrace2/>.
 - [62] Neo4j Inc. *Neo4j: The World's Leading Graph Database*. Accessed: 2025-06-16. 2025. URL: <https://neo4j.com/>.
 - [63] Diogo Fernandes, Jorge Bernardino, et al. “Graph Databases Comparison: AllegroGraph, ArangoDB, InfiniteGraph, Neo4J, and OrientDB”. In: *Data* 18 (2018), pp. 373–380.
 - [64] InfluxData Inc. *InfluxData: The Time Series Data Platform*. Accessed: 2025-06-16. 2025. URL: <https://www.influxdata.com/>.
 - [65] Bonil Shah, Pokhar Mal Jat, and Kalyan Sashidhar. “Performance study of time series databases”. In: *arXiv preprint arXiv:2208.13982* (2022).
 - [66] Open Source Robotics Foundation. *Writing an Action Server and Client (C++)*. Accessed: 2025-06-22. 2024.
 - [67] Steve Macenski et al. “Impact of ROS 2 node composition in robotic systems”. In: *IEEE Robotics and Automation Letters* 8.7 (2023), pp. 3996–4003.
 - [68] Google. *cAdvisor - Analyzes resource usage and performance characteristics of running containers*. Accessed: 2025-06-18. URL: <https://github.com/google/cadvisor>.
 - [69] NCabatoff. *process-exporter - Prometheus exporter that mines /proc to report on selected processes*. Accessed: 2025-06-18. URL: <https://github.com/ncabatoff/process-exporter>.
 - [70] Docker, Inc. *Docker: Accelerated Container Application Development*. Accessed: 2025-06-22. 2025.
 - [71] Patrick E McKnight and Julius Najab. “Mann-Whitney U Test”. In: *The Corsini encyclopedia of psychology* (2010), pp. 1–1.
-

- [72] Husarion. *Lynx - compact rugged Unmanned Ground Vehicle*. 2025. URL: <https://store.husarion.com/products/lynx> (visited on 07/02/2025).
- [73] Stereolabs. *ZED 2i Stereo Camera - neural depth, motion sensing, IP66*. 2025. URL: <https://www.stereolabs.com/en-de/store/products/zed-2i> (visited on 07/02/2025).
- [74] Fixposition. *Vision-RTK 2 Starter Kit - GNSS + vision sensor suite*. 2025. URL: <https://www.fixposition.com/products/vision-rtk2-starter-kit> (visited on 07/02/2025).
- [75] Neosys Technology. *Nuvo-9160GC - Intel 12th/13th/14th-Gen rugged edge AI computer*. 2025. URL: <https://www.neosys-tech.com/en/product/product-lines/edge-ai-gpu-computing/nuvo-9160gc-intel-12th-gpu-computing-platform> (visited on 07/02/2025).
- [76] YARP. *YARP: Yet Another Robot Platform*. Accessed: 2025-07-02. 2025.
- [77] PX4 Autopilot Project. *PX4: Open-Source Autopilot Software*. Accessed: 2025-07-02. 2025.

A. Further Tree Structures

Container-Driven Tree

The concept of Containers, as described in Section 4.3 and illustrated in Figure 4.8, organises Nodes into groups and thus creates a set of trees, whereas the root relates to the Container and the leaves to the Nodes. The only inner Member¹ of each tree refers to its root, representing the Container itself. As visualised in Figure A.1, the set of trees is joined by employing a passive Member as virtual organising entity and thus passive Member that connects to each root of the subtree to create a superior tree. It is imperative to mention that not every Node has to be part of a Container and thus not every Node has to be part of this tree.

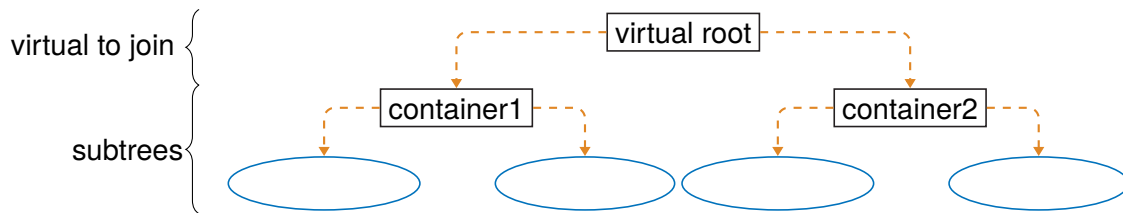


Figure A.1.: Exemplary and Joined Container-Driven Tree ❖

Process-Driven Tree

Based on the model that represents processes and their structure is described in Section 4.6 and illustrated in Figure 4.10. The process-driven tree is formed by traversing the tree structure of the processes in the opposite direction to their creation up to the initial direct start or launch (via a launch script described in Section 4.6). This creates a set of trees consisting of active Members only. To create a superior tree, the roots of all trees of the set are joined by a virtual organisational entity and thus passive Member. Each Node is assigned to a single process and thus part of this tree, but a process may encapsulate multiple Nodes (concept of Node Composition, see Section 4.3). Furthermore, as each Node is enabled to start further processes, those processes are also representable within the created tree. Figure A.2 illustrates an exemplary process tree. Within the example, all Nodes were started by a single launch script. Through the Utilisation of

¹inner Member: Analogous to a classic „inner node“ but called „inner Member“ to avoid confusion.

passive Members as organisational entities, the mathematical meta-model enables the creation of a superior tree that might even spread across multiple operating systems. E.g. a ROS2 system that is spread across multiple computers.

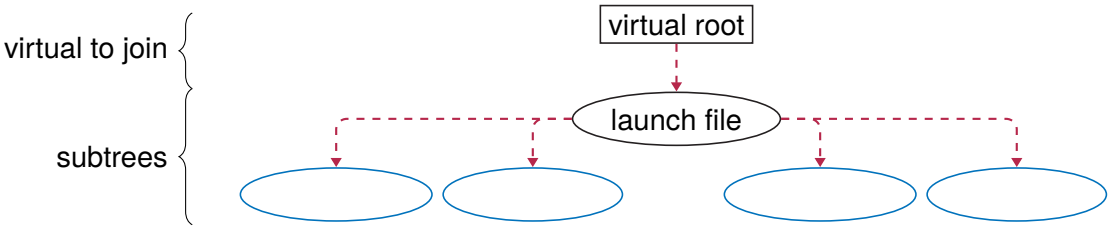


Figure A.2.: Exemplary Process-Driven Tree (of the ROS2 System) ❖

B. Further Setup Information

Table B.1.: Tracepoints Employed for the Proof-of-Concept [40]

Tracepoint	Purpose
ros2:rcl_node_init	Indicates the instantiation of a ROS2 Node.
ros2:rcl_publisher_init	Indicates the instantiation of a ROS2 publisher.
ros2:rcl_subscription_init	Indicates the instantiation of a ROS2 subscriber.
ros2:rcl_service_init	Indicates the instantiation of a ROS2 Service server.
ros2:rcl_client_init	Indicates the instantiation of a ROS2 Service client.
ros2:rcl_timer_init	Indicates the instantiation of a ROS2 Timer.
ros2:rclcpp_timer_link_node	Links an initialised Timer to its respective Node.
ros2:rcl_lifecycle_state_machine_init	Indicates the instantiation of a ROS2 state machine that is necessitated by managed Nodes.
ros2:rcl_lifecycle_transition	Indicates a lifecycle transition of managed Nodes.
ros2:rcl_publish	Indicates the publishing of a message.

C. Performance Test: Detailed Results

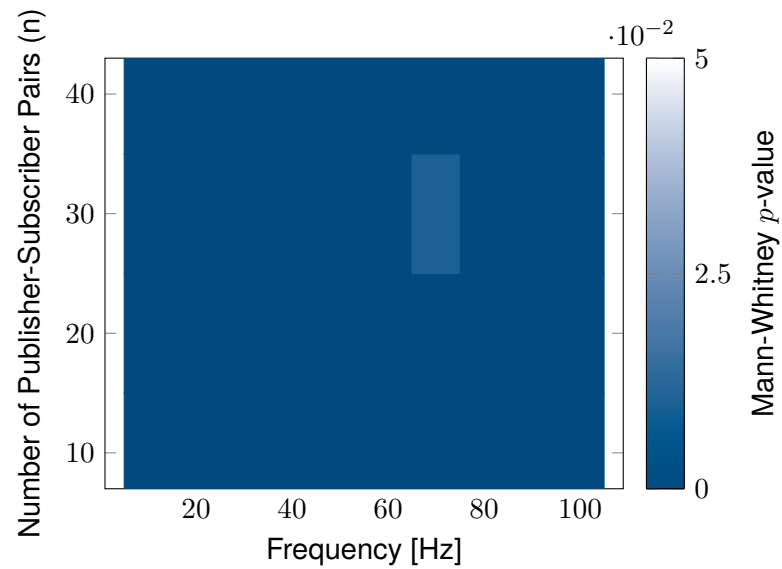


Figure C.1.: Mann-Whitney U Test Significance Results for Measured Latencies (Results below 0.05 are Considered Statistically Significant)

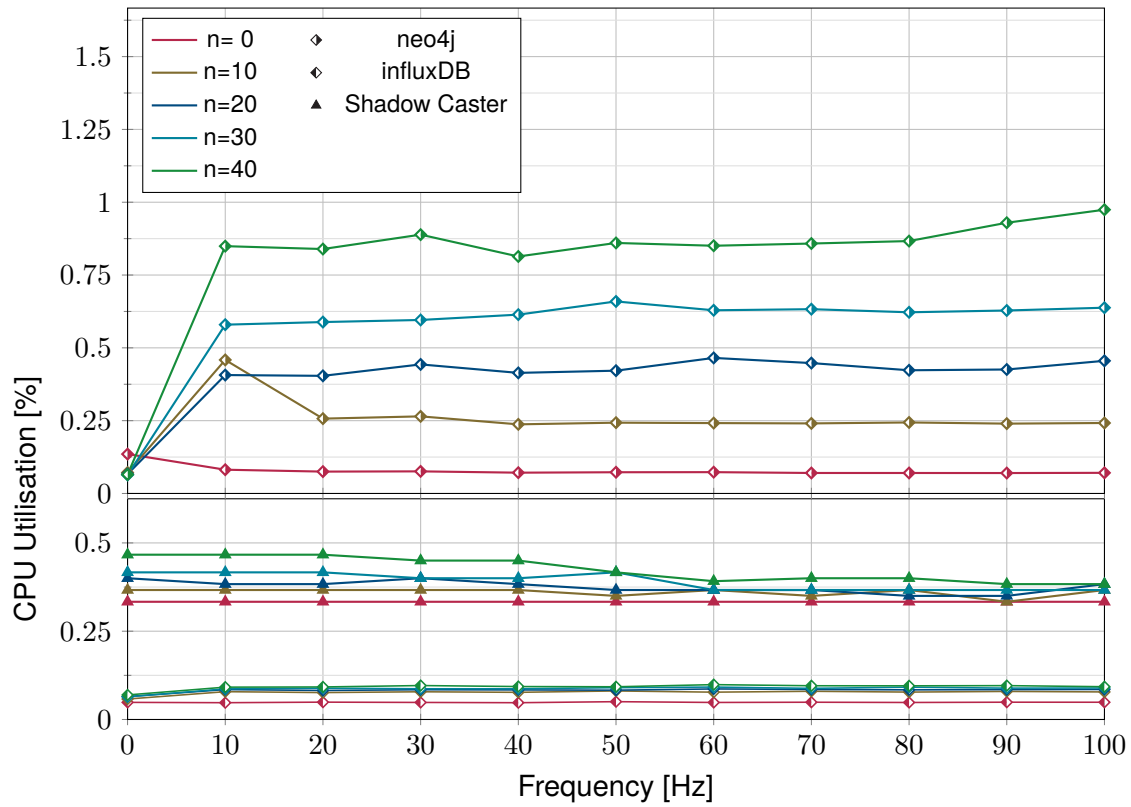


Figure C.2.: CPU Utilisation of Shadow Caster and Shadow Instance

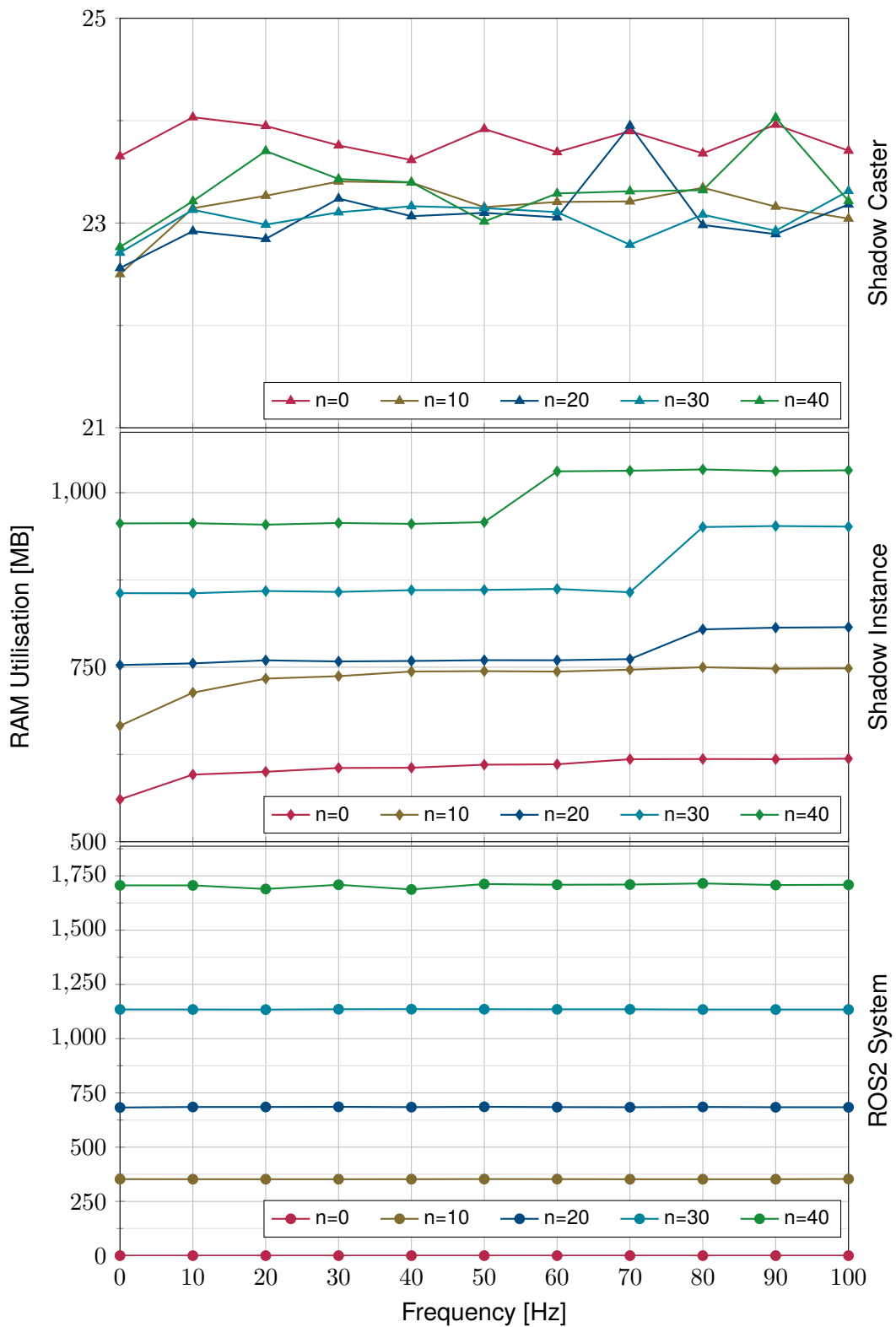


Figure C.3.: Detailed RAM Utilisation of Shadow Caster, Shadow Instance, and ROS2 system

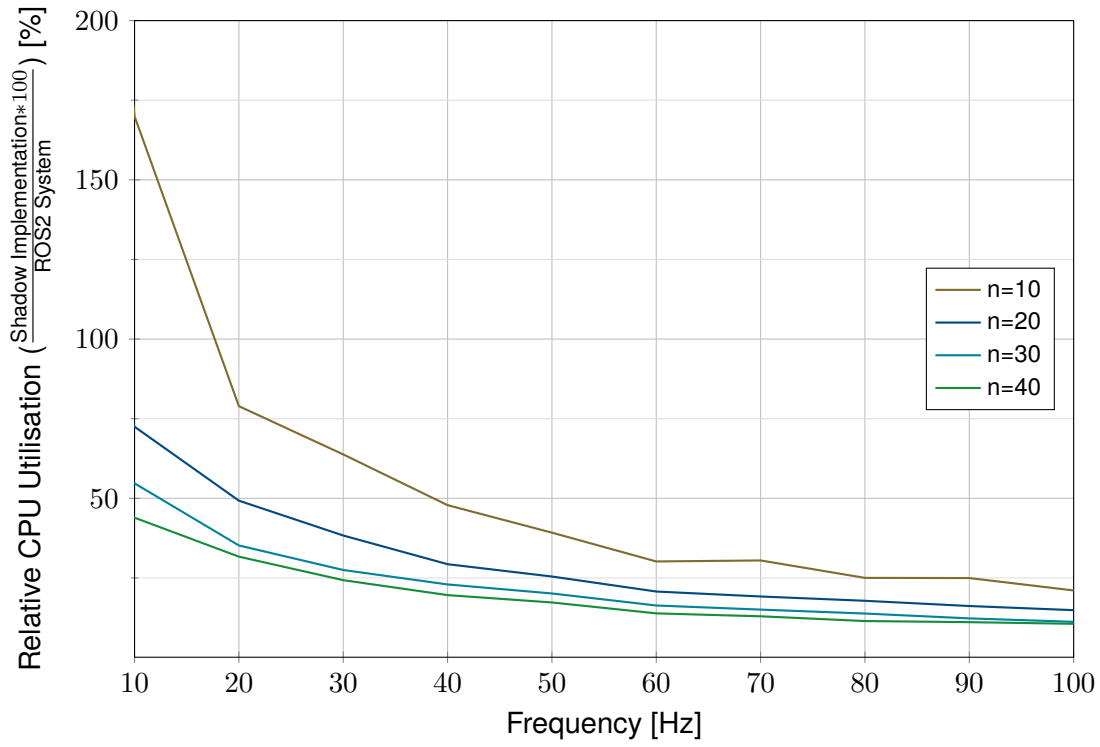


Figure C.4.: Relative CPU Utilisation of Shadow Implementation

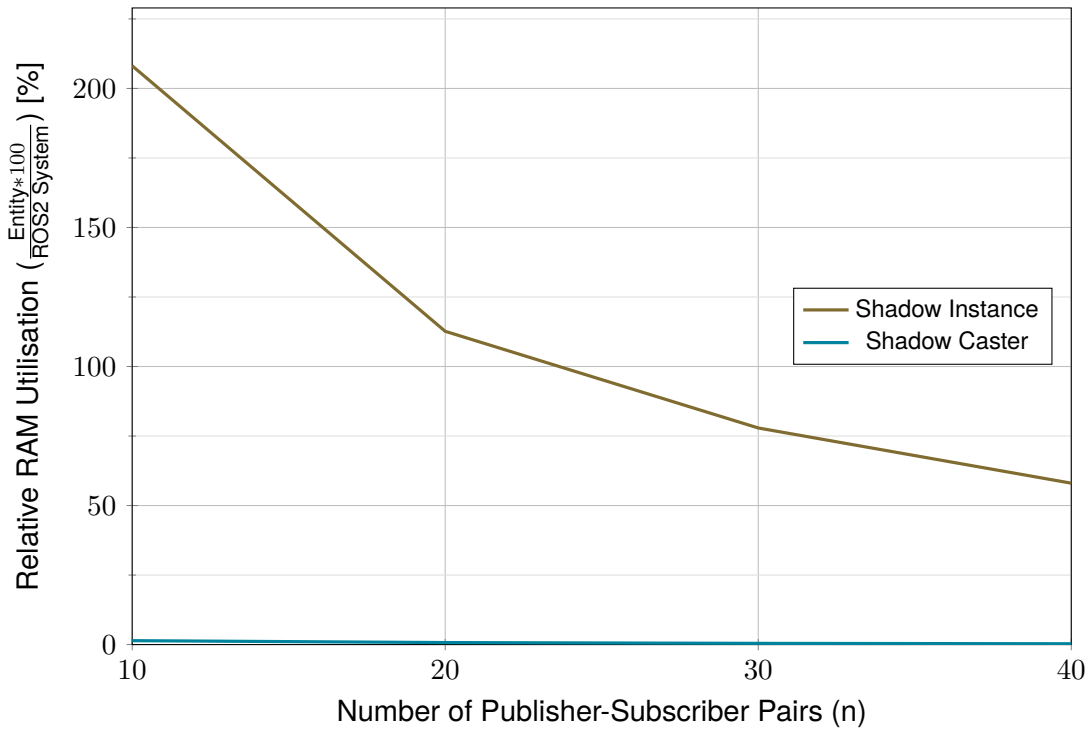


Figure C.5.: Relative RAM Utilisation of Shadow Caster and Shadow Instance

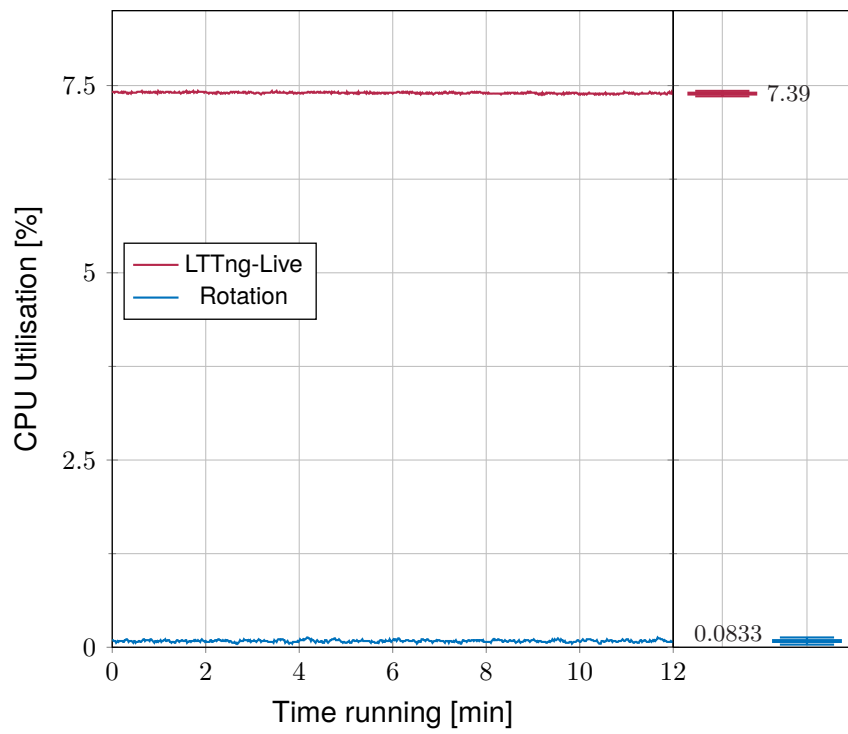


Figure C.6.: CPU Utilisation for Differing Tracer Implementations

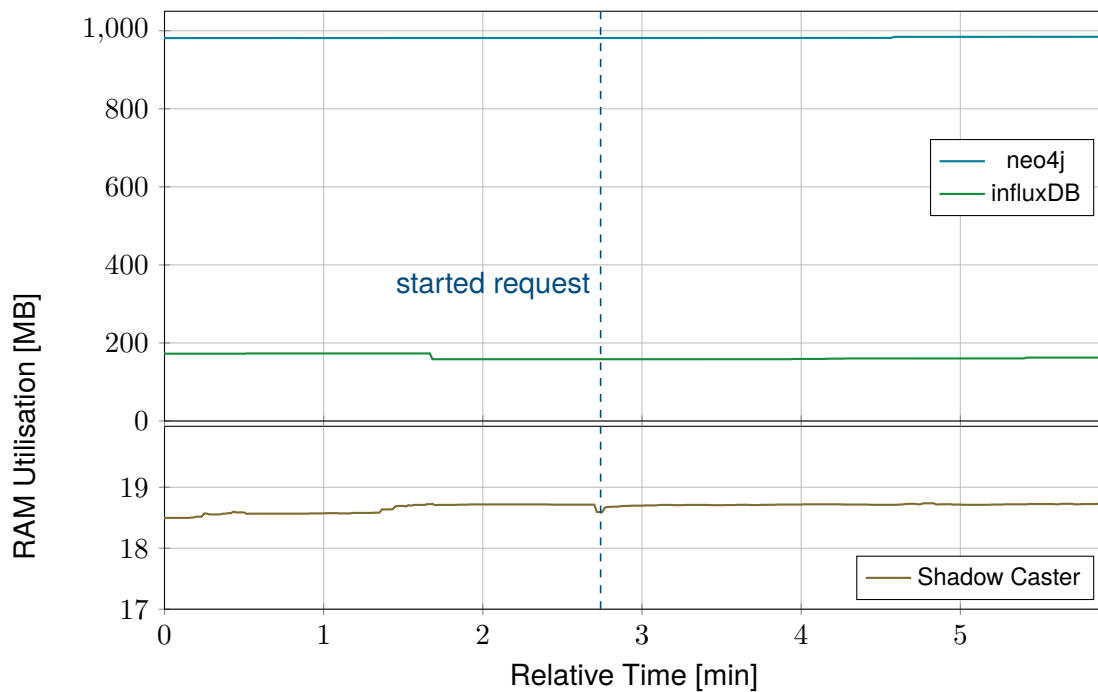


Figure C.7.: Influence of Requests on the RAM Utilisation of the Shadow Implementation

D. Real World Scenario

Table D.1.: List of Active Nodes in Stage 3 of the Real-World Scenario

/emergency_stop	/lynx/system_monitor
/fixposition_driver_ros2	/twist_mux
/foxglove_bridge	/velocity_smoother
/foxglove_bridge_component_manager	/wrapper/fixposition_wrapper_node
/joy_teleop/joy_node	/wrapper/livox_front_wrapper_node
/joy_teleop/teleop_twist_joy_node	/wrapper/sick_front_wrapper_node
/launch_ros_1	/wrapper/sick_rear_wrapper_node
/launch_ros_143238	/zed2i_front/transform_listener_impl...
/launch_ros_82636	/zed2i_front/zed2i_front_state_publisher
/lifecycle_manager_rosee	/zed2i_front/zed_container
/lynx/battery_driver	/zed2i_front/zed_node_front
/lynx/controller_manager	/zed2i_left/transform_listener_impl...
/lynx/drive_controller	/zed2i_left/zed2i_left_state_publisher
/lynx/hardware_controller	/zed2i_left/zed_container
/lynx/imu_broadcaster	/zed2i_left/zed_node_left
/lynx/joint_state_broadcaster	/zed2i_rear/transform_listener_impl...
/lynx/joy2twist_node	/zed2i_rear/zed2i_rear_state_publisher
/lynx/joy_node	/zed2i_rear/zed_container
/lynx/lights_container	/zed2i_rear/zed_node_rear
/lynx/lights_controller	/zed2i_right/transform_listener_impl...
/lynx/lights_driver	/zed2i_right/zed2i_right_state_publisher
/lynx/lights_manager	/zed2i_right/zed_container
/lynx/robot_state_publisher	/zed2i_right/zed_node_right
/lynx/safety_manager	

It is imperative to note, that only those entities are represented within the Graph, that are executed by the monitored host. Extending the Shadow Implementation to multi-host is addressed in Future Work.

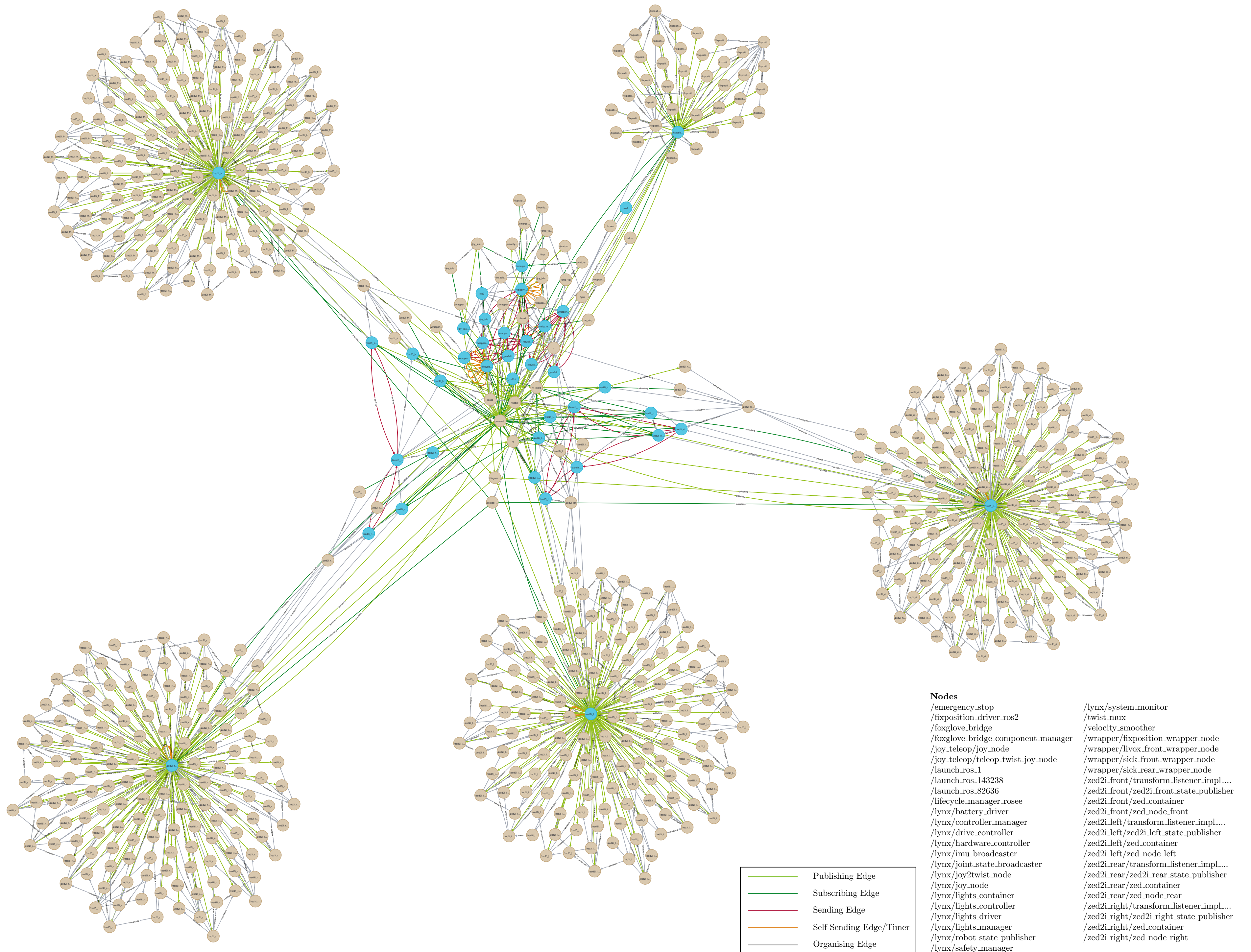


Figure D.1.: Real-World Scenario: Generated ROS2 Graph (Stage 3) 

Declaration of Authorship

I declare, that I have authored this work on my own under the guidance of my supervisors. I have not used any other tools than the declared. Parts of the work, which are taken from other works, if literally or in sense, I have cited correctly under specification of the source in each single case. This declaration also applies on images and figures. I declare, that this work satisfies the principles of academic integrity. I declare, that this work has not been presented to any other examination board, nor it will be.

Freiberg, 07.07.2025



Georg Muck

